Resumen de Discreta II

Maximiliano Illbele

16 de agosto de 2012

Índice

1. Coloreo y repaso de grafos **4**
1.0.1. BFS-DFS . . . 5
1.1. Coloreo . . . 6
1.1.1. Número Cromático . . . 6
1.1.2. Algoritmo "Greedy" para coloreo . . . 7
1.1.3. Propiedades de Greedy . . . 8
1.1.4. Heurísticas . . . 10

2. Flujos en networks **12**
2.1. Algoritmo de Greedy para hallar Flujo . . . 14
2.1.1. Ejemplos de Greedy . . . 14
2.1.2. Complejidad de Greedy . . . 15
2.2. Algoritmo de Ford y Fulkerson . . . 15
2.2.1. Lema del Camino Aumentante . . . 16
2.3. Teorema: Max-Flow Min-Cut . . . 20
2.3.1. Complejidad de Ford Fulkerson . . . 21
2.4. Algoritmo de Edmonds Karp . . . 23
2.4.1. Complejidad de Edmonds-Karp . . . 24
2.4.2. Ejemplo de Edmonds Karp . . . 27
2.4.3. Pseudo-Código de Edmonds Karp . . . 28
2.5. Algoritmos Tipo Dinic . . . 29
2.5.1. Esquema general Tipo Dinic . . . 30
2.5.2. Ejemplo de Dinic . . . 32
2.5.3. Pseudo Código algoritmos Tipo Dinic . . . 33
2.5.4. Complejidad de Dinic . . . 35
2.6. Algoritmo Wave de Tarjan . . . 36
2.6.1. Pseudo Código de Wave . . . 36
2.6.2. Complejidad de Wave . . . 38
2.6.3. Ejemplos de Wave . . . 39

3. MST **41**
3.1. Kruskal . . . 42
3.1.1. Pseudo Código de Kruskal . . . 43
3.1.2. Teorema: Kruskal devuelve un MST . . . 43
3.1.3. Complejidad de Kruskal . . . 44
3.1.4. Kruskal Delete . . . 44
3.1.5. Complejidad Kruskal Delete . . . 44
3.1.6. Ejemplo de Kruskal . . . 45
3.2. PRIM . . . 46
3.2.1. Pseudo Código de Prim . . . 46
3.2.2. Correctitud de Prim . . . 46
3.2.3. Ejemplo de Prim . . . 47
3.3. Implementaciones de Union Find . . . 48
3.3.1. Union Find con listas enlazadas . . . 48
3.3.2. Union Find con Representantes . . . 48
3.3.3. Union Find con ranking . . . 49

4. Matchings **51**
4.1. Transformación del problema de Matching a uno de flujo maximal . . . 51
4.2. Teorema de Hall . . . 53
4.3. Complejidad del Algoritmo de Matching . . . 53
4.4. Grafos con Peso . . . 53
4.4.1. Algoritmo de “Gross” . . . 54
4.4.2. Ejemplo del algoritmo de “Gross” . . . 55
4.5. Prueba del teorema de HALL . . . 60
4.6. Teorema del Matrimonio . . . 61
4.7. Minimizar la Suma . . . 62
4.7.1. Ejemplo de Matching que minimiza la Suma (A ojo) . . . 62
4.7.2. Algoritmo Húngaro . . . 63
4.7.3. Ejemplo del algoritmo Húngaro . . . 63
4.7.4. Complejidad de algoritmo Hungaro . . . 65
4.7.5. EJEMPLO DEL MEJORADO (FALTA) . . . 67

5. Códigos de correción de errores **68**
5.1. Cota de Hamming . . . 71
5.2. Matriz generadora . . . 74
5.3. Matriz de chequeo . . . 75
5.4. Cota Singleton . . . 80
5.5. Códigos Cíclicos . . . 80
5.5.1. Correción de errores “Error Trapping” . . . 87

6. P-NP **90**
6.1. La clase P . . . 90
6.2. La Clase NP . . . 90
6.3. Co-NP . . . 91
6.4. SAT . . . 91

6.4.1. Reducción Polinomial 92
6.4.2. $k - color \leq_{\mathcal{P}} SAT$ 92
6.5. NP Completo 94
6.5.1. 3-SAT es NP completo 94
6.5.2. 3-color es NP completo 96
6.5.3. Ejemplo 3-SAT $\leq_P$3-color 97
6.5.4. 2-SAT $\in$ P 100
6.5.5. HORN-SAT 101

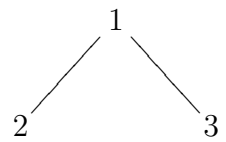
7. Introducción a la Inteligencia Artificial 102
7.1. Hill Climbing 102
7.2. Simulated Annealing 102
7.3. Algoritmos Evolutivos 102
7.4. Algoritmos Genéticos 102
7.4.1. Selección 103
7.5. Ejemplo de un AG 103
7.5.1. Codificación del ejemplo 104
7.5.2. Fitness del ejemplo 104
7.5.3. Entrecruzamiento del ejemplo 104
7.5.4. Mutación del ejemplo 104
7.5.5. Selección del ejemplo 105
7.5.6. Operadores de Selección 105
7.5.7. Remainde Methods (Métodos del “Resto”) 107
7.5.8. Otros métodos Fitness Proportional 107
7.5.9. Sigma Scaling 108
7.5.10. Crossover 109
7.5.11. Mutación 110
7.5.12. Mutaciones con Permutaciones 110
7.6. No Free Lunch 111

1. Coloreo y repaso de grafos

Recordemos que un grafo G es un par (V, E) donde $E \subseteq \{A \subseteq V : |A| = 2\}$.
Donde V es el conjunto de vértices o nodos y E es el conjunto de lados[1] o aristas.

<u>Ejemplo</u>: sea $G = \Big(\{1,2,3\}, \big\{\{1,2\},\{1,3\}\big\}\Big)$

Gráfico(G) :

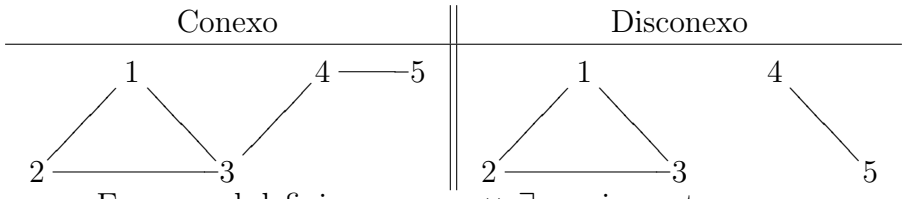


<u>Notación usual</u>

- $\{x, y\} \to xy$
- "Vecinos" de un nodo x: $\Gamma(x) = \{y \in V : xy \in E\}$
- Grado[2] o Valencia de un vértice: $d(x) = \Big|\Gamma(x)\Big|$
- Menor valencia de G: $\delta = \text{mín}\{d(x) : x \in V\}$
- Mayor valencia de G: $\Delta = \text{máx}\{d(x) : x \in V\}$

<u>Definiciones</u>

- Un grafo es **regular** si $\delta = \Delta$, a veces llamados "k-regular".
- Un **camino** es una sucesión de vértices $x_1, x_2, \ldots, x_t$ todos distintos tales que $x_i x_{i+1} \in E$.
- Un grafo es **conexo** si $\forall$par de vértices $\exists$ un camino entre ellos.



En general definimos $x \approx y \Leftrightarrow \exists$ camino entre x e y.

<u>Ejercicio</u>: demostrar que $\approx$ es una relación de equivalencia.

1. Reflexividad: $x \approx x$
2. Simetría: $x \approx y \Rightarrow y \approx x$
3. Transitividad: $\left.\begin{matrix} x \approx y \\ y \approx z \end{matrix}\right\} \Rightarrow x \approx z$

[1] Edges.
[2] Degree.

Una componente conexa de G es una clase de equivalencia de la relación $\approx$.
Notación

- La cantidad de vértices: $n = |V|$.
- La cantidad de lados: $m = |E|$.

Observación

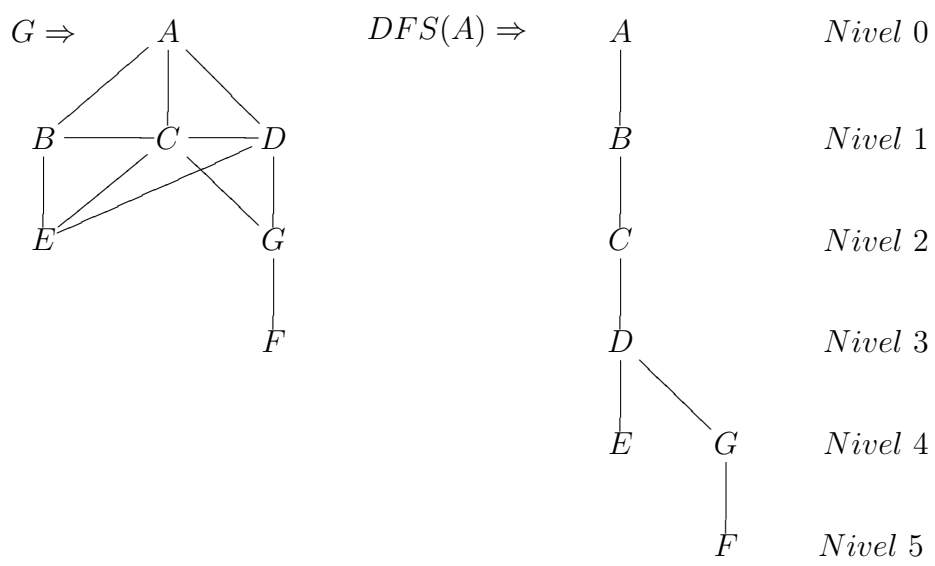
- $m \leq \binom{n}{2} = \frac{n*(n-1)}{2}$
- Si G es conexo $\Rightarrow n - 1 \leq m$

1.0.1. BFS-DFS

DFS(x)-Depth First Search[3]

Su funcionamiento consiste en ir expandiendo todos y cada uno de los nodos que va localizando, de forma recurrente, en un camino concreto. Cuando ya no quedan más nodos que visitar en dicho camino, sube en el árbol, de modo que repite el mismo proceso con cada uno de los hermanos del nodo ya procesado.

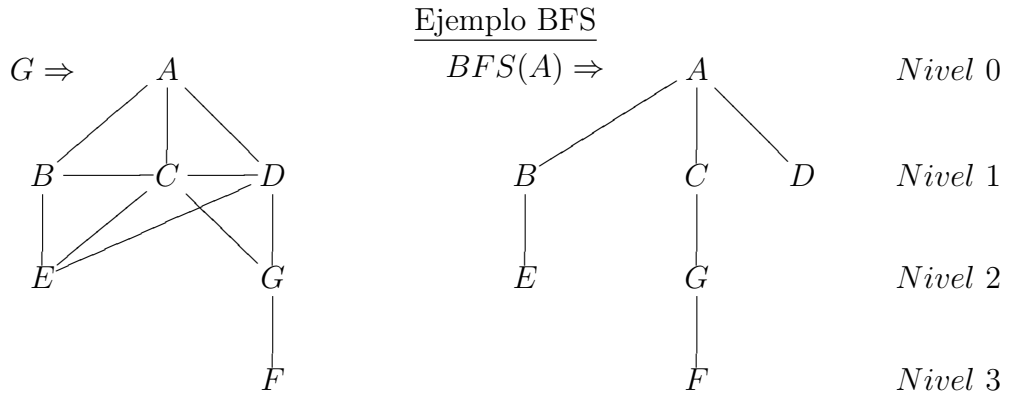
Ejemplo DFS



[3]DFS: Búsqueda en profundidad.

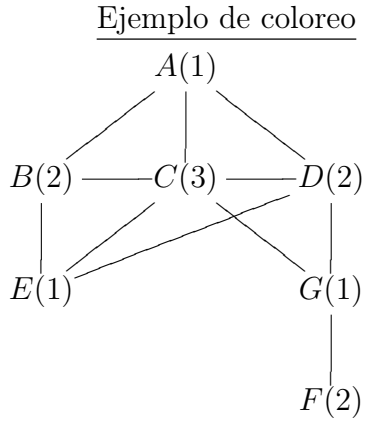
BFS(x) - Breadth First Search[4]

Intuitivamente, se comienza en la raíz x y se exploran todos sus vecinos. A continuación para cada uno de los vecinos se exploran sus respectivos vecinos adyacentes, y así hasta que se recorra todo el Grafo.



1.1. Coloreo

Un coloreo (propio) de $G = (V, E)$ con k colores es una función $C : V \to \{1, 2, \ldots, k\}$ tal que si: $xy \in E \Rightarrow C(x) \neq C(y)$



1.1.1. Número Cromático

En general deseamos calcular el número cromático de $G : \chi(G)$.
Donde $\chi(G) = \text{mín}\{k : \exists \text{ un coloreo propio de } G \text{ con } k \text{ colores }\}$.

Propiedad: $1 \leq \chi(G) \leq n$

[4]BFS: búsqueda en anchura.

1.1.2. Algoritmo "Greedy" para coloreo

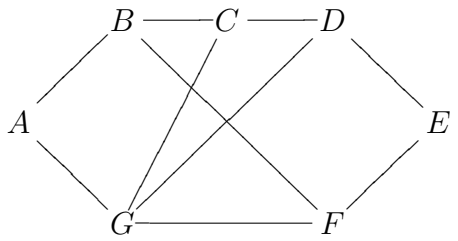
Requiere un <u>orden</u> de los vértices: $x_1, \ldots, x_n$.
Inicio: $C(x_1) = 1$
Para $j > 1$

$$C(x_j) = \text{ El menor color que no crea conflicto con los vértices ya coloreados.}$$

$$C(x_j) = \text{mín}\Big\{k : C(x_i) \neq k \ \forall i < j : x_i \in \Gamma(x_j)\Big\}$$

<u>Invariante</u>: en todo momento el coloreo es propio, por lo tanto al terminar da un coloreo propio.

<u>Ejemplo</u>:



Orden alfabético								Orden alternativo						
Vértices	A	B	C	D	E	F	G	G	C	D	F	B	A	E
Color	1	2	1	2	1	3	4	1	2	3	2	1	2	1

<u>Complejidad de coloreo con Greedy</u>

$$O\big(d(x_1) + d(x_2) + \ldots + d(x_n)\big) = O(2m) = O(m)$$

<u>Teorema</u>: sea $G = C_{2r+1} \Rightarrow \chi(G) = 3$

<u>Prueba</u>: Supongamos que $\chi(G) = 2$
Sea A el color del primer vértice.
Sea B el color del segundo vértice.

$$x_1x_2 \in E \Rightarrow A \neq B$$

Como estamos suponiendo $\chi(G) = 2$ estos son los únicos colores que podemos usar.
Como $x_2x_3 \in E \Rightarrow C(x_3) \neq B \Rightarrow C(x_3) = A$
Como[5] $x_3x_4 \in E \Rightarrow C(x_4) = B$
Por inducción:

$$C(x_i) = \begin{cases} A & \text{cuando } i \text{ es Impar} \\ B & \text{cuando } i \text{ es Par} \end{cases}$$

Pero entonces el lado $\underbrace{x_1}_{A}\underbrace{x_{2r+1}}_{A}$ tiene los 2 vértices del mismo color **Absurdo**.

[5]Este paso es sólo ilustrativo no forma parte de una demostración formal ya que no sabemos si $n > 3$ ó $n = 3$.

$$\therefore \chi(G) \geq 3$$

Y está claro que con 3 colores alcanza definimos:

$$C(x_i) = \begin{cases} A & \text{cuando } i \text{ es Impar } < 2r+1 \\ B & \text{cuando } i \text{ es Par} \\ C & \text{cuando } i = 2r+1 \end{cases}$$

■

<u>Corolario</u>: si $C_{2r+1} \subseteq G \Rightarrow \chi(G) \geq 3$
<u>Definición</u>: un grafo G es **bipartito** si: $\chi(G) = 2$

<u>Teorema</u>: G es bipartito $\Leftrightarrow \nexists C_{2r+1} \subseteq G$

Más aún existe un algoritmo de complejidad polinomial para determinar si G es bipartito o no.

<u>Prueba</u>: el algoritmo que daremos da un coloreo (propio) con dos colores, si $\chi(G) = 2$, y si el algoritmo colorea con más de 2 colores entonces veremos que existe $C_{2r+1} \subseteq G \ \therefore \chi(G) \geq 3$

En realidad esto lo haremos para cada componente conexa de G.

Vamos a suponer G conexo.

Sea $x \in V$, y corramos $BFS(x)$ y tomemos el siguiente coloreo que puede no ser propio:

$$C(z) = \begin{cases} 0 & \text{Nivel}\Big(BFS(x), z\Big) \text{ es Par} \\ 1 & \text{Nivel}\Big(BFS(x), z\Big) \text{ es Impar} \end{cases}$$

Si ese coloreo es propio ya está.

Supongamos que no $\Rightarrow \exists v, w : vw \in E : C(v) = C(w)$

Como $vw \in E$ y estamos usando BFS, Si v fue puesto antes que w en el árbol entonces debe ser:

$$nivel(w) = nivel(v+1) \Rightarrow \textbf{Absurdo} \text{ pues } nivel(v) \equiv nivel(w) \ (2)$$

O bien $nivel(v) = nivel(w) \ \therefore$ Hay un ciclo impar $\Rightarrow \chi(G) \geq 3$ ■

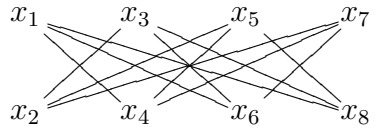
1.1.3. Propiedades de Greedy

Analizando a Greedy, podemos decir que en ciertos casos puede andar muy mal

<u>Ejemplo</u>:

$$\text{Sea } G : V = \{1, \ldots, 2r\} \text{ y con } E = ij \begin{cases} i & es\ Impar \\ j & es\ Par \end{cases} \quad \text{y } j \neq i+1$$

Para simplificar veamos el grafo correspondiente a $r = 4$



Luego Greedy corriendo en orden $x_1, \ldots, x_{2r} = x_8$ va a necesitar $r = 4$ colores:

Color	1	2	3	4
Vértice	x_1	x_3	x_5	x_7
	x_2	x_4	x_6	x_8

Pero si definimos $C(x_i) = \begin{cases} 1 & i \text{ es impar} \\ 2 & i \text{ es par} \end{cases}$ nos queda que $\chi(G) = 2$

<u>Conclusión</u>:

$Greedy(G)$ devuelve un coloreo que usa r colores, pero vimos que $\chi(G) = 2$.

$\boxed{\textbf{Sin embargo}}$ ...

<u>Propiedad</u>: $\forall G\ \exists$ un ordenamiento $\{x_1, \ldots, x_n\}$ de los vértices de G en el cual Greedy() obtiene: $\chi(G)$

<u>Prueba</u>: sea $t = \chi(G)$

$$\left.\begin{array}{c} \text{Sean } V_1 \text{ los vértices de color } 1 \\ \text{Sean } V_2 \text{ los vértices de color } 2 \\ \vdots \\ \text{Sean } V_t \text{ los vértices de color } t \end{array}\right\} \text{ en algún coloreo (propio) con } t \text{ colores.}$$

<u>Inducción en t</u>:

Si $t = 1$ obvio.

Supongamos que vale para $t - 1$

<u>Orden de los vértices</u>:

$$\text{Los de } V_1 < \text{ los de } V_2 < \ldots < \text{ los de } V_t \text{ , en } V_i \text{ los ordeno como quiero.}$$

Por hipótesis inductiva hasta V_{t-1} lo coloreamos con $t - 1$ colores.

Los de V_t **no** están unidos entre si $\Rightarrow$ Greedy en cada uno de sus vértices usará uno de los primeros $t - 1$ colores o el color t, nunca hay conflicto entre ellos que demande un color $t + 1$. ■

<u>Propiedad</u>: $\chi(G) \leq \Delta + 1$

<u>Prueba</u>: usar Greedy en cualquier orden.

Cuando vayamos a colorear x_i lo peor que puede pasar es que todos los vecinos ya estén coloreados y con colores distintos, esto forzaría a Greedy() a usar un color distinto a esos $d(x_i)$ de esos colores.

Pero $d(x_i) \leq \Delta \ \therefore$ Greedy con $\Delta + 1$ colores se las arregla.

■

Más aún, si G es conexo o todas sus componentes conexas sean no regulares, tenemos que:

$$\delta \neq \Delta \Rightarrow \chi(G) \leq \Delta$$

<u>Prueba</u>: sea $x \in V : d(x) = \delta$

Corramos $BFS(x)$[6] y sea $x_1, \ldots, x_n$ el <u>orden inverso</u> al de BFS(x) $\therefore x_n = x$

En $BFS(x)$[7] <u>todo vértice</u>, salvo x es incorporado al árbol por medio de un vecino que <u>ya está</u> en el árbol.

$\therefore$ En el orden inverso todo vértice distinto de x tiene un vecino <u>posterior</u> en el orden $(x_1, \ldots, x_n)$.

$\therefore$ Cuando Greedy va a colorear $x_i : i < n$, como x_i tiene por lo menos un vecino posterior entonces x_i tiene $\leq d(x_i) - 1$ vecinos anteriores.

$$\therefore \text{ Hay a lo sumo } d(x_i) - 1 \leq \Delta - 1 \text{ posibles “conflictos”.}$$

$\therefore$ Greedy puede colorear con alguno de los Δ colores.

Ahora queda colorear $x_n = x$

Greedy necesita usar $d(x) + 1$ colores: $d(x) = \delta < \Delta \Rightarrow \delta + 1 \leq \Delta$ ■

Ejemplos donde $\chi(G) = \Delta + 1$

1. Los ciclos impares: $\chi(C_{2r+1}) = 3 = 2 + 1 = \Delta + 1$
2. Los grafos completos: $\chi(K_n) = n = (n - 1) + 1 = \Delta + 1$

El **teorema de Brooks** dice que si G es <u>conexo</u> esos son los únicos ejemplos.

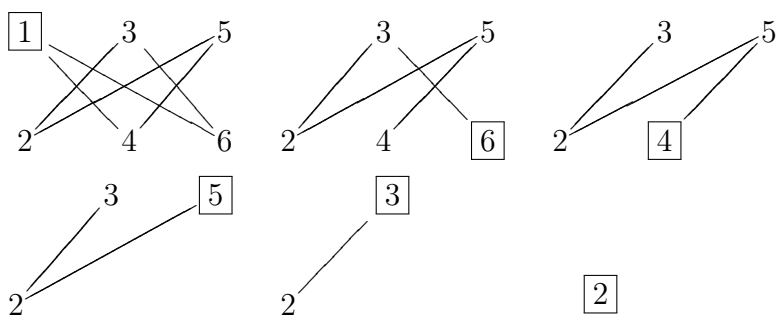
1.1.4. Heurísticas

1. Welsh-Powell

   Greedy en el orden: $d(x_1) \geq d(x_2) \geq \ldots \geq d(x_n)$

2. Matul-Marble-Isaacson

   - $x_n : d(x_n) = \delta$
   - Sea $G_{n-1} = G_n - x_n$
   - Tomo $x_{n-1} : \delta_{G_{n-1}}(x_{n-1}) = \delta(G_{n-1})$ y así seguimos ...



[6] O lo mismo $DFS(x)$.

[7] O $DFS(x)$.

Luego corro Greedy con: $x_6 = 1$ $x_5 = 6$ $x_4 = 4$ $x_3 = 5$ $x_2 = 3$ $x_1 = 2$

Es decir en el orden: $2, 3, 5, 4, 6, 1$

Y nos da $\chi(G) = 2$

3. <u>RLF</u>: Recursive Largest First

```
RLF()
{
  color = 1;
  R = V; // Vértices no Coloreados
  while(R ≠ ∅){
    L = R;
    while (L ≠ ∅){
      Tomar v de L : V tenga la mayor cantidad de vecinos en R
      Color(v) = color;
      Remover(v) de R;
      Remover(v) y Γ(v) de L;
    }
    Color ++;
  }
}
```

4. D-Satur

$$d_5(x) = \#\text{de colores de los vecinos } \underline{\text{coloreados}} \text{ de } x \ .$$

Corre Greedy con un orden dinámico eligiendo como "próximo vértice" el que tenga d_5 más grande.

2. Flujos en networks

Un network (dirigido) es un grafo dirigido, es decir un par $(V, E) : E \subseteq V * V$ con pesos en los lados, es decir con una función $C : E \to \mathbb{R}$.

En este contexto siempre pediremos $C : E \to \mathbb{R}^{\geq 0}$ y los $C(x, y)$ se llaman <u>capacidades</u>.

<u>Notación</u>:

- (x,y) $\to \vec{xy}$ (si se sobreentiende por el contexto también xy).
- $\Gamma^+(x) = \{y \in V : \vec{xy} \in E\}$.
- $\Gamma^-(x) = \{y \in V : \vec{yx} \in E\}$.
- $\Gamma(x) = \Gamma^+(x) \cup \Gamma^-(x)$.

<u>Definición</u>: dado un network $N = (V, E, C)$ y vértices distinguidos $s, t \in V$, un **flujo** en N, de s a t es una función $f : E \to \mathbb{R}$ tal que:

1. Feasability[8]: $0 \leq f(\vec{xy}) \leq C(\vec{xy})\ \forall \vec{xy} \in E$
2. Conservación: $\sum\limits_{y \in \Gamma^+(x)} f(\vec{xy}) = \sum\limits_{y \in \Gamma^-(x)} f(\vec{yx})\ \forall x \neq s, t$
3. s Productor:
$$\sum_{y \in \Gamma^+(s)} f(\vec{sy}) \geq \sum_{y \in \Gamma^-(s)} f(\vec{ys})$$

Como s produce se llama la **fuente**[9] y t se llamará **resumidero**[10].

En algunos textos se pide directamente que: $\sum\limits_{y \in \Gamma^-(s)} f(\vec{ys}) = 0$

O más aún: $C(\vec{ys}) = 0\ \forall y \in \Gamma^-(s)$

<u>Notación</u>:

- $\sum\limits_{y \in \Gamma^+(x)} f(\vec{xy}) = Out_f(x)$
- $\sum\limits_{y \in \Gamma^-(x)} f(\vec{yx}) = In_f(x)$

<u>Definición</u>: el **valor del flujo** es $V(f) = Out_f(s) - In_f(s)$

[8] Viabilidad.
[9] Source.
[10] Sink.

$$\underline{\text{Propiedad}}: V(f) = In_f(t) - Out_f(t)$$

<u>Prueba</u>:

Sea $D = In_f(t) - Out_f(t)$

$$Out_f(x) - In_f(x) = \begin{cases} 0 & x \neq s, t \\ V(f) & x = s \\ -D & x = t \end{cases}$$

$$\boxed{\therefore \sum_{x \in V} \Big(Out_f(x) - In_f(x) \Big) = V(f) - D} \star$$

<u>Notación</u>: Si $A, B \subseteq V$ entonces $f(A, B) = \sum\limits_{\substack{x \in A \\ y \in B \\ \overrightarrow{xy} \in E}} f(\overrightarrow{xy})$

$$\begin{aligned} \sum_{x \in V} Out_f(x) &= \sum_{x \in V} \sum_{y \in \Gamma^+(x)} f(\overrightarrow{xy}) && (1) \\ &= \sum_{\substack{x, y \in V \\ \overrightarrow{xy} \in E}} f(\overrightarrow{xy}) \\ &= f(V, V) && (2) \end{aligned}$$

(1) Por definición de $Out_f(x)$.
(2) Notación.
<u>Similarmente</u>:

$$\begin{aligned} \sum_{x \in V} In_f(x) &= \sum_{x \in V} \sum_{y \in \Gamma^-(x)} f(\overrightarrow{yx}) && (3) \\ &= \sum_{\substack{x, y \in V \\ \overrightarrow{yx} \in E}} f(\overrightarrow{yx}) \\ &= f(V, V) \end{aligned}$$

(3) Por definición de $In_f(x)$.

$$\therefore \star \text{ Nos queda: } \underbrace{f(V, V) - f(V, V)}_{=0} = V(f) - D$$

$$\therefore V(f) = D$$

■

Problema a resolver: dado un network N hallar f que maximice $V(f)$.

Esto se conoce como el problema del flujo maximal[11].

Notación:

Sea $s = x_0, x_1, \ldots, x_r = t : \overrightarrow{x_i x_{i+1}} \in E$, la notación: $s, x_1, \ldots, x_{r-1}, t : \epsilon$

Significa que se mandan ϵ unidades de flujo a través de ese camino, es decir, si G era un flujo existente definimos:

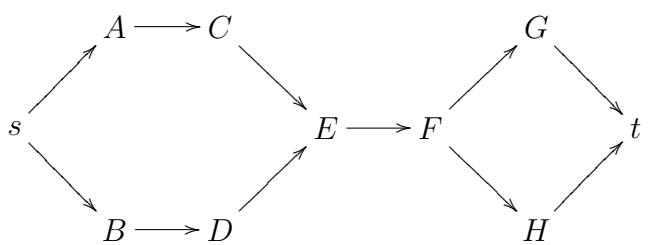
$$f(\overrightarrow{xy}) = \begin{cases} G(\overrightarrow{x_i x_{i+1}}) + \epsilon & Si\ \overrightarrow{xy} = \overrightarrow{x_i x_{i+1}} \\ G(\overrightarrow{xy}) & \text{Si no} \end{cases}$$

2.1. Algoritmo de Greedy para hallar Flujo

"Hasta que no se pueda más buscar caminos de s a t y mandar flujo por ellos."

2.1.1. Ejemplos de Greedy

- Todas las capacidades son 10, excepto $C(\overrightarrow{EF}) = 15$.

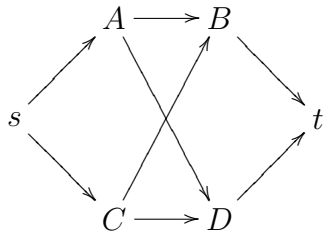


Corremos Greedy en orden alfabético

1. Primer camino: s, A, C, E, F, G, t: 10
2. Segundo camino: s, B, D, E, F, H, t: 5
3. No hay más caminos posibles.

Para este ejemplo podemos demostrar que no sólo es flujo sino también que es maximal, ya que, en pocas palabras $C(\overrightarrow{EF}) = 15$ y es un cuello de botella, y nos da: $f(\overrightarrow{EF}) = 15$

- Ejemplo: todas las capacidades son 1000 excepto: $C(\overrightarrow{CD}) = 1$ y $C(\overrightarrow{CB}) = 900$



Corremos Greedy con el orden alfabético

[11] Por ahora no está claro que f exista.

1. Primer camino: s, A, B, t: 1000
2. Segundo camino: s, C, D, t: 1
3. Terminamos porque no hay más posibles caminos.

En este caso el flujo obtenido (1001) no es maximal, el flujo maximal correspondiente es: (1901)

- Eligiendo caminos obtenemos el flujo maximal correspondiente :1901

1. Primer camino: s, A, D, t: 999
2. Segundo camino: s, A, B, t: 1
3. Tercer camino: s, C, B, t: 900
4. Cuarto camino: s, C, D, t: 1

2.1.2. Complejidad de Greedy

Greedy

- Buscar camino dirigido: $O(m)$
- Aumentar en ese camino: $O(n)$
- Total por aumento: $O(m) + O(n) = O(m)$

¿Cuántos aumentos de flujo hay?

En Greedy una vez que un lado se "satura", es decir $f(\overrightarrow{xy}) = C(\overrightarrow{xy})$, no se vuelve a usar, ya que no se devuelve flujo.

Además sabemos que por cada aumento se satura al menos un lado.

$\therefore$ Hay a lo sumo m aumentos.

$\therefore\ Complejidad(Greedy) = m * O(m) = O(m^2)$

2.2. Algoritmo de Ford y Fulkerson

Definición: un **camino aumentante**[12] de s a t es una sucesión de vértices:

$$s = x_0,\ x_1, \ldots,\ x_r = t : \forall i \leq r - 1$$

- O bien: $x_i\overrightarrow{x_{i+1}} \in E : f(x_i\overrightarrow{x_{i+1}}) < C\left(x_i\overrightarrow{x_{i+1}}\right)$ estos son los lados **Forward**.
- O bien: $x_{i+1}\overrightarrow{x_i} \in E : f(x_{i+1}\overrightarrow{x_i}) > 0$ i.e. que haya mandado flujo, lados **Backward**.

"Hasta que no se pueda más buscar caminos aumentantes y mandar flujo por ellos".

[12] Augmenting path.

2.2.1. Lema del Camino Aumentante

Si f es un flujo y se aumenta a lo largo de un camino aumentante el resultado también es flujo. Más aún si se manda ϵ por el camino aumentante, el valor del nuevo flujo es $V(f) + \epsilon$.

<u>Prueba</u>:

Sea $s = x_0, x_1, \ldots, x_r = t$ un camino aumentante.

$$Sea\ \epsilon_i = \begin{cases} C(x_i\overrightarrow{x_{i+1}}) - f(x_i\overrightarrow{x_{i+1}}) & \text{para los lados forward} \\ f(\overrightarrow{x_{i+1}x_i}) & \text{para los lados backward} \end{cases}$$

Elijo $\epsilon = \text{mín}\{\epsilon_i\}$

"Aumentar f" es tomar

$$f^*(\overrightarrow{xy}) = \begin{cases} f(x_i\overrightarrow{x_{i+1}}) + \epsilon & Si\ \overrightarrow{xy} = x_i\overrightarrow{x_{i+1}} \text{ forward} \\ f(\overrightarrow{x_{i+1}x_i}) - \epsilon & Si\ \overrightarrow{xy} = \overrightarrow{x_{i+1}x_i} \text{ backward} \\ f(\overrightarrow{xy}) & \text{en caso contrario} \end{cases}$$

Queremos ver que f^* es flujo y que $V(f^*) = V(f) + \epsilon$

- <u>Feasability</u>: Obvio por definición de ϵ_i , ϵ.
- <u>Conservación</u>: $x \neq s, t \Rightarrow x = x_i : 0 < i < r$.

  Hay que analizar 4 casos:

  1. F-F: $x_{i-1} \xrightarrow{+\epsilon} x_i \xrightarrow{+\epsilon} x_{i+1}$

     $$Out_{f^*}(x_i) = Out_f(x_i) + \epsilon \ //\ x_i\overrightarrow{x_{i+1}}$$

     $$In_{f^*}(x_i) = In_f(x_i) + \epsilon \ //\ \overrightarrow{x_{i-1}}x_i$$

     $$Out_{f^*}(x_i) - In_{f^*}(x_i) = Out_f(x_i) - In_f(x_i) + \epsilon - \epsilon = 0 \text{ porque } f \text{ es flujo.}$$

  2. F-B: $x_{i-1} \xrightarrow{+\epsilon} x_i \xleftarrow{-\epsilon} x_{i+1}$

     $$Out_{f^*}(x_i) = Out_f(x_i)$$

     $$In_{f^*}(x_i) = In_f(x_i) + \cancel{\epsilon} - \cancel{\epsilon}$$

     $$\text{Luego: } Out_{f^*}(x_i) - In_{f^*}(x_i) = 0$$

  3. B-F: $x_{i-1} \xleftarrow{-\epsilon} x_i \xrightarrow{+\epsilon} x_{i+1}$

     $$In_{f^*}(x_i) = In_f(x_i)$$

     $$Out_{f^*}(x_i) = Out_f(x_i) + \cancel{\epsilon} - \cancel{\epsilon}$$

     $$\text{Luego: } Out_{f^*}(x_i) - In_{f^*}(x_i) = 0$$

4. B-B: $x_{i-1} \xleftarrow{-\epsilon} x_i \xleftarrow{-\epsilon} x_{i+1}$

$$In_{f^*}(x_i) = In_f(x_i) - \epsilon$$

$$Out_{f^*}(x_i) = Out_f(x_i) - \epsilon$$

$$\text{Luego: } Out_{f^*}(x_i) - In_{f^*}(x_i) = 0$$

- <u>En el caso de s</u>:

1. $s \xrightarrow{+\epsilon} x_1$

$$Out_{f^*}(s) = Out_f(s) + \epsilon$$

$$In_{f^*}(s) = In_f(s)$$

$$\therefore V(f^*) = V(f) + \epsilon$$

2. $s \xleftarrow{-\epsilon} x_1$

$$\begin{aligned} Out_{f^*}(s) &= Out_f(s) \\ In_{f^*}(s) &= In_f(s) - \epsilon \\ \therefore V(f^*) &= V(f) - (-\epsilon) \\ &= V(f) + \epsilon \end{aligned}$$

■

<u>Definición</u>: dado $N = (V, E, C)$ con vértices $s =$ fuente, $t =$ resumidero.

Un **corte** es un subconjunto $S \subseteq V : s \in S,\ t \notin S$

<u>Ejemplos de cortes</u>:

- $S = \{s\}$.
- $S = V - \{t\}$.

¿Cuántos cortes hay? 2^{n-2}
La <u>capacidad de un corte</u> S es:

$$\begin{aligned} Cap(S) &= C(S, \overline{S}) : \overline{S} = V - S \\ &= \sum_{\substack{x \in S \\ y \in \overline{S} \\ \overrightarrow{xy} \in E}} C(\overrightarrow{xy}) \end{aligned}$$

<u>Lema</u>: sea N network, f flujo y S corte entonces:

1. $V(f) = f(S, \overline{S}) - f(\overline{S}, S)$
2. $V(f) \leq Cap(S)$
3. Si $V(f) = Cap(S)$ entonces f es <u>Maximal</u>, y S es <u>Minimal</u> i.e. el corte de **menor** capacidad.

<u>Prueba</u>:

1. Observemos que:
   - $f(A \cup B, C) = f(A, C) + f(B, C) : A, B$ disjuntos.
   - $f(A, B \cup C) = f(A, B) + f(A, C) : B, C$ disjuntos.

   <u>Además</u>:
   - $Out_f(x) = f\Big(\{x\}, V\Big)$
   - $In_f(x) = f\Big(V, \{x\}\Big)$

   <u>Además si $x \in S$</u>:

$$Out_f(x) - In_f(x) = \begin{cases} V(f) & Si\ x = s \\ 0 & Si\ x \neq s \text{ pues } t \notin S \end{cases}$$

$$\begin{aligned}
\sum_{x \in S} \Big(Out_f(x) - In_f(x)\Big) &= 0 + \ldots + 0 + V(f) \\
&= V(f) \\
\therefore V(f) &= \sum_{x \in S} Out_f(x) - \sum_{x \in S} In_f(x) \\
&= \sum_{x \in S} f\Big(\{x\}, V\Big) - \sum_{x \in S} f\Big(V, \{x\}\Big) && (4) \\
&= f(S, V) - f(V, S) \\
&= f(S, S \cup \overline{S}) - f(S \cup \overline{S}, S) && (5) \\
&= \cancel{f(S, S)} + f(S, \overline{S}) - f(\overline{S}, S) - \cancel{f(S, S)} && (6) \\
&= f(S, \overline{S}) - f(\overline{S}, S)
\end{aligned}$$

(4) Por la observación.

(5) $V = S \cup \overline{S}$.

(6) Disjuntos.

2. $V(f) \overset{1}{=} \underbrace{f(S,\overline{S}) - \underbrace{f(\overline{S},S)}_{\geq 0}}_{<0}$

$$\therefore V(f) \leq f(S,\overline{S}) \leq C(S,\overline{S}) = Cap(S)$$

3. Sea g un flujo cualquiera:

$$V(g) \overset{2}{\leq} Cap(S) \overset{Hip(3)}{=} V(f) \Rightarrow \text{f es } \underline{\text{maximal}}$$

Similarmente, si T es corte tenemos que:

$$Cap(T) \overset{2}{\geq} V(g) \ \forall g$$

En particular elijo como flujo a f y me queda:

$$Cap(T) \overset{2}{\geq} V(f) \overset{Hip}{=} Cap(S) \therefore \text{S es } \underline{\text{minimal}}$$

■

2.3. Teorema: Max-Flow Min-Cut

En todo network el valor de todo flujo maximal es igual a la capacidad de todo corte minimal, es decir si f es maximal $\Rightarrow \exists S$ corte : $V(f) = Cap(S)$.

Demostración: dado f maximal[13] construyamos S **recursivamente** así:

- $s \in S$ por definición.
- $\star$ Si $x \in S, y \in \Gamma^+(x) : f(\overrightarrow{xy}) < C(\overrightarrow{xy})$ entonces agregamos "y" a S.
- $\dagger$ Si $x \in S, y \in \Gamma^-(x) : f(\overrightarrow{yx}) > 0$ entonces agregamos "y" a S.

Es decir S es el conjunto de vértices que revisamos tratando de obtener un camino aumentante de s a t.

Si t estuviera en S habría un camino aumentante de s a t.

Por el lema del camino aumentante podríamos construir un flujo g tal que:

$$V(g) = V(f) + \epsilon \text{ para algún } \epsilon > 0$$
$$\therefore V(g) > V(f) \textbf{ Absurdo} \text{ pues } f \text{ es maximal.}$$

$\therefore t \notin S$ *i.e.* S es **corte**.

$$\text{Sólo resta ver que: } V(f) = Cap(S)$$

$\therefore$ Por el lema anterior: $V(f) = f(S, \overline{S}) - f(\overline{S}, S)$

- Si $\left.\begin{array}{r} x \in S \\ y \notin S \\ \overrightarrow{xy} \in E \end{array}\right\} \overset{\star}{\Rightarrow} f(\overrightarrow{xy}) = C(\overrightarrow{xy})$ porque sino hubieramos agregado a "y" en algún momento.
$$\overset{Def}{\Rightarrow} f(S, \overline{S}) = C(S, \overline{S}) = Cap(S)$$
- Si $\left.\begin{array}{r} x \in S \\ y \notin S \\ \overrightarrow{yx} \in E \end{array}\right\} \overset{\dagger}{\Rightarrow} f(\overrightarrow{yx}) = 0 \Rightarrow f\big(\overline{S}, S\big) = 0$

$$\begin{aligned} \therefore V(f) &= f(S, \overline{S}) - f(\overline{S}, S) \\ &= Cap(S) - 0 \\ &= Cap(S) \end{aligned}$$

■

Corolario[14]: Ford-Fulkerson es **correcto** i.e. si el output es un flujo f, entonces f es maximal.

Prueba: el algoritmo aumenta a partir del flujo "0" usando caminos aumentantes hasta que no encuentra más caminos.

Sea f el flujo obtenido allí.

[13] Todavía no demostramos que exista.

[14] De la demostración.

Cuando $< F - F >$[15] intentó aumentar f no pudo, entonces $\nexists$ camino aumentante para f entre s y t.

$\therefore$ Los vértices que se buscaron formaron un corte S.

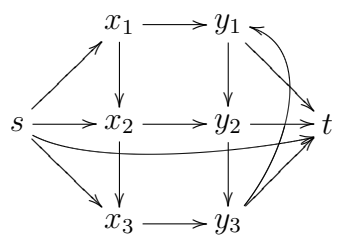
Por la prueba del teorema[16], tenemos que $V(f) = Cap(S)$ entonces f es maximal por el lema previo al teorema. ■

2.3.1. Complejidad de Ford Fulkerson

Los lados se pueden des-saturar por lo tanto el análisis de Greedy no vale.

De todas maneras existen ejemplos en que $< F - F >$ no termina, por lo que no podemos hablar de su complejidad.

<u>Ejemplo en que $< F - F >$ no termina</u>



Capacidades: todas 2 salvo: $\overrightarrow{st} : 1000$ $\overrightarrow{x_1y_1} : 1$ $\overrightarrow{x_2y_2} : r$ $\overrightarrow{x_3y_3} : r^2$

Donde: $r = \frac{-1+\sqrt{5}}{2}$ es raíz de: $P(r) = r^2 + r - 1$

<u>Observación</u>:

- $1 - r = r^2$
- $r - r^2 = r^3$
- Satisface: $1 > r > r^2 > \ldots > r^k > r^{k+1}$

Corramos $< F - F >$

1. $s, x_1, y_1, t : 1$
2. $s, x_3, y_3, \overleftarrow{y_1, x_1}, x_2, y_2, t : r^2$
3. $s, x_2, y_2, \overleftarrow{y_3, x_3}, x_1, y_1, t : r^3$
4. $s, x_1, y_1, \overleftarrow{y_2, x_2}, x_3, y_3, t : r^4$

Luego $< F - F >$ repite (2) (3) (4) con r^5 r^6 r^7 $\ldots$

[15] Denotaremos así al algoritmo de Ford-Fulkerson.

[16] Max Flow Min Cut.

$\therefore < F - F >$ no termina, más aún si f_i son los flujos intermedios:

$$\begin{aligned}\lim_{i\to\infty} V(f_i) &= 1 + r + r^2 + r^3 + \dots \\ &= \sum_{i=0}^{\infty} r^i - r \\ &= \frac{1}{1-r} - r \\ &= \frac{1 - r(1-r)}{1-r} = 2\end{aligned}$$

Pero el flujo maximal es: 1002, mandando $st = 1000$.

<u>Observación</u>: si las capacidades <u>son enteras</u> entonces como el primer valor del flujo es $0 \in \mathbb{Z}$, y aumenta por:

$$\epsilon = \text{mín}\{\epsilon_i\} : \epsilon_i = \begin{cases} C - f \\ f \end{cases} \in \mathbb{Z}$$

Entonces ϵ será un entero y los flujos intermedios también.

$\therefore$ Si les llamamos $f_1, f_2, \dots \Rightarrow V(f_{i+1}) \geq V(f_i) + 1$.

Como hay cota superior, como por ejemplo: $Cap(\{s\})$, en algún momento <u>debemos parar</u>.

Es decir si las capacidades son enteras $< F - F >$ siempre termina, inclusive cuando termina sabemos que el flujo que devuelve es entero, sumado al hecho de que $< F - F >$ terminó, sabemos que ese flujo es maximal. Y llegamos a demostrar el **Teorema de la integralidad**: si las capacidades son enteras entonces existe un flujo maximal entero.

<u>Ejemplo</u>

$$\begin{tikzcd} & A \arrow[dr] \arrow[dd] & \\ s \arrow[ur] \arrow[dr] & & t \\ & B \arrow[ur] & \end{tikzcd}$$

Capacidades: $\overrightarrow{AB} : 1$, las demás: 10^6

Corremos $< F - F >$:

1. $s, A, B, t : 1$
2. $s, \overleftarrow{B, A}, t : 1$

Luego repetimos los pasos: $(1 \leftrightarrow 2)$ $2 * 10^6$ veces, pero vamos a terminar en algún momento.

<u>Anexo</u>: recordemos la notación $\delta(u, v)$ que referencia la longitud del menor camino entre u y v.

Lema: sea G un grafo conexo, dirigido o no, sea $T = BFS(x)$ para algún $x \in V$, entonces:

$$\forall z \in V \ \delta(z,x) = Nivel_T(z)$$

Prueba:

$$\text{Como } T \subseteq G \Rightarrow \delta_G(z,x) \leq \delta_T(z,x) \stackrel{?}{=} Nivel_T(z)$$

Para demostrar que vale el "=" lo vamos a hacer por **inducción** en $\delta(z,x)$.

- Caso base:

$$\delta(z,x) = 0 \Rightarrow z = x \Rightarrow Nivel_T(z) = 0 = \delta(z,x)$$

- Paso inductivo:

Supongamos que vale para $\delta(z,x) = k$ y tomemos un z con $\delta(z,x) = k+1$.

Por lo que vimos al comienzo de la prueba:

$$k + 1 = \delta_G(z,x) \leq Nivel_T(z)$$

Si es igual ya está.

Entonces supongamos que $Nivel_T(z) = j > k + 1$
Luego $\delta(z,x) = k + 1 \Rightarrow \exists$ un camino de longitud mínima entre x y z de la forma:

$$x = x_0, x_1, \ldots, x_k, x_{k+1} = z$$

Entonces $x = x_0, x_1, \ldots, x_k$ es de longitud mínima entre x y $x_k \Rightarrow \delta(x, x_k) = k \stackrel{H.Ind}{\Rightarrow} Nivel_T(x_k) = k$

$$\left.\begin{array}{c} x_k z \in E \\ Nivel_T(z) = j \\ Nivel_T(x_k) = k \end{array}\right\} \stackrel{T=BFS(x)}{\Rightarrow} |j-k| \leq 1$$

Ya que como son vecinos la diferencia de nivel es 0 ó 1.

$$Pero \left.\begin{array}{c} |j-k| \leq 1 \\ j > k+1 \end{array}\right\} \Rightarrow \textbf{Absurdo}.$$

■

2.4. Algoritmo de Edmonds Karp

En 1970 Edmonds y Karp dieron 2 "heurísticas", que de ahora en adelante llamaremos $< E - K >$.

Consiste en correr $< F - F >$ usando caminos aumentantes de longitud mínima:

$$< E - K > = < F - F > + BFS$$

2.4.1. Complejidad de Edmonds-Karp

Teorema: la complejidad[17] de $< E - K >$ es $O(nm^2)$.
Prueba:
Supongamos que $< E - K >$ crea flujos intermedios: $O = f_0, f_1, \ldots$
El paso k es el paso que "crea" f_k
Defino:

- $d_k(x) =$ "distancia" entre s y x en el paso k.
- $b_k(x) =$ "distancia" entre x y t en el paso k.

"Distancia": longitud del menor camino aumentante entre dos vértices.

Lema interno

- $d_k(x) \leq d_{k+1}(x)$
- $b_k(x) \leq b_{k+1}(x)$

Prueba: lo haremos para d ya que para b es similar.

$$\text{Sea } A = \Big\{ z \in V : d_{k+1}(z) < d_k(z) \Big\}$$

Queremos ver que $A = \emptyset$, supongamos que **no** y lleguemos a un absurdo.

$$\text{Si } A \neq \emptyset \text{ tomamos } q \in A \text{ tal que } \boxed{d_{k+1}(q) \leq d_{k+1}(z) \ \forall z \in A} \ (1)$$

Como $q \in A \Rightarrow \boxed{d_{k+1}(q) < d_k(q)}$ (2)
Observación: $q \neq s$ pues $d_{k+1}(s) = d_k(s) = 0$
Sea $s = x_0, x_1, \ldots, x_{r-1}, x_r = q$
Un camino aumentante de longitud mínima $r = d_{k+1}(q)$ entre s y q.
(El camino existe pues (2) $\Rightarrow d_{k+1}(q) < \infty$).
Sea $x = x_{r-1}$
Tenemos: $d_{k+1}(x) = r - 1 = d_{k+1}(q) - 1 < d_{k+1}(q)$

$$\therefore \ \boxed{d_{k+1}(x) < d_{k+1}(q)} \ (3)$$

$$d_{k+1}(x) < d_{k+1}(q) \Rightarrow x \notin A \text{ por } (1)$$

(En otras palabras, como x está antes que q, no puede estar en A)

$$\text{Luego como } x \notin A \Rightarrow \boxed{d_k(x) \leq d_{k+1}(x)} \ (4)$$

[17] Recordando que: $n = |V|$ y $m = |E|$.

Diremos que un lado (u, v) está "disponible" en un paso j si:

- Es Forward con $f_{j-1}(\overrightarrow{uv}) < C(\overrightarrow{uv})$
- Es Backward con $f_{j-1}(\overrightarrow{vu}) > 0 \left(i.e.\ \overrightarrow{vu} \in E\right)$

¿Está disponible (x, q) en el paso k?

Supongamos que **sí** entonces:

$$\begin{aligned} d_k(q) &\leq d_k(x) + 1 \\ &\overset{(4)}{\leq} d_{k+1}(x) + 1 \\ &\overset{(3)}{=} d_{k+1}(q) \\ &\Rightarrow \text{Contradice (2) } \textbf{Absurdo!} \end{aligned}$$

Por lo tanto xq **no** está disponible en el paso k, pero **sí** lo está en el paso $k+1$. Entonces lo usamos para el otro lado en el paso k, i.e. (q, x) se usó en el paso k. Como estamos usando $< E - K >$ los caminos aumentantes son de longitud mínima.

$$\therefore \boxed{d_k(x) = d_k(q) + 1} \ (5)$$

$$\begin{aligned} \text{Entonces: } d_{k+1}(q) &\overset{(3)}{=} d_{k+1}(x) + 1 \\ &\overset{(4)}{\geq} d_k(x) + 1 \\ &\overset{(5)}{=} d_k(q) + 1 + 1 \\ &= d_k(q) + 2 \\ &\overset{(2)}{>} d_{k+1}(q) + 2 \\ &\Leftrightarrow 0 > 2 \ \textbf{Absurdo!} \end{aligned}$$

Fin Lema Interno

Continuando con el teorema, llamemos "crítico" a un lado disponible en un paso y no en el siguiente.

Supongamos que xz se vuelve crítico en el paso k, antes de poder ser crítico otra vez debe ser usado en la otra dirección, sea $L > k$ el paso donde se usa la próxima vez para el otro lado.

Pues estoy en $< E - K >$ entonces $\begin{cases} \boxed{d_k(z) = d_k(x) + 1} \ \star \\ \boxed{d_L(x) = d_L(z) + 1} \ \dagger \end{cases}$

$$\begin{aligned} d_L(t) &= d_L(x) + b_L(x) \\ &= d_L(z) + 1 + b_L(x) && (7) \\ &\geq d_k(z) + 1 + b_k(x) && (8) \\ &= d_k(x) + 1 + 1 + b_k(x) && (9) \\ &= d_k(t) + 2 \end{aligned}$$

$$\therefore d_L(t) \geq d_k(t) + 2$$

(7) Por †
(8) Por el Lema Interno
(9) Por $\star$

Por lo tanto cuando un lado se vuelve crítico recién puede volver a usarse cuando la distancia a t haya aumentado en por lo menos 2.

$$\#\text{ Veces que un lado puede volverse crítico } = O(n)$$

Exactamente sería: $\frac{n-1}{2}$

Complejidad de cada búsqueda $BFS +$ Aumento $= O(m) + O(n) = O(m)$
En cada aumento al menos un lado se vuelve crítico.
Hay m lados y cada uno se vuelve crítico $\leq O(n)$ veces.

$$\therefore\ Complejidad(< E - K >) = O(m) * O(m) * O(n) = O(nm^2)$$

■

<u>Corolario de la complejidad de $< E - K >$</u>

Siempre existe un flujo maximal.

2.4.2. Ejemplo de Edmonds Karp

Capacidades	Caminos
sA:20	
sC:20	
AB:20	
AE:3	
AF:10	
AI:3	
Bt:10	
BI:10	
BJ:10	
CD:10	
CK:10	
Dt:22	
EG:10	
EH:10	
FG:10	
GD:3	
HD:10	
It:10	
Jt:2	
KI:10	

Primer Camino: $s, A, B, t : 10$

s	A	C	B	E	F	I	D	K	t
	s	s	A	A	A	A	C	C	B
	20	20	20	3	10	3	10	10	10

Segundo Camino: $s, A, I, t : 3$

s	A	C	B	E	F	I	D	K	J	G	H	t
	s	s	A	A	A	A	C	C	B	E	E	I
	10	20	10	3	10	3	10	10	10	3	3	3

Tercer Camino: $s, C, D, t : 10$

s	A	C	B	E	F	D	K	I	J	G	H	t
	s	s	A	A	A	C	C	B	B	E	E	D
	7	10	7	3	7	10	10	7	7	3	3	10

Cuarto Camino: $s, A, B, I, t : 7$

s	A	C	B	E	F	K	I	J	G	H	t
	s	s	A	A	A	C	B	B	E	E	I
	7	10	7	3	7	10	7	7	3	3	7

Quinto Camino: $s, C, K, I, B, D, t : 2$

s	C	K	I	A	B	E	F	J	G	H	t
	s	C	K	I^-	I^-	A	A	B	E	E	J
	10	10	10	3	7	3	3	7	3	3	2

Sexto Camino: $s, C, K, \overleftarrow{I}, A, E, G, D, t : 3$

s	C	K	I	A	B	E	F	J	G	H	D	t
	s	C	K	I^-	I^-	A	A	B	E	E	G	D
	8	8	8	3	5	3	3	5	3	3	3	3

Séptimo Camino: $s, C, K, \overleftarrow{I}, \overleftarrow{B}, A, F, \overleftarrow{G}, E, H, D, t : 3$

s	C	K	I	B	J	A	F	G	E	H	D	t
	s	C	K	I^-	B	B^-	A	F	G^-	E	H	D
	5	5	5	5	5	5	5	5	3	3	3	3

Octavo Camino?: No llega a t

s	C	K	I	B	J	A	F	G
	s	C	K	I^-	B	B^-	A	F
	2	2	2	2	2	2	2	2

Obtengo el Corte: S = {s,c,k,i,b,j,a,f,g}

$$\underline{\text{Capacidad del corte S}}$$

$$\begin{aligned} Cap(S) =& 10 + 10 + 10 + 2 + 3 + 3 \ = 38 \\ & cd + it + bt + dt + ae + gd \end{aligned}$$

$$\begin{aligned} \Rightarrow V(f) &= Out_f(s) - \overbrace{In_f(s)}^{=0} \\ &= f(\overrightarrow{sa}) + f(\overrightarrow{sc}) \\ &= 20 + 18 = 38 \end{aligned}$$

2.4.3. Pseudo-Código de Edmonds Karp

```
E-K(Network N)
{
  F = 0; //Flujo
  v = 0; // Valor del Flujo
  S = V; // Corte
  while(t ∈ S){ //mientras el vertice t este en el corte
    //buscar camino y aumentar
    Q = new_queue(S); //Crear cola
    E[x] = ∞ ∀x ∈ V;
    S = {s}; //corte
    while(head(q) ≠ t){
      x = head(q);
      Forward_search(x);
      Backward_search(x);
      dequeue(q); //saco x de la cola
    }
    if (t ∈ S){
      Aumentar_flujo();
    }
  }
  return (F,v,S) //flujo, valor, corte
}

Forward_search(vertice x)
{
  for(q ∈ Γ+(x) && q ∉ S){
    if(f(xq) < C(xq)){
        enqueue(Q,q); //Agregar q a Q
        S = S ∪ {q};
        A[q] = x;   //Ancestro
        E[q] = min{E(x),C(xq) - F(xq)};
        b[q] = 1; //forward
    }
  }
}
```

```
Backward_search(vertice x)
{
  for(q ∈ Γ⁻(x) && (q ∉ S)){
    if(f($\overrightarrow{qx}$)>0) {
        enqueue(Q,q); //Agregar q a Q
        S = S ∪ {q};
        A[q] = x;   //Ancestro
        b[q] = −1; //Backward
        E[q] = min{E(x),F($\overrightarrow{qx}$)};
    }
  }
}

Aumentar_Flujo()
{
  p = t; //pivote
  ϵ = E(t);
  v = v + ϵ;
  while( p ≠ s){
    q = a[p];
    if (b[p] == 1){ //FORWARD
      f($\overrightarrow{qp}$) = f($\overrightarrow{qp}$) + ϵ;
    }else{
      f($\overrightarrow{pq}$) = f($\overrightarrow{pq}$) − ϵ;
    }
  p = q;
  }
}
```

2.5. Algoritmos Tipo Dinic

Idea: usar un network auxiliar que se construye de la siguiente manera: corremos BFS de caminos aumentantes, como en $< E - K >$, agregando los vértices y lados correspondientes excepto que en $< E - K >$ cuando p encuentra un q que ya está, no lo agrega, mientras que acá a veces sí, i.e. supongamos que estamos revisando p y hallamos $q \in \Gamma^+(p)$ con $f(\overrightarrow{pq}) < C(\overrightarrow{pq})$ ó $q \in \Gamma^-(p) : f(\overrightarrow{qp}) > 0$, pero que q ya está en el network auxiliar, entonces agregamos un lado pq al network auxiliar si el $Nivel(q) = Nivel(p) + 1$.

Definición: un flujo G se dice **bloqueante** o saturante si para todo camino:

$$s = x_0, x_1, \ldots, x_r = t \text{ con } (\overrightarrow{x_i x_{i+1}}) \in E \ \exists i : G(\overrightarrow{x_i x_{i+1}}) = C(\overrightarrow{x_i x_{i+1}})$$

Dado f flujo en N y g flujo en un network auxiliar denotemos por:

$$(f \oplus g)(x, y) = \begin{cases} f(x,y) + g(x,y) & \text{Forward} \\ f(x,y) - g(y,x) & \text{Backward} \end{cases}$$

2.5.1. Esquema general Tipo Dinic

```
Dinic()
{
  f = 0;
  Stop = NO;
  while(Stop == NO){
    NA = New_Na(f); //construir Network auxiliar a partir de f
    if (t ∈ NA){
      g = Hallar_bloqueante(NA); //busco un flujo bloqueante en NA
      f = f ⊕ g;
    }else{
      Stop = Sí;
    }
  }
  return f;
}
```

<u>Teorema</u>

La distancia en NA[18] sucesivos <u>aumenta</u>, por lo tanto hay a lo sumo n, NA's.

<u>Demostración</u>:

Sea A un NA y A′ el siguiente NA, denotamos con δ la distancia en A y δ' la distancia en A′.

Como A y A′ se construyen con BFS $\Rightarrow \delta \leq \delta'$

Queremos ver que vale el "<".

Sea $s = x_0, x_1, \ldots, x_r = t$ un camino en A'.

- <u>Caso 1</u>: $\exists i \in \{1, \ldots, r-1\} : x_i \notin A$

$$\begin{aligned} \Rightarrow \delta(t) &\leq \delta(x_i) && (10) \\ &\leq \delta'(x_i) && (11) \\ &= i \boxed{<} r = \delta'(t) \end{aligned}$$

  (10) Sino lo hubiera agregado.

  (11) Por $< E - K >$

$$\therefore \delta(t) < \delta'(t)$$

- <u>Caso 2</u>: $x_i \in A \ \forall i$

  Como antes de pasar a A' debemos saturar todos los caminos de A, se sigue que ningún camino de A es un camino de A'.

  Luego: $s, x_1, \ldots, x_{r-1}, t$ **no** es camino en A, como $x_i \in A \ \forall i \Rightarrow \exists i : \vec{x_i x_{i+1}} \notin A$

  Y tomemos el primer tal i.

[18] NA: networks auxiliares.

- <u>Caso 2-a</u>: $\delta(x_{i+1}) < i+1$
  Como $i+1 = \delta'(x_{i+1}) \Rightarrow \boxed{\delta(x_{i+1}) < \delta'(x_{i+1})}\ \star$
  Luego:

$$\begin{aligned} \delta(t) &= \delta(x_{i+1}) + b(x_{i+1}) \\ &\boxed{<}\ \delta'(x_{i+1}) + b(x_{i+1}) && (12) \\ &\leq \delta'(x_{i+1}) + b'(x_{i+1}) && (13) \\ &= \delta'(t) \end{aligned}$$

  (12) Por $\star$
  (13) Por $< E - K >$

- <u>Caso 2-b</u>: $\delta(x_{i+1}) = i+1$
  Observación: es $\leq$ por $< E - K >$ no hace falta ver "$>$".
  Como i es el primero con $\overrightarrow{x_i x_{i+1}} \notin A \Rightarrow \overrightarrow{x_{j-1} x_j} \in A\ \forall j \leq i$

$$\therefore s = x_0, x_1, \ldots, x_i \in A$$
$$\therefore \delta(x_i) = i \Rightarrow \delta(x_{i+1}) = i+1 = \delta(x_i) + 1$$

  Como los niveles son los "correctos" la única razón por la cual $x_i x_{i+1} \notin A$ es porque no está "disponible" en N, i.e. es Forward saturado o Backward vacío.

Pero $\overrightarrow{x_i x_{i+1}} \in A'$ entonces se usó <u>para el otro lado</u> en A, es decir: $\overrightarrow{x_{i+1} x_i} \in A$ **Absurdo** pues $Nivel(x_{i+1}) = i+1$, y $Nivel(x_i) = i$. ■

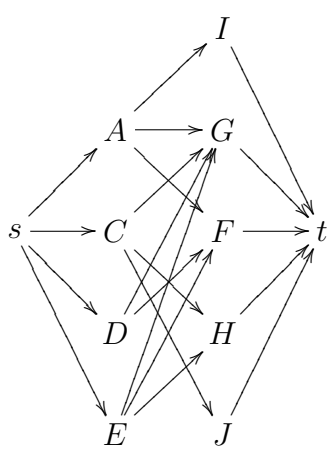
<u>**Corolario**</u>: si un algoritmo tiene complejidad $O(b)$ para hallar flujo bloqueante en el NA, entonces la complejidad del algoritmo completo es $O(n * b)$.
<u>Dinic</u> = Algoritmo tipo Dinic + Greedy "DFS" para hallar flujo bloqueante.
A simple vista sería: $Complejidad(Dinic) = O(nm^2) = Complejidad(< E - K >)$
Parecería que no ganamos nada.
Pero veremos que en realidad Complejidad(Greedy en NA) $= O(nm)$.
$\therefore$ Tendremos $Complejidad(Dinic) = O(n^2 m)$

2.5.2. Ejemplo de Dinic

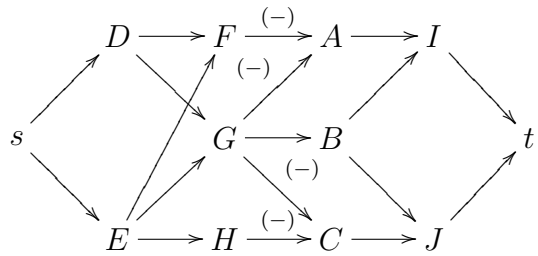
Ejemplo: Aplicar Dinic al siguiente Network.

Inicial	Primera Modificación	caminos (4)	Segunda Modificación (final)	Caminos (6)
sA:8	sA:0	sAFt:4	sA:0	$s\overleftarrow{DFA}It : 1$
sC:7	sC:0	sAGt:3	sC:0	$s\overleftarrow{DGA}It : 2$
sD:10	sD:7	sAIt:1	sD:4	$s\overleftarrow{EFA}It : 3$
sE:12	sE:11	sCGt:2	sE:1	$s\overleftarrow{EGA}It : 1$
AF:4	AF:0	sCHt:3	AF:4	$sEGBIt : 1$
AG:3	AG:0	sCJt:2	AG:3	$SEGBJt : 2$
AI:8	AI:7	sDFt:3	AI:0	$S\overleftarrow{EGC}Jt : 1$
BI:2	BI:2	sEHt:1	BI:1	$s\overleftarrow{EHC}Jt : 2$
BJ:2	BJ:2		BJ:2	
CG:2	CG:0		CG:1	
CH:3	CH:0		CH:2	
CJ:5	CJ:3		CJ:0	
DF:4	DF:1		DF:0	
DG:2	DG:2		DG:0	
EF:3	EF:3		EF:0	
EG:5	EG:5		EG:0	
EH:4	EH:3		EH:1	
Ft:7	Ft:0		Ft:0	
Gt:5	Gt:0		Gt:0	
GB:4	GB:4		GB:1	
Ht:4	Ht:0		Ht:0	
It:9	It:8		It:0	
Jt:15	Jt:13		Jt:8	

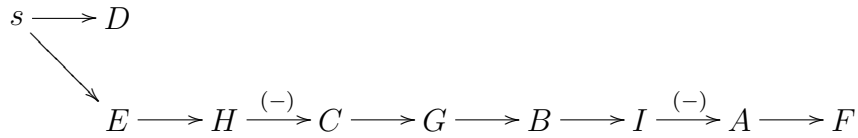
Primer Network Auxiliar



Segundo Network auxiliar



Tercer Network auxiliar



Obtengo el corte del conjunto de vértices del NA que no llega a t, el último.

$$S = \{s, d, e, h, c, g, b, i, a, f\}$$

$$\begin{array}{lcccccccc} Cap(S) & = & 4+ & 5+ & 5+ & 2 & + & 9+ & 7 & = 32 \\ & & ht+ & cj+ & gt+ & bj & + & it+ & ft & \end{array}$$

$$\text{Se ve que } Cap(S) = 32 = Out_f(s)$$

Entonces tengo una garantía de la resolución del ejercicio.

Otra garantía es ver que los caminos obtenidos de un NA tengan el mismo tamaño y que a medida que voy construyendo otros NA's sean más largos, en este caso los primeros tenían 4 de largo y los caminos del segundo network auxiliar tenian un largo de 6.

2.5.3. Pseudo Código algoritmos Tipo Dinic

```
Dinic(Network Na){ // Parte Bloqueante, Version con Goto
  g = 0;
2 p = s; //label 2 , pivote
1 if(p ≠ t){ //label 1
    if(Γ+(p) ≠ ∅){
      avanzar();
      goto 1;
    }else if(p ≠ s){
      retroceder();
      goto 1;
    }else return g;
  }else{
    Incrementar_flujo();
    goto 2;
  }
}

Dinic(Network Na){  \\ Parte Bloqueante ,Version goto less;
  g = 0;
  E(s) = ∞;
  continuar = 1;
  while(continuar){
    p = s;
    while (p ≠ t && continuar){
      if( Γ+(p) ≠ ∅){
        Avanzar();
      }else if(p ≠ s){
        Retroceder();
      }else{
        continuar = 0;
      }
    }
    if (p == t){
      Incrementar_flujo();
    }
  }
  Return g
}
```

```
// Pre p tiene vecino
Avanzar(pivote p) {
  Tomar q ∈ Γ+(p)
  A(q) = p; //ancestro de q es p
  E(q) = min{E(p),C(pq)−G(pq)};
  p = q;
}

// Pre p tiene vecino
Retroceder(pivote p)
{
  Borrar p de Γ+(A(p))
  p = A(q);
}

Incrementar_Flujo()
{
  ϵ = E(t);
  q = t;
  while(q ≠ s){
    p = a(q);
    G(pq) = G(pq) + ϵ;
    if (G(pq) == C (pq)){
      Borrar q de Γ+(p)
    }
  }
  q = p;
}
```

2.5.4. Complejidad de Dinic

Teorema: la complejidad de Dinic es $O(n^2m)$

Prueba: por el teorema general de los algoritmos "tipo Dinic" basta ver que la complejidad del paso bloqueante es $O(n * m)$.

Sea:

- A = Avanzar()
- R = Retroceder()
- I = Incrementar_Flujo() + Inicialización ($O(1)$)

Una corrida de Dinic luce como:

$$A \dots AIA \dots ARAIA \dots ARAI \dots RRRRA \dots AI$$

Sea X = I ó R.

Tomemos una sucesión de palabras de la forma: $A \dots AX$

1. ¿Cuántas palabras hay?

   Si $X = R$: borramos un lado.

   Si $X = I$: borramos al menos un lado.

   $\therefore$ Cada X borra al menos un lado $\Rightarrow$ hay $\leq m$ palabras de la forma $A \dots AX$

2. ¿Cuál es la complejidad máxima de cada palabra?

   Complejidad de $R = O(1)$

   Complejidad de $I = O(n)$; hay $\leq n$ vértices en el camino $s, \dots, t$.

   $\therefore$ Complejidad$(X) = O(n)$

   Complejidad de $A = O(1)$

   $\therefore$ $Complejidad(\underbrace{A \dots A}_{r} X) = O(r) + O(n)$

Como cada Avanzar() mueve el pivote un nivel más cerca de t entonces hay a lo sumo n Avanzar() antes de un R o un I $\therefore r \leq n$.

$$Complejidad(A \dots AX) = O(n) + O(n) = O(n)$$

(1) + (2) $\Rightarrow$ Complejidad(Paso Bloqueante de Dinic) $= m * O(n) = O(n * m)$ ■

2.6. Algoritmo Wave de Tarjan

Es un ejemplo de una clase de algoritmos en los cuales la invariante no es más "ser flujo".

Definición: un **Preflujo** es como un flujo sin la condición de balanceo.

En el caso de Wave[19] el desbalanceo sólo podrá ser por exceso de entrada, pero otros algoritmos permiten desbalanceo de los dos lados.

Idea: una ola hacia adelante ("Forward") que balancea si puede y marca ("bloquea") si no.

Una ola hacia atrás que balancea los bloqueados ("Backward").

2.6.1. Pseudo Código de Wave

```
1  Wave(){
2     inicializar();
3     olaForward();
4     olaBackward();
5     while(N != 2){
6        olaForward();
7        olaBackward();
8     }
9     return g;
10 }
11
12 inicializar(){
13    g = 0;
14    ∀(x ≠ s){
15       B(x) = 0; //No bloqueado
16       D(x) = 0 //Desbalanceo
17       A(x) = ∅ //subconjunto de aquellos que le mandaron flujo a x
18    }
19
20    D(s) = 0;
21    N = 1; // #Vertices balanceados
22
23    ∀(x ∈ Γ+(s)){
24       G(sx⃗) = C(sx⃗);
25       D(s) = D(s) − C(sx⃗);
26       D(x) = C(sx⃗);
27       N++;
28       A(x) = {s};
29    }
30 }
31
32 olaForward()
33 {
34    for (x = s +1; To ;t−1){ //Orden BFS()
35       if(D(x)>0)
36          ForwardBalance(x);
37    }
38 }
```

[19] Wave: inglés, ola.

```
olaBackward()
{
  for (x = t - 1 DownTo s+1){ //Orden BFS()
    if(D(x)>0 && B(x) == 1)
      BackwardBalance(x);
  }
}

BackwardBalance(vertice x){
  while(D(x)>0){
    Tomar y ∈ A(x);
    m = min{D(x),G(yx)};
    g(yx)= g(yx) - m;
    D(x) = D(x) - m;
    if(D(y) == 0){
      N++;
    }
    D(y) = D(y) + m;
    if(G(yx) == 0){
      A(x) = A(x) - {y};
    }
  }
  N--;
}


ForwardBalance(vertice x){
  while(D(x)>0 && (Γ+(x) ≠ ∅)){
    Tomar y ∈Γ+(x);
    if(B(y) == 1) {
      Γ+(x) = Γ+(x) - {y}
    }else{
      m = min{D(x),C(xy) - G(xy)}
      G(xy) = G(xy) + m ;
      D(x) = D(x) - m;
      if(D(y) == 0){
        N++;
      }
      D(y) = D(y) + m;
      A(y) = A(y) ∪ {x};
      if(G(xy) == C(xy)){
        Γ+(x) = Γ+(x) - {y}
      }
    }
  }
  if(D(x)>0)
    B(x) = 1; //Bloqueamos x
  else
    N--;
}
```

2.6.2. Complejidad de Wave

Teorema: la complejidad de Wave es $O(n^3)$

Prueba: como es "Tipo Dinic" sólo basta ver que la complejidad del paso bloqueante es $O(n^2)$.

Sea $()_t$ partes del algoritmo en donde se satura totalmente un lado o se vacía totalmente un lado.

Y sea $()_p$ donde se manda o devuelve parcialmente flujo.

$$Compl(Wave_bloqueante) = Compl(Wave_bloqueante)_t + Compl(Wave_bloqueante)_p$$

- Analicemos primero la $Compl(Wave_bloqueante)_p$.

  Cada ola Forward, salvo quizás la última bloquea al menos un vértice nuevo.

  $\therefore$ $\boxed{\text{Hay a lo sumo } n \text{ ciclos: (ola forward,ola backward)}}$ $\star\star$

  En cada ola forward hay $n-2$, $ForwardBalance(x)$

  Entonces:

$$\begin{aligned} Compl(olaForward)_p &= n * Compl(ForwardBalance(x))_p \\ &= n * \underbrace{Compl(ForwardBalance(x))_p}_{O(1)} \\ &= O(n) \end{aligned} \tag{14}$$

  (14) Pues en cada $FB(x)$ a sólo **un** lado le mandamos flujo parcial.

$$\boxed{Compl(ola\ Forward)_p = O(n)}\ (2)$$

  Luego:

$$\boxed{(2)\ y\ \star\star} \Rightarrow Compl(Wave_bloqueante)_p = O(n^2)$$

- Ahora analicemos $Compl(Wave_bloqueante)_t$.

  Cada vez que un lado se satura completamente no puede volver a saturarse, pues si el lado es $\vec{xy}$ lo quisiera volver a saturar debería primero hacer que "y" le devuelva el flujo a "x".

  Pero esto sólo puede pasar si "y" está bloqueado y en ese caso "x" no le puede mandar más flujo, en particular no lo puede volver a saturar.

  Similarmente, un lado $\vec{xy}$ sólo se puede vaciar totalmente una vez porque sólo puede hacerlo si "y" está bloqueado.

$$\therefore \#Saturamientos + \#Vaciamientos \leq 2m$$

$$\therefore Compl(Wave_bloqueante)_t = O(m)$$

$$
\begin{aligned}
Compl(Wave_bloqueante) &= Compl(Wave_bloqueante)_t + Compl(Wave_bloqueante)_p \\
&= O(m) + O(n^2) \\
&= O(n^2)
\end{aligned}
$$

■

2.6.3. Ejemplos de Wave

Aplicar Wave al siguiente Network Auxiliar

sA:8	→ **0**
sC:7	→ **0**
sD:10	→ **7**
sE:7	→ **0**
AF:4	→ **0**
AG:3	→ **0**
AI:8	→ **7**
CG:2	→ **0**
CH:3	→ **0**
CJ:5	→ **3**
DF:4	→ **1**
DG:2	→ **2**
EF:3	
EG:5	
EH:4	
Ft:7	→ **3**
Gt:5	→ **0**
Ht:4	→ **0**
It:9	→ **8**
Jt:15	→ **13**

Nivel 0	Nivel 1				Nivel 2					Nivel 3
s	A	C	D	E	F	G	I	H	J	t
-32	~~8~~	~~7~~	~~10~~	~~7~~	~~4~~	~~3~~	~~1~~	~~3~~	~~2~~	~~7~~
	~~4~~	~~5~~	~~6~~	~~4~~	~~8~~	~~5~~	0	~~0~~	0	~~12~~
	~~1~~	~~2~~	$\boxed{\cancel{4}}$	~~0~~	~~11~~	~~7~~				~~13~~
	0	0			$\boxed{\cancel{4}}$	~~11~~				~~16~~
→						$\boxed{\cancel{6}}$				~~18~~
-25			$\boxed{\cancel{6}}$	~~4~~	$\boxed{\cancel{1}}$	$\boxed{\cancel{2}}$				
			$\boxed{\cancel{7}}$	~~7~~	$\boxed{0}$	$\boxed{0}$				
			$\boxed{0}$							←
→				$\boxed{\cancel{3}}$				~~4~~		19
								$\boxed{\cancel{3}}$		
-19				$\boxed{0}$				$\boxed{0}$		
										←

Aplicar Wave Respetando el orden alfabético y observar que el flujo es distinto

sA:8 $\rightarrow$ **0**
sC:7 $\rightarrow$ **0**
sD:10 $\rightarrow$ **5**
sE:7 $\rightarrow$ **0**
AF:4
AG:3
AI:8 $\rightarrow$ **0**
CG:2
CH:3 $\rightarrow$ **0**
CJ:5 $\rightarrow$ **1**
DF:4 $\rightarrow$ **0**
DG:2 $\rightarrow$ **1**
EF:3 $\rightarrow$ **0**
EG:5 $\rightarrow$ **1**
EH:4
Ft:7 $\rightarrow$ **0**
Gt:5 $\rightarrow$ **0**
Ht:4 $\rightarrow$ **0**
It:9 $\rightarrow$ **1**
Jt:15 $\rightarrow$ **11**

Nivel 0	Nivel 1				Nivel 2					Nivel 3
s	A	C	D	E	F	G	I	H	J	t
-32	~~8~~	~~7~~	~~10~~	~~7~~	~~4~~	~~3~~	~~1~~	~~3~~	~~2~~	~~7~~
	~~4~~	~~5~~	~~6~~	~~4~~	~~8~~	~~5~~	~~0~~	0	~~0~~	~~12~~
	~~1~~	~~2~~	$\boxed{\cancel{4}}$	0	~~11~~	~~7~~				~~13~~
	~~0~~	~~0~~			$\boxed{\cancel{4}}$	~~11~~				~~16~~
$\rightarrow$						$\boxed{\cancel{6}}$				~~18~~
-27	~~3~~	~~2~~	$\boxed{\cancel{5}}$		$\boxed{0}$	$\boxed{\cancel{3}}$	~~7~~		~~2~~	~~25~~
	~~7~~	0	$\boxed{0}$			$\boxed{\cancel{1}}$	0		0	27
	0					$\boxed{0}$				$\leftarrow$

Importante ver que termino porque $Out_f(s) = In_f(t)$

3. MST

MST: minimal spanning tree, árbol de expansión o generador minimal.

Definición: sea G un grafo, un **árbol generador** de G es un subgrafo de G que es árbol y generador.

Es decir T es árbol generador de G si:

1. T subgrafo de G.
2. T árbol $\Rightarrow$ Conexo sin ciclos.
3. T generador $\Rightarrow V(T) = V(G)$.

Obviamente si G tiene árbol generador debe ser G conexo si lo es DFS o BFS dan un árbol generador.

Repaso de árboles

Propiedad: T árbol $\Rightarrow m = n - 1$

Demostración: si T es árbol, entonces $DFS(T) = T$.

Si $A(q)$ denota el antecesor de q en DFS, todos los lados de DFS son de la forma: $A(q)q$ para algún $q \neq$ raíz.

$$\therefore \text{ Hay } n-1 \text{ lados. (uno por cada vértice} \neq \text{raíz)}$$

■

Propiedad:

$$\begin{aligned} (1)\ T \text{ árbol} &\Leftrightarrow T \text{ conexo y } m = n-1 && (2) \\ &\Leftrightarrow T \text{ no tiene ciclos y } m = n-1 && (3) \end{aligned}$$

Prueba:

$(1) \Rightarrow \Big((2)\ y\ (3)\Big)$ ya está.

$$\underline{(3) \Rightarrow (1)} : T \text{ no tiene ciclos y } m = n-1 \overset{?}{\Rightarrow} T \text{ árbol}$$

Sólo tengo que ver que es conexo.

Sean $\{x, y\} \in T$, supongamos que **no** hay un camino entre x e y.

En particular $xy \notin E(T)$

Sea $T' = T \cup \{xy\}$

Como no hay caminos en T entre x e y en T' no hay ciclos.

Si C fuese ciclo

- Como T es acíclico $\Rightarrow xy \in T$
- Entonces $C - xy$, es un camino en T entre x e y **absurdo**.

Conclusión: si x, y no tiene un camino entre ellos, $T \cup \{x, y\}$ sigue siendo acíclico.
Continuando de esta forma terminamos con un $\widehat{T}$ conexo y acíclico.
Es decir árbol tal que $T \cup \{xy\} \subseteq \widehat{T}$

$$\begin{aligned} \left|E\left(\widehat{T}\right)\right| &= \left|V\left(\widehat{T}\right)\right| - 1 && (15) \\ &= |V(T)| - 1 \\ &= |E(T)| && (16) \end{aligned}$$

(15) $\widehat{T}$ es árbol
(16) Por $HIP.(3)$
Absurdo pues $T \cup \{x, y\} \subseteq \widehat{T} \Rightarrow \left|E\left(\widehat{T}\right)\right| \geq |E(T)| + 1$

$$\underline{(2) \Rightarrow (1)} : T \text{ conexo y } m = n - 1 \overset{?}{\Rightarrow} T \text{ árbol}$$

Sólo tengo que ver que no contenga ciclos.
Supongamos que T tenga un ciclo, borrando el lado del ciclo, el grafo resultante sigue siendo conexo.
Por lo tanto puedo ir borrando un lado de cada ciclo manteniendo continuidad.
Eventualmente llegamos a un grafo H sin ciclos que sigue siendo conexo.
$\therefore H$ es árbol.

$$\left|E(H)\right| = \left|V(H)\right| - 1 = \left|V(T)\right| - 1 = \left|E(T)\right|$$

Absurdo porque por lo menos quité un lado. ■

Definición: dado un grafo G conexo con pesos en los lados, es decir $W : E(G) \to \mathbb{R}^{\geq 0}$.
Un MST[20] para G es un spanning tree T tal que:

$$\sum_{xy \in E(T)} W(xy) \leq \sum_{xy \in E(T')} W(xy) \ \forall spanning\ Tree\ T'$$

Notación

- $\sum_{xy \in E(T)} W(xy) = W(T)$.

Problema: dado G conexo, hallar un MST para G.

3.1. Kruskal

Repetir hasta tener $n - 1$ lados, agregar el lado de menor peso que no forme ciclo.
Se necesita una estructura de datos de "conjuntos disjuntos".

1. $Makeset(x)$: construye un conjunto $\{x\}$.
2. $Find(x)$: devuelve el conjunto al cual pertenece x.
3. $Union(A, B)$: devuelve la unión de A y B.

[20] Minimal Spanning Tree.

3.1.1. Pseudo Código de Kruskal

```
Kruskal()
{
//α
  ∀(x ∈ V){
     Makeset(x);
  }

//β
  Edges_Sort(); //Ordenar los lados : W(E[i]) ≤ W(E[i+1])
  m = 0; //cantidad de lados
  i = 1; // indice

// γ

  while(M < N−1){
    A = find(x[i]);
    B = find(y[i]);
    if (A ≠ B){
       tree.add(e[i]); //agregar al arbol e[i]
      m++;
       Union(A,B);
    }
    i++;
  }


}
```

3.1.2. Teorema: Kruskal devuelve un MST

Demostración: el invariante es ser acíclico y termina con $n-1$ lados, entonces tenemos garantizado que termina con un spanning tree.

Veamos ahora que es minimal.

Prueba por inducción: sea K_j el grafo en el paso j

- Si $j = 0$ queda demostrado porque los vértices desconectados siempre están.
- Hipótesis inductiva: $\forall j \ \exists$ un MST $T_j : K_j \subseteq T_j$
- Supongamos que $K_j \subseteq T_j$ MST.
  Sea $e : K_{j+1} = K_j \cup \{e\}$
  - Si $e \in T_j$ ya está pues $K_{j+1} \subseteq T_j$
  - Por lo tanto supongamos: $e \notin T_j$
    En este caso, $T_j \cup \{e\}$ tiene: $n - 1 + 1 = n$ lados $\therefore$ no es árbol.
    Como T_j es conexo $\Rightarrow T_j \cup \{e\}$ es conexo.
    Como no es árbol $\Rightarrow T_j \cup \{e\}$ tiene un ciclo C.
    Pero T_j **no** tiene ciclos $\Rightarrow e$ forma parte del ciclo C.

Sea f un lado de C que no esté en K_{j+1} ($\exists f \Leftarrow K_{j+1}$ es acíclico).

$$\begin{aligned} T_{j+1} &= (T_j - f) \cup \{e\} \\ &= (T_j \cup \{e\}) - f \end{aligned}$$

Como C es un ciclo: y $f \in C \Rightarrow (T_j \cup \{e\}) - f$ sigue siendo conexo.

$\therefore T_{j+1}$ es conexo y tiene: $n - 1 + 1 - 1 = n - 1$ lados $\Rightarrow T_{j+1}$ es un árbol.

Como $K_{j+1} \subseteq T_{j+1}$ basta ver que T_{j+1} es minimal.
Pero $W(T_{j+1}) = W(T_j) - W(f) + W(e)$
Pero $f \notin K_{j+1}$
Si $W(f) < W(e) \Rightarrow f$ se "analizó" antes de e.
Como $f \notin K_{j+1}$ entonces se rechazó.
Entonces $K_j \cup \{f\}$ tiene un ciclo **absurdo** pues $K_j \cup \{f\} \subseteq \underbrace{T_j}_{\text{árbol}}$

Ya que $\Big(K_j \subseteq T_j$ y $f \in T_j\Big)$
(estoy diciendo que un subconjunto de un árbol tiene un ciclo, **absurdo**).

$$\therefore W(f) \geq W(e)$$

$\left.\begin{array}{c} W(T_{j+1}) \leq W(T_j) \\ \text{Como } T_j \text{ es MST} \end{array}\right\} \Rightarrow T_{j+1}$ también. ■

3.1.3. Complejidad de Kruskal

Volviendo al pseudocódigo:

$$\begin{aligned} &\alpha = O(n) \\ &\beta = O(m \log(m)) = O(m \log(n^2)) = O(2m \log(n)) = O(m \log(n)) \\ &\gamma = n * (2 * Compl(Find) + Compl(Union)) \end{aligned}$$

Como en general $\beta > \gamma$

$$\therefore Compl(Kruskal) = O(m \log(n))$$

3.1.4. Kruskal Delete

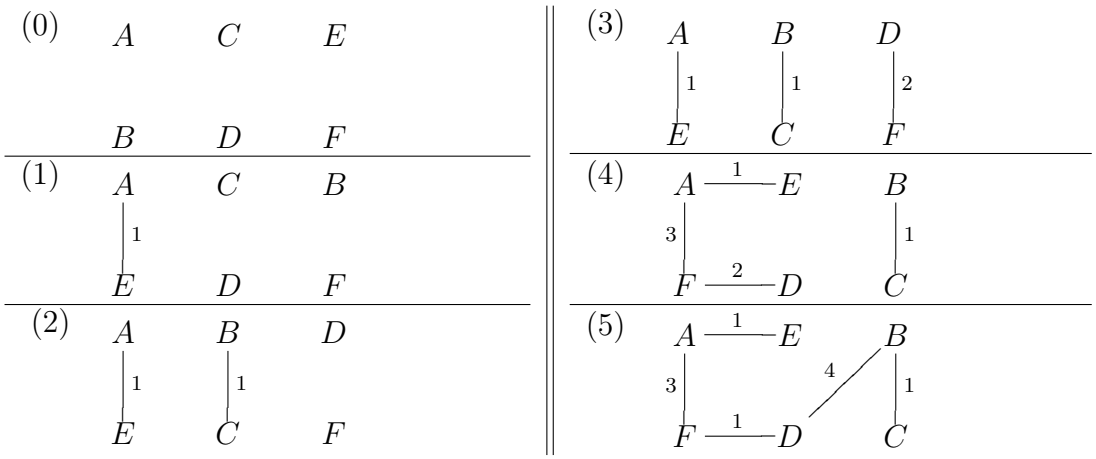
Repetir($\star$) hasta tener $n - 1$ lados.
($\star$) Borrar el lado de mayor peso que no desconecte el grafo.

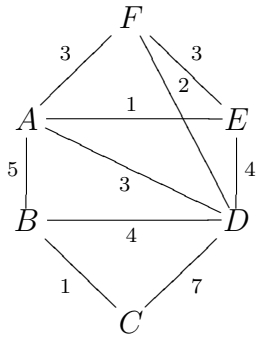
3.1.5. Complejidad Kruskal Delete

No demostraremos el orden pero diremos que la complejidad es: $O\left(m \log(n) \left(\log\left(\log(n)\right)\right)^3\right)$

3.1.6. Ejemplo de Kruskal

	A	B	C	D	E	F
A	–	5	–	3	1	3
B	5	–	1	4	–	–
C	–	1	–	7	–	–
D	3	4	7	–	4	2
E	1	–	–	4	–	3
F	3	–	–	2	3	–





3.2. PRIM

Prim(x) = agregar el lado de menor peso que conecte x con otro vértice, creando una región R luego ir agregando el lado de menor peso que conecte R con $V - R$.

Generalmente se implementa con un heap.

3.2.1. Pseudo Código de Prim

```
Prim(vertice x)
{
  ∀(p ∈ V){
     Costo[p] = ∞;
     Vecino[p] = NULL;
  }

  Costo(x) = 0;
  M = 0;
  Aux = V;

  //FALTA CASO BASE

  while(m < n-1){
     //pre: costo (p)< costo(q)  ∀q ∈ Aux
     ∀(q ∈ Γ(p)){
        if (costo(q)> V(pq)){
           costo(q) = W(pq);
           vecino(q) = p;
        }
     }
     Add(p,vecino(p));
     m++;
  }


}
```

3.2.2. Correctitud de Prim

Como el invariante es ser árbol cuanto termina es efectivamente spanning tree.
Sólo resta ver que es minimal y lo haremos por **inducción**.
Si P_j es el árbol en el paso j, probaremos que $\exists\ MST\ T_j : P_j \subseteq T_j$

- $j = 0$: Trivial.
- Supongamos que vale para j
- Si $P_{j+1} \subseteq T_j$ ya está.

  De lo contrario existe $e \in P_{j+1} : e \notin T_j$ (y de hecho, será $P_{j+1} = P_j \cup \{e\}$).

  e une P_j con $V - P_j$

  Como T_j es Spanning Tree $\Rightarrow$ debe conectar P_j con $V - P_j$.

Sea f un lado en T_j que conecte estas dos regiones.

Como e es el <u>menor</u> lado que conecta esas dos regiones tenemos que:

$$W(e) \leq W(f)$$

Entonces: $W(T_j \cup \{e\} - f) \leq W(T_j)$

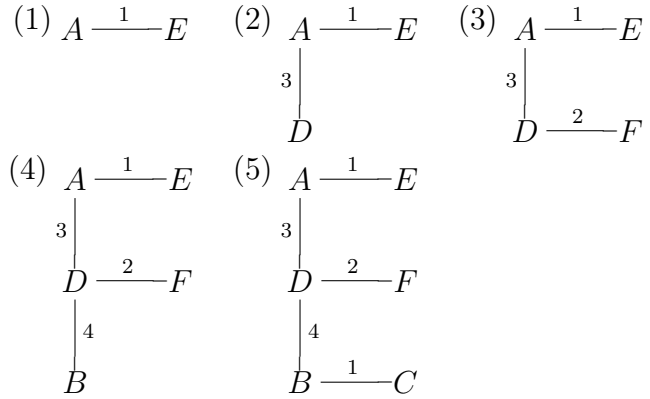
$\therefore T_j \cup \{e\} - f$ es un MST que contiene a P_{j+1}.

■

3.2.3. Ejemplo de Prim

	A	B	C	D	E	F
A	–	5	–	3	1	3
B	5	–	1	4	–	–
C	–	1	–	7	–	–
D	3	4	7	–	4	2
E	1	–	–	4	–	3
F	3	–	–	2	3	–

$Prim(A)$, Método Gráfico:



Prim(A), Método Analítico;

	B	C	D	E	F
1)	$(5, A)$	$(\infty, -)$	$(3, A)$	$\boxed{(1,\text{A})}$	$(3, A)$
2)	$(5, A)$	$(\infty, -)$	$\boxed{(3,\text{A})}$	–	$(3, A)$
3)	$(4, D)$	$(7, D)$	–	–	$\boxed{(2,\text{D})}$
4)	$\boxed{(4,\text{D})}$	$(7, D)$	–	–	–
5)	–	$\boxed{(1,\text{B})}$	–	–	–

$\therefore\ Complejidad(Prim) = O(n^2)$

3.3. Implementaciones de Union Find

1. Linked lists. (listas enlazadas)
2. Representantes.
3. Union Find By Rank.

3.3.1. Union Find con listas enlazadas

Union es $O(1)$
Find es $O(n)$
Recordemos que Kruskal hace: $2m$ "Finds()" y $n-1$ "Unions()".

$$\therefore O(mn) + O(n) = O(nm)$$

Como $mn > \underbrace{m\log(n)}_{\text{Ordenar los lados}}$ no sirve.

3.3.2. Union Find con Representantes

```
Makeset(x){ //O(1)
  R(x) = x;
}

Find(x){ // O(1)
  return R(x);
}

Union(R_1, R_2){ // O(n)
  ∀(x ∈ V){
    if (R(x) == R_2){
      R(x) = R_1;
    }
  }
}
```

$$Compl(Kruskal) = \underbrace{O(m\log(n))}_{\text{Ordenar los lados}} + \underbrace{O(m)}_{\text{m Finds}} + \underbrace{O(n^2)}_{\text{n Unions}}$$

$$\therefore Compl(Kruskal) = O(m\log(n) + n^2) \text{ bastante bien si: } m \approx n^2$$

Es decir no funciona tan bien si es un grafo ralo, i.e. con pocos lados.

3.3.3. Union Find con ranking

```
Makeset(vertice x){
  Π(x) = x;
  Rk(x) = 0; //Ranking
}
Find(vertice x){ //recursivo a la cola
  if (Π(x) == x)
    return x;
  else return find(Π(x))
}
Find(vertice x){ //iterativo
  y = x;
  while(y ≠ Π(y))
    y = Π(y);
  return y;
}
/* Pre: F1 = Find(x[1]); F2 = Find(x[2]); F1 ≠ F2 */
Union(F1,F2){
  k1 = Rank(F1);
  k2 = Rank(F2);
  if(K1 > K2){
    Π(F2) = F1; //cabeza del 2do apunta al primero
  }else{
    Π(F1) = F2; //cabeza del primero apunta al segundo
    if (k1 == k2){
      Rank(F2) = k2 +1;
    }
  }
}
```

Con esta implementación:

$$makeset(x) = (1)$$

$$Union = O(1);$$

$$Find(x) = O(rank(representante(x)) \leq O(\log(n))$$

<u>Teorema</u> Find es $O(\log(n))$

<u>Corolario</u>: $Compl(Kruskal) = O(m\log(n))$
<u>Demostración</u>:

$$Compl(Kruskal) = \underbrace{O(m\log(n))}_{\text{Ordenar los lados}} + \underbrace{O(m)}_{\text{m Finds}} + \underbrace{O(n^2)}_{\text{n Unions}} = O(m\log(n))$$

<u>Demostración del Teorema</u>

- <u>Lema interno 1</u>: $(Rank(x) < Rank(\Pi(x)) : \forall x : x \neq \Pi(x))$
  Obvio pues Rank solo cambia con Union, y lo cambia sólo a F_2 cuando $\Pi(F_1) == F_2$
  En ese caso incrementa el rango de F_2

- <u>Lema interno 2</u>: $Rank(x) = k \Rightarrow \exists \geq 2^k$ elementos "por debajo" de x, contando a x.

  Prueba por **inducción**.

  - $k = 0$ obvio
  - Supongo que vale para k
  - Veamos para $k + 1$

    $Rank(x) = k + 1 \Rightarrow$ alguna vez fue k y union lo subió a $k + 1$ y esto sólo pasa si unimos 2 de rango k.

    Entonces la cantidad de elementos por debajo de $x \geq 2^k + 2^k = 2^{k+1}$

- <u>Lema interno 3</u>: Hay $\leq \frac{n}{2^k}$ elementos de rango k, está demostrado en los lemas.

- **Finalmente** $Rango(x) \leq \log(n)\ \forall x$

  Pues sea k el mayor rango $\Rightarrow \exists$ por lo menos un elemento de rango k.

  $\Rightarrow 1 \leq \frac{n}{2^k}$

  $\Rightarrow 2^k \leq n$

  $\therefore k \leq \log_2(n)$.

■

¿Qué pasa si los lados vienen ordenados o se pueden ordenar[21] en O(m)?

Se puede usar "Path compresion"
Solo cambia Find

```
Find(x){
   if(Π(x) ≠ x){
         Π(x) = Find(Π(x));
   }
   return Π(x);
}
```

El análisis es más complicado, <u>cada</u> $Find()$ puede ser $O(\log(n))$
Se puede ver:

<u>Teorema</u>: la complejidad de m Finds + n Unions, está acotado por $O\Big(m \log^*(n)\Big)$

Esencialmente: $O(m)\ \forall n$ "razonable".
$O \log^*(n) = \#$ de veces que hay que aplicar log para llegar a algo ≤ 1

n	1	2	{3,4}	{5,...,16}	{17,...,2^{16}}	{2^{16}+1,...,2^{65536}}
$\log(n)$	0	1	2	3	4	5

[21] Bucket sort.

4. Matchings

Dado un grafo G, un matching en G es un subgrafo M tal que el grado en M de todos sus vértices es 1.

Un matching es **perfecto** si "cubre" a G, es decir: $V(M) = V(G)$.

De ahora en más supondremos que G es bipartito.

Cuando escribamos $G = (X \cup Y, E)$ asumimos que X e Y son las partes.

- Es decir: $pq \in E \Rightarrow p \in X, q \in Y$

<u>Problema</u>: hallar un matching perfecto en G, o en su defecto uno "maximal" (con la mayor cantidad de lados posibles).

4.1. Transformación del problema de Matching a uno de flujo maximal

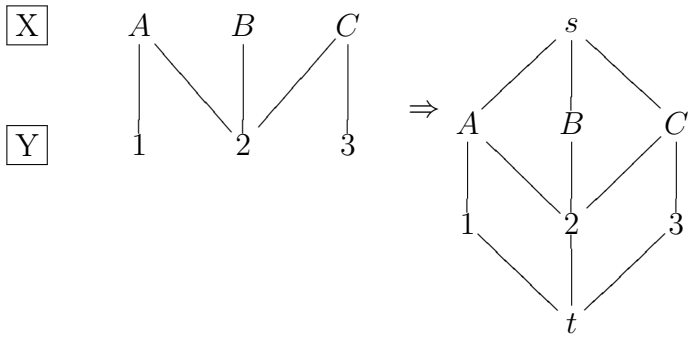
Dado $G = (X \cup Y, E)$ creamos el siguiente network.

$N = \left(X \cup Y \cup \{s,t\}, \vec{E}, C\right)$

$$\vec{E} = \left\{\overrightarrow{sx}\right\}_{x\in X} \bigcup \left\{\overrightarrow{xy} : \begin{array}{c} x \in X \\ y \in Y \\ xy \in E \end{array}\right\} \bigcup \left\{\overrightarrow{yt}\right\}_{y\in Y}$$

La capacidad de todos los lados es : 1.

<u>Ejemplo</u>



<u>Teorema</u>

Flujos maximales en N corresponden con matchings maximales en G y $V(f_{max}) = \#lados$.

Dicho de otra manera: existe una correspondencia entre matchings maximales en G (bipartito) y flujos maximales <u>enteros</u> en N.

<u>Prueba</u>: dado un flujo entero f construimos un matching M_f y dado un Matching M construimos un flujo entero f_M.

Dado f sea $M_f = (V_f, E_f)$: $\begin{array}{l} V_f = \bigcup\limits_{xy\in E_f} \{x,y\} \\ E_f = \{xy \in E : f(\overrightarrow{xy}) = 1\} \end{array}$

Hay que ver que M_f es matching.

Observemos que por definición de V_f tenemos que $\delta_{M_f}(z) \geq 1 \ \forall z \in V_f$

$$\text{Si } M_f \text{ no es matching } \Rightarrow \exists z : \delta_{M_f}(z) \geq 2$$

- <u>Caso 1</u>: $z \in X$ como $\delta_{M_f}(z) \geq 2$ entonces $\exists y_1, y_2 : \begin{array}{l} y_1 \neq y_2 \\ zy_1, zy_2 \in E_f \end{array}$

  Entonces $f(\overrightarrow{zy_1}) = f(\overrightarrow{zy_2}) = 1$

  $$\begin{array}{l} \therefore \ Out_f(z) \geq 2 \\ \text{Pero } In_f(z) = f(\overrightarrow{sz}) \leq 1 \end{array} \Rightarrow \textbf{Absurdo}.$$

- <u>Caso 2</u>: $z \in Y \Rightarrow \exists x_1, x_2 : \begin{array}{l} x_1 \neq x_2 \\ x_1z, x_2z \in E_f \end{array}$

  Entonces $f(\overrightarrow{x_1z}) = f(\overrightarrow{x_2z}) = 1$

  $$\begin{array}{l} \therefore In_f(z) \geq 2 \\ \text{Pero } Out_f(z) = f(\overrightarrow{zt}) \leq 1 \end{array} \Rightarrow \textbf{Absurdo}$$

Concluimos que M_f es matching. ■

Dado ahora un Matching M sea f_M definido así:

$$f_M(\overrightarrow{xy}) = \begin{cases} 1 & \text{Si } xy \in E(M) \\ 0 & \text{Si no} \end{cases}$$

$$f_M(\overrightarrow{sx}) = \begin{cases} 1 & \text{Si } x \in V(M) \cap X \\ 0 & \text{Si no} \end{cases}$$

$$f_M(\overrightarrow{yt}) = \begin{cases} 1 & \text{Si } y \in V(M) \cap Y \\ 0 & \text{Si no} \end{cases}$$

Como M es matching, está claro que:

$$M_f(z) = Out_f(z) = \begin{cases} 1 & \text{Si } z \in V(M) \\ 0 & \text{Si } z \notin V(M) \end{cases}$$

Además está claro que: $M_{f_M} = M$ y que: $f_{M_f} = f$

Veamos ahora que: $V(f) = |E_{M_f}|$

$$\begin{aligned}
V(f) &= Out_f(s) - \overbrace{In_f(s)}^{=0} \\
&= \sum_{x \in X} f(\overrightarrow{sx}) \\
&= \#\{x : f(\overrightarrow{sx}) = 1\} \\
&= \#\{x : In_f(x) = 1\} \\
&= \#\{x : Out_f(x) = 1\} \text{ por ser } f \text{ un flujo} \\
&= |V_f \cap X| \\
&= |E_{M_f}|
\end{aligned}$$

Por lo tanto flujos maximales enteros corresponden a matchings maximales.

■

<u>Definición</u>: Sea $S \subseteq V \Rightarrow \Gamma(S) = \bigcup_{z \in S} \Gamma(z) = \{w \in V : \exists z \in S : wz \in E\}$

<u>Definición</u>: Si $G = (X \cup Y, E)$ es bipartito, un matching se dice **completo** de X a Y si cubre a X, i.e. $X \subseteq V(M)$.

4.2. Teorema de Hall

Sea $G = (X \cup Y, E)$ bipartito entonces:

$\exists$ un matching completo de X a $Y \Leftrightarrow : \left|\Gamma(S)\right| \geq \left|S\right| \; \forall S \subseteq X$

4.3. Complejidad del Algoritmo de Matching

Asumiendo $|X| = |Y| = n$

- Matching inical: $O(n^2)$.
- Extender el matching en un lado: $O(n^2)$.
- Extenderlo $O(n)$ veces.

$\therefore$ Nos queda: $O(n^3)$

4.4. Grafos con Peso

Sea $G = (X \cup Y, E)$ grafo bipartito, con $|X| = |Y| = n$ y sea C una matriz de costos $n * n$

$C_{x,y}$ = Costo de asignar el trabajo X al trabajador Y.[22]

Hay 2 (al menos) formas de minimizar:

1. Minimizar el máximo.

[22] A veces pueden ser "Ganancias".

2. Mimizar la suma.

El problema 1 a veces es llamado el: bottleneck problem[23], se resuelve con el algoritmo de "Gross" y es bastante simple.

4.4.1. Algoritmo de "Gross"

Dado C:

1. Ordenar los $C_{i,j}$ como : $b_1 < b_2 < \ldots < b_k$ $(k \leq n^2)$
2. Tomar, para r= 1, . . . , k:

$$(A_r)_{i,j} = \begin{cases} 1 & \text{Si } C_{i,j} \leq b_r \\ 0 & \text{Si no} \end{cases} \quad (r \text{ es el "umbral"})$$

3. Hallar r: A_r tenga Matching perfecto y A_{r-1} no, eso da un matching M tal que: $\text{máx}\{C_{i,j} : \{i,j\} \in E(M)\} \leq \text{máx}\{C_{i,j} : \{i,j\} \in E(\widetilde{M})\}\ \forall \widetilde{M}$

[23]Problema de cuello de botella.

4.4.2. Ejemplo del algoritmo de "Gross"

Hallar un Matching que minimice el Máximo

	I	II	III	IV	V	VI	VII
A	1	2	15	2	7	5	4
B	π	e	e^2	5	8	2	1
C	9	2	1	10	8	100	9
D	e^2	1	2	10	4	10	10
E	10	9	7	8	1	9	10
F	1	2	3	4	5	6	7
G	8	2	9	10	5	10	10

Ordeno los $C_{i,j}$: $1, 2, e, 3, \pi, 4, 5, 6, 7, e^2, 8, 9, 10, 15, 1000$

$\text{máx}\{\text{mín}(filas); \text{mín}(columnas)\} = \text{máx}\{1,1,1,1,1,1,2;1,1,1,2,1,2,1\} = 2$

- <u>Umbral 7</u>

Busco Matching

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	
B	1	$\boxed{1}$	0	1	0	1	1	
C	0	1	$\boxed{1}$	0	0	0	0	
D	0	1	1	0	$\boxed{1}$	0	0	
E	0	0	1	0	1	0	0	$\star$
F	1	1	1	$\boxed{1}$	1	1	1	
G	0	1	0	0	1	0	0	$\star$

Intento extenderlo

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	
B	1	$\boxed{1}$	0	1	0	1	1	
C	0	1	$\boxed{1}$	0	0	0	0	
D	0	1	1	0	$\boxed{1}$	0	0	
E	0	0	1	0	1	0	0	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	
G	0	1	0	0	1	0	0	$\cancel{\star}$
		G	E		E			

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	
B	1	$\boxed{1}$	0	1	0	1	1	II
C	0	1	$\boxed{1}$	0	0	0	0	III
D	0	1	1	0	$\boxed{1}$	0	0	V
E	0	0	1	0	1	0	0	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	
G	0	1	0	0	1	0	0	$\cancel{\star}$
		$\cancel{G}$	$\cancel{E}$		$\cancel{E}$			

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	
B	1	$\boxed{1}$	0	1	0	1	1	$\cancel{II}$
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$
D	0	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$
E	0	0	1	0	1	0	0	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	
G	0	1	0	0	1	0	0	$\cancel{\star}$
	B	$\cancel{G}$	$\cancel{E}$	B	$\cancel{E}$	B	B	

Extiendo el Matching

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	I
B	1	$\cancel{1}$	0	1	0	$\boxed{1}$	1	$\cancel{II}$
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$
D	0	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$
E	0	0	1	0	1	0	0	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	IV
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	B	B	

	I	II	III	IV	V	VI	VII		
A	$\boxed{1}$	1	0	1	1	1	1	$\cancel{I}$	
B	1	$\cancel{\boxed{1}}$	0	1	0	$\boxed{1}$	1	$\cancel{II}$	
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$	
D	0	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$	
E	0	0	1	0	1	0	0	$\cancel{\star}$	$\star$
F	1	1	1	$\boxed{1}$	1	1	1	$\cancel{IV}$	
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$	
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	$\cancel{B}$		

	I	II	III	IV	V	VI	VII		
A	$\boxed{1}$	1	0	1	1	1	1	$\cancel{I}$	
B	1	$\cancel{\boxed{1}}$	0	1	0	$\boxed{1}$	1	$\cancel{II}$	
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$	
D	0	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$	
E	0	0	1	0	1	0	0	$\cancel{\star}$	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	$\cancel{IV}$	
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$	
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	$\cancel{B}$		
			E		E				

	I	II	III	IV	V	VI	VII		
A	$\boxed{1}$	1	0	1	1	1	1	$\cancel{I}$	
B	1	$\cancel{\boxed{1}}$	0	1	0	$\boxed{1}$	1	$\cancel{II}$	
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$	III
D	0	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$	V
E	0	0	1	0	1	0	0	$\cancel{\star}$	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	$\cancel{IV}$	
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$	
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	$\cancel{B}$		
			$\cancel{E}$		$\cancel{E}$				

	I	II	III	IV	V	VI	VII		
A	$\boxed{1}$	1	0	1	1	1	1	$\cancel{I}$	
B	1	$\cancel{\boxed{1}}$	0	1	0	$\boxed{1}$	1	$\cancel{II}$	
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$	$\cancel{III}$
D	0	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$	$\cancel{V}$
E	0	0	1	0	1	0	0	$\cancel{\star}$	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	$\cancel{IV}$	
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$	
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	$\cancel{B}$		
		C	$\cancel{E}$		$\cancel{E}$				

	I	II	III	IV	V	VI	VII		
A	$\boxed{1}$	1	0	1	1	1	1	$\cancel{I}$	
B	1	$\cancel{\boxed{1}}$	0	1	0	$\boxed{1}$	1	$\cancel{II}$	
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$	$\cancel{III}$
D	0	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$	$\cancel{V}$
E	0	0	1	0	1	0	0	$\cancel{\star}$	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	$\cancel{IV}$	
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$	II
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	$\cancel{B}$		
		$\cancel{C}$	$\cancel{E}$		$\cancel{E}$				

- <u>Obtengo</u>:
  - $S = \{C, D, E, G\}$ = Filas etiqueteadas.
  - $\Gamma(S) = \{II, III, V\}$ = Columnas etiqueteadas.
  - Luego $|\Gamma(S)| = 3 < 4 = |S| \ \therefore \nexists$ Matching Perfecto.
    Entonces tengo que subir el umbral.

- <u>Umbral 9</u>

	I	II	III	IV	V	VI	VII
A	1	1	0	1	1	1	1
B	1	1	1	1	1	1	1
C	1	1	1	0	1	0	1
D	1	1	1	0	1	0	0
E	0	1	1	1	1	1	0
F	1	1	1	1	1	1	1
G	1	1	1	0	1	0	0

Busco Matching

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	
B	1	$\boxed{1}$	1	1	1	1	1	
C	1	1	$\boxed{1}$	0	1	0	1	
D	1	1	1	0	$\boxed{1}$	0	0	
E	0	1	1	$\boxed{1}$	1	1	0	
F	1	1	1	1	1	$\boxed{1}$	1	
G	1	1	1	0	1	0	0	$\star$

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	
B	1	$\boxed{1}$	1	1	1	1	1	
C	1	1	$\boxed{1}$	0	1	0	1	
D	1	1	1	0	$\boxed{1}$	0	0	
E	0	1	1	$\boxed{1}$	1	1	0	
F	1	1	1	1	1	$\boxed{1}$	1	
G	1	1	1	0	1	0	0	$\not\star$
	G	G	G		G			

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	I
B	1	$\boxed{1}$	1	1	1	1	1	II
C	1	1	$\boxed{1}$	0	1	0	1	III
D	1	1	1	0	$\boxed{1}$	0	0	V
E	0	1	1	$\boxed{1}$	1	1	0	
F	1	1	1	1	1	$\boxed{1}$	1	
G	1	1	1	0	1	0	0	$\not\star$
	$\cancel{G}$	$\cancel{G}$	$\cancel{G}$		$\cancel{G}$			

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	$\cancel{I}$
B	1	$\boxed{1}$	1	1	1	1	1	$\cancel{II}$
C	1	1	$\boxed{1}$	0	1	0	1	$\cancel{III}$
D	1	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$
E	0	1	1	$\boxed{1}$	1	1	0	
F	1	1	1	1	1	$\boxed{1}$	1	
G	1	1	1	0	1	0	0	$\not\star$
	$\cancel{G}$	$\cancel{G}$	$\cancel{G}$	A	$\cancel{G}$	A	A	

Extiendo el Matching

	I	II	III	IV	V	VI	VII	
A	$\cancel{\boxed{1}}$	1	0	1	1	1	$\boxed{1}$	$\cancel{I}$
B	1	$\boxed{1}$	1	1	1	1	1	$\cancel{II}$
C	1	1	$\boxed{1}$	0	1	0	1	$\cancel{III}$
D	1	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$
E	0	1	1	$\boxed{1}$	1	1	0	
F	1	1	1	1	1	$\boxed{1}$	1	
G	$\boxed{1}$	1	1	0	1	0	0	$\not\star$
	$\cancel{G}$	$\cancel{G}$	$\cancel{G}$	$\cancel{A}$	$\cancel{G}$	$\cancel{A}$	A	

Existe Matching $\therefore$ hay que bajar el umbral:

M = { A-VII:4 ; B-II:e ; C-III:1 ; D-V:4 ; E-IV:8 ; F-VI:6 ; G-I:8 }

$\therefore$ Tengo un Matching de 8, entonces no hago el umbral de 8, sólo basta ver si hay Matching de $1'$s con umbral e^2.

- <u>Umbral de e^2</u>

	I	II	III	IV	V	VI	VII	
A	1	1	0	1	1	1	1	
B	1	1	1	1	0	1	1	
C	0	1	1	0	0	0	0	
D	1	1	1	0	1	0	0	
E	0	0	1	0	1	0	0	
F	1	1	1	1	1	1	1	
G	0	1	0	0	1	0	0	

Busco Matching

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	
B	1	$\boxed{1}$	1	1	0	1	1	
C	0	1	$\boxed{1}$	0	0	0	0	
D	1	1	1	0	$\boxed{1}$	0	0	
E	0	0	1	0	1	0	0	$\star$
F	1	1	1	$\boxed{1}$	1	1	1	
G	0	1	0	0	1	0	0	$\star$

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	
B	1	$\boxed{1}$	1	1	0	1	1	
C	0	1	$\boxed{1}$	0	0	0	0	
D	1	1	1	0	$\boxed{1}$	0	0	
E	0	0	1	0	1	0	0	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	
G	0	1	0	0	1	0	0	$\cancel{\star}$
		G	E		E			

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	
B	1	$\boxed{1}$	1	1	0	1	1	II
C	0	1	$\boxed{1}$	0	0	0	0	III
D	1	1	1	0	$\boxed{1}$	0	0	V
E	0	0	1	0	1	0	0	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	
G	0	1	0	0	1	0	0	$\cancel{\star}$
		$\cancel{G}$	$\cancel{E}$		$\cancel{E}$			

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	
B	1	$\boxed{1}$	1	1	0	1	1	$\cancel{II}$
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$
D	1	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$
E	0	0	1	0	1	0	0	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	
G	0	1	0	0	1	0	0	$\cancel{\star}$
	B	$\cancel{G}$	$\cancel{E}$	B	$\cancel{E}$	B	B	

Extiendo el Matching

	I	II	III	IV	V	VI	VII	
A	$\boxed{1}$	1	0	1	1	1	1	I
B	1	$\boxed{\cancel{1}}$	1	1	0	$\boxed{1}$	1	$\cancel{II}$
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$
D	1	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$
E	0	0	1	0	1	0	0	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1	
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	B	

	I	II	III	IV	V	VI	VII		
A	$\boxed{1}$	1	0	1	1	1	1	I	
B	1	$\cancel{1}$	1	1	0	$\boxed{1}$	1	$\cancel{VI}$	
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$	III
D	1	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$	V
E	0	0	1	0	1	0	0	$\cancel{\star}$	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1		
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$	
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	B		
			$\cancel{E}$		$\cancel{E}$				

	I	II	III	IV	V	VI	VII		
A	$\boxed{1}$	1	0	1	1	1	1	I	
B	1	$\cancel{1}$	1	1	0	$\boxed{1}$	1	$\cancel{VI}$	
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$	$\cancel{III}$
D	1	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$	$\cancel{V}$
E	0	0	1	0	1	0	0	$\cancel{\star}$	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1		
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$	
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	B		
	D	C	$\cancel{E}$		$\cancel{E}$				

	I	II	III	IV	V	VI	VII		
A	$\boxed{1}$	1	0	1	1	1	1	I	I
B	1	$\cancel{1}$	1	1	0	$\boxed{1}$	1	$\cancel{VI}$	
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$	$\cancel{III}$
D	1	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$	$\cancel{V}$
E	0	0	1	0	1	0	0	$\cancel{\star}$	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1		
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$	II
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	B		
	$\cancel{D}$	$\cancel{C}$	$\cancel{E}$		$\cancel{E}$				

	I	II	III	IV	V	VI	VII		
A	$\boxed{1}$	1	0	1	1	1	1	I	$\cancel{I}$
B	1	$\cancel{1}$	1	1	0	$\boxed{1}$	1	$\cancel{VI}$	
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$	$\cancel{III}$
D	1	1	1	0	$\boxed{1}$	0	0	$\cancel{V}$	$\cancel{V}$
E	0	0	1	0	1	0	0	$\cancel{\star}$	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1		
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$	$\cancel{VI}$
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	B		
	$\cancel{D}$	$\cancel{C}$	$\cancel{E}$	A	$\cancel{E}$	A	A		

Extiendo el Matching

	I	II	III	IV	V	VI	VII		
A	$\boxed{\cancel{1}}$	1	0	1	1	1	$\boxed{1}$	I	$\cancel{I}$
B	1	$\boxed{\cancel{1}}$	1	1	0	$\boxed{1}$	1	$\cancel{VI}$	
C	0	1	$\boxed{1}$	0	0	0	0	$\cancel{III}$	$\cancel{III}$
D	$\boxed{1}$	1	1	0	$\boxed{\cancel{1}}$	0	0	$\cancel{V}$	$\cancel{V}$
E	0	0	1	0	$\boxed{1}$	0	0	$\cancel{\star}$	$\cancel{\star}$
F	1	1	1	$\boxed{1}$	1	1	1		
G	0	$\boxed{1}$	0	0	1	0	0	$\cancel{\star}$	$\cancel{VI}$
	$\cancel{B}$	$\cancel{G}$	$\cancel{E}$	$\cancel{B}$	$\cancel{E}$	$\cancel{B}$	B		
	$\cancel{D}$	$\cancel{C}$	$\cancel{E}$	$\cancel{A}$	$\cancel{E}$	$\cancel{A}$	$\cancel{A}$		

Hallé un Matching que minimiza el máximo: (e^2)

$$M = \left\{A - VII : 4; B - VI : 2; C - III : 1; D - I : e^2; E - V : 1; F - IV : 4; G - II : 2\right\}$$

4.5. Prueba del teorema de HALL

Sea $G = (X \cup Y, E)$ grafo bipartito entonces:
existe matching completo de X a $Y \Leftrightarrow \ |S| \leq |\Gamma(S)| \ \forall S \leq X$

Prueba:

$\boxed{\Rightarrow}$ Obvio pues el matching induce una funcion $\psi : 1-1$ de X en Y y la imagen de S por esa función está incluida en $\Gamma(S)$

$$\therefore |S| \overset{\psi 1-1}{=} |\psi(S)| \leq |\Gamma(S)|$$

$\boxed{\Leftarrow}$ Es equivalente a ver que:

$$\text{Si } \nexists \text{ un matching completo} \Rightarrow \exists S \subseteq X : |S| > |\Gamma(S)|$$

Corramos el algoritmo de hallar matching (Filas X, Columnas Y)
Cuando termine habrá filas sin Matchear[24].
Sea $S_0 =$ Filas sin Matchear.
Sea $T_1 = \Gamma(S_0)$ son las columnas etiqueteadas por las filas de S_0
$S_1 =$ Filas etiqueteadas por las columas de T_1
$T_2 =$ columnas etiqueteadas por las filas de S_1.
No necesariamente $T_2 \neq \Gamma(S_1)$.
Pero si tenemos que $T_1 \cup T_2 = \Gamma(S_0 \cup S_1)$
Y así seguimos:
Nos queda definido:

$$S_j = \text{Filas etiqueteadas por las columnas de } T_j$$

$T_{j+1} =$ las columnas etiqueteadas por las filas de S_j
$S_j =$ filas Matcheadas con T_j.
$\therefore \ Si \ T_j \neq \emptyset \Rightarrow S_j$ también.
El algoritmo se detiene sin extender el matching (por hipótesis).
Por lo tanto se detiene en un k con $T_{k+1} = \emptyset$
Observación:

1. $|S_j| = |T_j| \ \forall j$ Pues S_j son las filas Matcheadas con T_j
2. $\Gamma(S_0 \cup \ldots \cup S_j) = T_1 \cup T_2 \cup \ldots \cup T_{j+1}$

Sea $S = S_0 \cup S_1 \cup \ldots \cup S_k$

$$\begin{aligned}
|\Gamma(S)| &= |\Gamma(S_0 \cup \ldots \cup S_k)| \\
&= |T_1 \cup \ldots \cup T_k \cup T_{k+1}| && (17)\\
&= |T_1 \cup \ldots \cup T_k| && (18)\\
&= |T_1| + \ldots + |T_k| && (19)\\
&= |S_1| + \ldots + |S_k| && (20)\\
&= |S| - |S_0| \\
&< |S|
\end{aligned}$$

[24] Las marcadas con $\star$.

(17) Por las observación 2.
(18) $T_{k+1} = \emptyset$
(19) Unión Disjunta.
(20) Por las observación 1. ■

4.6. Teorema del Matrimonio

Todo grafo bipartito regular (con lados) tiene un matching perfecto.

<u>Prueba</u>: sea $G = (X \cup Y, E) : E \neq \emptyset$, regular.
Dado $W \subseteq V$ con $W \subseteq X$ ó $W \subseteq Y$
<u>Definimos</u>:

$$\begin{aligned} E_w &= \text{Lados con un extremo en } w \\ &= \{xy \in E : x \in W \text{ó } y \in W\} \end{aligned}$$

Supongamos $W \subseteq X$

$$\begin{aligned} |E_w| &= \Big|\{xy \in E : x \in w\}\Big| \\ &= \sum_{x \in w} \Big|\{y : xy \in E\}\Big| \\ &= \sum_{x \in w} \underbrace{d(x)}_{\Delta} \qquad (21) \\ &= \Delta * |w| \end{aligned}$$

(21) Por ser G regular.
Similar prueba si: $W \subseteq Y$
Observemos que por ser G bipartito: $E_x = E = E_y$
$\therefore |E_x| = |E_y| \Rightarrow \Delta * |X| = \Delta * |Y|$
Como $\Delta \neq 0 \Rightarrow |X| = |Y|$
Por lo tanto para ver si existe un matching perfecto en G, basta ver que existe un matching completo de X a Y.
Por el teorema de Hall basta entonces con ver: $|S| \leq |\Gamma(S)| \ \forall S \subseteq X$
Sea entonces $S \subseteq X$

$$\text{Sea } e \in E_s \Rightarrow \exists \begin{array}{l} x \in S \\ y \in Y \end{array} : e = xy$$

$\Rightarrow y \in \Gamma(x) \subseteq \Gamma(S)$
$\Rightarrow e = x\boxed{\text{y}} \in E_{\Gamma(s)}$

$$\therefore \text{Hemos probado que: } E_s \subseteq E_{\Gamma(S)}$$

$$\begin{aligned} \Rightarrow |E_s| &\leq |E_{\Gamma(S)}| \\ \Delta|S| &\leq \Delta|\Gamma(S)| \\ \therefore |S| &\leq |\Gamma(S)| \end{aligned}$$

■

4.7. Minimizar la Suma

Problema: dada una matriz de costos C, hallar el menor M, Matching.

Simbólicamente buscamos M: $M = \text{mín} \sum_{\substack{i=1 \\ j:ij \in M}}^{n} C_{i,j}$

La solución a este problema usa una idea sencilla: sea $\widetilde{C}$ una matriz de costos que se obtiene de C restándole o sumándole una constante a una fila o columna.

Entonces sea $M_{\text{mín}}(C)$ un Matching que minimiza la suma.

Si a la fila $\hat{i}$ le resto una constante k, i.e. $\widetilde{C}_{i,j} \begin{cases} C_{i,j} & \text{Si } i \neq \hat{i} \\ C_{i,j} - k & \text{Si } i = \hat{i} \end{cases}$

Está claro que: $\sum_{\substack{i=1 \\ j:ij \in M}}^{n} \widetilde{C}_{i,j} = \left(\sum_{\substack{i=1 \\ j:ij \in M}}^{n} C_{i,j} \right) - k$, similar para las columnas.

$$\therefore M_{\text{mín}}(C) \text{ también minimiza } \widetilde{C}$$

Segunda Observación: si $C_{i,j} \geq 0$.

Entonces si M es un matching de $0's$ es decir $C_{i,j} = 0 \ \forall i, j \in M \Rightarrow$ M minimiza la suma.

Idea básica: restar o sumar fila "inteligentemente" hasta hallar un matching de ceros. Ese matching automáticamente minimiza la matriz original.

4.7.1. Ejemplo de Matching que minimiza la Suma (A ojo)

	I	II	III	IV
A	2	5	7	7
B	9	10	5	11
C	1	6	7	10
D	10	1	4	4

Restamos a cada fila su mínimo

	I	II	III	IV
A	0	3	5	5
B	4	5	0	6
C	0	5	6	9
D	9	0	3	3

Restamos a cada columna su mínimo

	I	II	III	IV
A	0	3	5	2
B	4	5	0	3
C	0	5	6	6
D	9	0	3	0

$\nexists$ Matching de $0's$.

Restamos 5 a la tercer fila.

	I	II	III	IV
A	0	3	5	2
B	4	5	0	3
C	-5	0	1	1
D	9	0	3	0

Existe Matching Pero tengo un número Negativo.

Sumamos 5 a la columna I

	I	II	III	IV
A	5	3	5	2
B	9	5	0	3
C	0	0	1	1
D	14	0	3	0

Restamos el mínimo a cada fila.

	I	II	III	IV
A	3	1	3	0
B	9	5	0	3
C	0	0	1	1
D	14	0	3	0

Hallé un Matching de 0′s.

	I	II	III	IV
A	3	1	3	$\boxed{0}$
B	9	5	$\boxed{0}$	3
C	$\boxed{0}$	0	1	1
D	14	$\boxed{0}$	3	0

$$M = \left\{AIV : 7; BIII : 5; CI : 1; DII : 1\right\}$$

$\therefore$ Hallé un matching que minimiza la suma(14).

4.7.2. Algoritmo Húngaro

Algoritmo de Khun-Munkres, conocido como algoritmo húngaro.

1. Restar el mínimo de cada fila a cada fila.
2. Restar el mínimo de cada columna a cada columna.
3. Buscar Matchings de $0's$
   - Si existe hallé un matching que minimiza la suma.
   - Sino tomar S: $|S| > |\Gamma(S)|$, y restarle un cierto m a las filas de S y sumarle m a las columnas de $\Gamma(S)$
     - ¿Cuál m?
     - $m = \text{mín}\, S \cap \overline{\Gamma(S)}$
   - Equivalentemente: restar m a $S \cap \Gamma(S)$ y sumar m a $\overline{S} \cap \Gamma(S)$

4.7.3. Ejemplo del algoritmo Húngaro

$$\begin{array}{c} \\ A \\ B \\ C \\ D \end{array} \begin{array}{c} \begin{array}{cccc} I & II & III & IV \end{array} \\ \begin{pmatrix} 2 & 5 & 7 & 7 \\ 9 & 10 & 5 & 11 \\ 1 & 6 & 7 & 10 \\ 10 & 1 & 4 & 4 \end{pmatrix} \end{array}$$

1. Restamos a cada fila su mínino:

$$\begin{array}{c} \\ A \\ B \\ C \\ D \end{array} \begin{array}{c} \begin{array}{cccc} I & II & III & IV \end{array} \\ \begin{pmatrix} 0 & 3 & 5 & 5 \\ 4 & 5 & 0 & 6 \\ 0 & 5 & 6 & 9 \\ 9 & 0 & 3 & 3 \end{pmatrix} \end{array}$$

2. Restamos a cada columna su mínimo:

$$\begin{array}{c} \\ A \\ B \\ C \\ D \end{array} \begin{array}{c} \begin{array}{cccc} I & II & III & IV \end{array} \\ \begin{pmatrix} 0 & 3 & 5 & 2 \\ 4 & 5 & 0 & 3 \\ 0 & 5 & 6 & 6 \\ 9 & 0 & 3 & 0 \end{pmatrix} \end{array}$$

3. Buscamos Matching de $0's$

	I	II	III	IV	
A	$\boxed{0}$	3	5	2	
B	4	5	$\boxed{0}$	3	
C	0	5	6	6	*
D	9	$\boxed{0}$	3	0	

	I	II	III	IV	
A	$\boxed{0}$	3	5	2	
B	4	5	$\boxed{0}$	3	
C	0	5	6	6	$\cancel{*}$
D	9	$\boxed{0}$	3	0	
	C				

	I	II	III	IV	
A	$\boxed{0}$	3	5	2	I
B	4	5	$\boxed{0}$	3	
C	0	5	6	6	$\cancel{*}$
D	9	$\boxed{0}$	3	0	
	$\cancel{C}$				

4. No hay matching perfecto

	I	II	III	IV	
A	$\boxed{0}$	3	5	2	$\cancel{I}$
B	4	5	$\boxed{0}$	3	
C	0	5	6	6	$\cancel{*}$
D	9	$\boxed{0}$	3	0	
	$\cancel{C}$				

Tengo $S = \{A, C\}$, las filas etiqueteadas.

$\Gamma(S) = I$, las columas etiqueteadas.

Con $|S| > |\Gamma(S)|$

5. Calculo: $m = \text{mín}\, S * \overline{\Gamma(S)}$

	I	II	III	IV	
A	$\boxed{0}$	3	5	2	$\cancel{I}$
B	4	5	$\boxed{0}$	3	
C	0	5	6	6	$\cancel{*}$
D	9	$\boxed{0}$	3	0	
	$\cancel{C}$				

$\xrightarrow{m=2}$

$$\begin{array}{c} \\ A \\ B \\ C \\ D \end{array} \begin{array}{c} \begin{array}{cccc} I & II & III & IV \end{array} \\ \begin{pmatrix} 0 & 1 & 3 & 0 \\ 6 & 5 & 0 & 3 \\ 0 & 3 & 4 & 4 \\ 11 & 0 & 3 & 0 \end{pmatrix} \end{array}$$

<u>Referencias</u>:

$\overline{S} \cap \Gamma(s)$	No lo modifico
$S \cap \Gamma(s)$	Sumo m
$\overline{S} \cap \overline{\Gamma(S)}$	No lo modifico
$S \cap \overline{\Gamma(S)}$	Resto m

6. Busco Matching de $0'S$

   a) No encuentro: $\begin{array}{c} \\ A \\ B \\ C \\ D \end{array} \begin{array}{c} \begin{array}{cccc} I & II & III & IV \end{array} \\ \begin{pmatrix} \boxed{0} & 1 & 3 & 0 \\ 6 & 5 & \boxed{0} & 3 \\ 0 & 3 & 4 & 4 \\ 11 & \boxed{0} & 3 & 0 \end{pmatrix} \end{array}$

   b) Intento extenderlo:

	I	II	III	IV	
A	$\boxed{0}$	1	3	0	
B	6	5	$\boxed{0}$	3	
C	0	3	4	4	$\cancel{*}$
D	11	$\boxed{0}$	3	0	
	C				

$\Rightarrow$

	I	II	III	IV	
A	$\boxed{0}$	1	3	0	I
B	6	5	$\boxed{0}$	3	
C	0	3	4	4	$\cancel{*}$
D	11	$\boxed{0}$	3	0	
	$\cancel{C}$				

	I	II	III	IV	
A	$\boxed{0}$	1	3	0	$\cancel{I}$
B	6	5	$\boxed{0}$	3	
C	0	3	4	4	$\cancel{*}$
D	11	$\boxed{0}$	3	0	
	$\cancel{C}$			A	

<u>Extiendo el matching</u>

	I	II	III	IV	
A	$\boxed{\cancel{0}}$	1	3	$\boxed{0}$	$\cancel{I}$
B	6	5	$\boxed{0}$	3	
C	$\boxed{0}$	3	4	4	$\cancel{*}$
D	11	$\boxed{0}$	3	0	
	$\cancel{C}$			A	

7. Llegué al Matching de $0's$: $\begin{array}{c} \\ A \\ B \\ C \\ D \end{array} \begin{array}{c} \begin{array}{cccc} I & II & III & IV \end{array} \\ \begin{pmatrix} 0 & 1 & 3 & \boxed{0} \\ 6 & 5 & \boxed{0} & 3 \\ \boxed{0} & 3 & 4 & 4 \\ 11 & \boxed{0} & 3 & 0 \end{pmatrix} \end{array}$

<u>"A mano"</u>

1. Tachar filas de $\overline{S}$, y las columnas de $\Gamma(S)$.
2. De lo no tachado, calcular el mínimo m.
3. Restar de lo no tachado m.
4. Sumar, a lo tachado dos veces, m.

4.7.4. Complejidad de algoritmo Hungaro

- Restar el mínimo en cada fila: $O(n^2)$.
- Restar el mínimo en cada columna: $O(n^2)$.
- Buscar Matching inicial: $O(n^2)$
- Extender matching en un lado: **"Paso"** $\therefore$ hay $O(n)$ pasos.

Complejidad de cada Paso

- Buscar m, restarlo y sumarlo: $O(n^2)$.
- Buscar Matching en la nueva matriz $O(n^3)$.

¿Cuántas veces buscamos m en un paso? No sabemos

$$\therefore\ Compl(Hungaro) = O(n) * O(n^3) * (\text{No sabemos})$$

Llamémosle "**cambio de matriz**" a buscar el m, restarlo y sumarlo.

Lema: luego de un cambio de matriz el matching parcial que se tenía no se pierde.

Prueba: la idea está en fijarse dónde está el matching.

$$\begin{array}{cc} \left(\begin{array}{c|c} \mathrm{A} & \mathrm{A} \\ \hline \mathrm{B} & \mathrm{C} \end{array}\right) & \begin{array}{c} \overline{S} \\ S \end{array} \\ \begin{array}{cc} \overline{\Gamma(S)} & \Gamma(S) \end{array} & \end{array}$$

Referencias

- A: puede haber ceros.
- B: No hay ceros, no hay matching.
- C: Ceros del Matching.

$\overline{S} \cap \overline{\Gamma(S)}$ y $S \cap \Gamma(S)$ no cambian.

Por lo tanto cualquier 0 de matching ahí permanece.

En $S \cap \overline{\Gamma(S)}$ no hay ceros, por lo tanto no hay matching.

Sólo queda por ver: $\overline{S} \cap \Gamma(S)$, donde puede haber ceros, pero ninguno del matching, pues supongamos que hubiera uno en la columna j, fila i.

$i \notin S, j \in \Gamma(s)$.

Cuando revisamos la columna j buscamos el elemento del matching, $\therefore$ hubieramos agregado la fila i a S, **absurdo**. ■

Corolario: luego de un cambio de matriz, aumenta el matching, (i.e. se termina el paso) o bien aumenta S.

Prueba:

En $S \cap \overline{\Gamma(S)}$ habrá un nuevo cero, digamos en la columna j, al revisar la j si está libre extendemos el matching.

Sino hay una fila i con $\{i, j\} \in M$:matching, por lo tanto agregamos la fila i a S. Entonces S crece.

Observación: i no formaba parte de S, pues por el lema anterior i debe estar en $\overline{S}$.

Corolario: hay a lo sumo n cambios de matriz en cada paso, pues S sólo puede crecer n veces.

Por lo tanto:

$$Compl(Hungaro) = n * Compl(paso)$$

$$Compl(paso) = \underbrace{n}_{\text{cambio de matriz}} *Compl(\text{cambiar la matriz}) + O(n^2)$$

$$Compl(\text{cambiar la matriz}) = O(n^2)$$

$$Compl(paso) = O(n^3)$$

$$Compl(Hungaro) = O(n^4)$$

Sin embargo, siendo "truquero" se puede reducir la complejidad a $O(n^3)$.

$$\therefore Compl(Hungaro) = O(n^3)$$

1. ¿Cómo restar m de $\overline{S}$ y sumar m a $\Gamma(S)$ en tiempo $O(n)$?

   Agregamos etiquetas: cada fila y cada columna tiene una etiqueta que indica cuánto se le suma o resta.

   Cambiar las etiquetas es: $O(n)$.

   <u>Costo</u>: en vez de chequear **if**$(C_{i,j} == 0)$ hay que revisar: $(C_{i,j} - ET_i + ET_j == 0)$

   Acá suponemos que el costo de sumar y restar es $O(1)$.

2. ¿Cómo calcular mín$\{S \cap \overline{\Gamma(S)}\}$ en tiempo $O(n)$?

   Tendremos una etiqueta ψ que por cada columna guardará el mínimo, de la intersección de esa columna con S.

   $\therefore$ Verificar el mínimo de $\{S \cap \overline{\Gamma(S)}\}$ es simplemente calcular:
   $\text{mín}\{\psi(y) : \psi(y) \neq 0\} = O(n)$ ya que $\psi(y) == 0 \Leftrightarrow y \in \Gamma(S)$

   Y los $\psi(y)$ los vamos actualizando a medida que examinamos las filas de S.

4.7.5. EJEMPLO DEL MEJORADO (FALTA)

5. Códigos de correción de errores

En general un código, sobre un alfabeto A, de longitud n es un subconjunto de A^n.
En general usaremos: $A = \{0, 1\}$.
Llamaremos "Código", a un código binario de bloque[25].
Definición: dados $x, y \in \{0, 1\}^n$ la distancia de Hamming entre x e y es:

$$d_H(x, y) = d(x, y) = \#\{i : x_i \neq y_i\}$$

El peso de Hamming de $x \in \{0, 1\}^n$ es: $W(x) = |x| = \#\{i : x_i \neq 0\} = d(x, 0^n)$
Claramente: $d(x, y) = d(x \oplus y, 0) = |x \oplus y| \Rightarrow (x \oplus y)_i = x_i \oplus y_i$ i.e. $\oplus =$ suma en $\mathbb{Z}_2^n$
Con $\oplus$ xor definido de la siguiente manera:

$$\begin{aligned} 0 \oplus 0 &= 0 \\ 0 \oplus 1 &= 1 \\ 1 \oplus 0 &= 1 \\ 1 \oplus 1 &= 0 \end{aligned}$$

es decir la suma módulo 2.
Propiedad d es una distancia.
Prueba:

1. $d(x, x) = 0$
2. $d(x, y) = 0 \Rightarrow x = y$
3. $d(x, y) = d(y, x)$
4. Desigualdad triangular: $d(x, y) \leq d(x, z) + d(z, y)\ \forall z$

   Sean: $A = \{i : x_i = y_i\}$, $B = \{i : x_i = z_i\}$, $C = \{i : y_i = z_i\}$.

   Si $\left.\begin{matrix} x_i = z_i \\ y_i = z_i \end{matrix}\right\} \Rightarrow x_i = y_i$

   $\therefore i \in B \cap C \Rightarrow i \in A$, es decir $(B \cap C) \subseteq A$

   Aplico De morgan: $(B \cap C) \subseteq A$ y obtengo:

$$\begin{aligned} \overline{A} &\subseteq \overline{B} \cup \overline{C} \\ |\overline{A}| &\leq |\overline{B} \cup \overline{C}| \\ &\leq |\overline{B}| + |\overline{C}| \\ \therefore d(x, y) &\leq d(x, z) + d(z, y) \end{aligned}$$

■

[25] "Block code": código de bloque.

Definiciones

- $B_r(x) = \{y \in \mathbb{Z}_2^n : d(x,y) \leq r\}$
- Un código C **detecta** r errores si: $C \cap B_r(x) = \{x\}\ \forall x \in C$
- Un código C **corrige** t errores si: $B_t(x) \cap B_t(y) = \emptyset\ \forall x, y \in C : x \neq y$

Notación: $\delta = \text{mín}\{d(x,y) : x \neq y : x, y \in C\}$
Ejemplos:

1. $C_1 = \{00, 01, 10, 11\} \Rightarrow \delta(C_1) = 1$
2. $C_2 = \{000, 011, 110, 101\}$

C_2 \ C_2	000	011	110	101
000		2	2	2
011			2	2
110				2
101				

$\Rightarrow \delta(C_2) = 2$

3. $C_3 = \{000000, 010101, 101010, 111111\}$

C_3 \ C_3	000000	010101	101010	111111
000000		3	3	6
010101			6	3
101010				3
111111				

$\Rightarrow \delta(C_3) = 3$

Teorema: sea C código binario y $d = \delta(C)$ entonces:

1. C detecta $d-1$ errores y no detecta d errores.
2. C corrige $\lfloor \frac{d-1}{2} \rfloor$ errores y no corrige $\lfloor \frac{d-1}{2} \rfloor + 1$ errores.

Prueba:

1. Detecta $d-1$:

   Recordemos que δ es la "menor distancia entre palabras del código "

   Sea $x \in C, y \in B_{d-1}(x) \cap C \overset{?}{\Rightarrow} x = y$

   $\left.\begin{array}{c} d(x,y) \leq \delta - 1 < \delta \\ \text{Pero } x, y \in C \end{array}\right. \Rightarrow x = y$

   No detecta d:

   Sean $\left.\begin{array}{c} x, y \in C : x \neq y \\ d(x,y) = \delta \end{array}\right\} \Rightarrow \left.\begin{array}{c} y \in B_d(x) \cap C \\ Como\ y \neq x \end{array}\right\} \Rightarrow$no detecta d errores.

2. Corrige $\lfloor \frac{d-1}{2} \rfloor$ errores:

   Sea $t = \lfloor \frac{d-1}{2} \rfloor$.

   Sean $x, y \in C, x \neq y$.

   Supongamos $B_t(x) \cap B_t(y) \neq \emptyset \Rightarrow \exists z \in B_t(x) \cap B_t(y)$

$$\begin{aligned} \delta &\leq d(x,y) && (22) \\ &\leq d(x,z) + d(z,y) \\ &\leq 2t && (23) \\ &\leq \delta - 1 \\ \therefore 0 &\leq -1 \textbf{ absurdo} \end{aligned}$$

   (22) Ya que $x \neq y$ y las dos están en C.

   (23) Ya que z está en las dos Bolas.

   No corrige $\lfloor \frac{d-1}{2} \rfloor + 1$ errores:

   Sean $x, y \in C : d(x,y) = d$, sea $e = x \oplus y$

   Entonces e tiene d unos.

   Sea e' con $\lfloor \frac{d-1}{2} \rfloor + 1$ unos de estos.

   Sea $z = x \oplus e'$

   Quiero ver que $z \in B_{\lfloor \frac{d-1}{2} \rfloor + 1}(x) \cap B_{\lfloor \frac{d-1}{2} \rfloor + 1}(y)$ y ya está porque justamente la intesección es no vacía.

   Luego $d(x,z) = |x \oplus z| = |x \oplus x \oplus e'| = |e'| = \lfloor \frac{d-1}{2} \rfloor + 1$

$$d(y,z) = |y \oplus z| = |y \oplus x \oplus e'| = |e \oplus e'| = d - \left(1 + \lfloor \frac{d-1}{2} \rfloor\right) \overset{\star}{\leq} \lfloor \frac{d-1}{2} \rfloor + 1$$

Demostración de $\star$

- Si $d = 2k+1$ (Impar)
  $d - \left(1 + \lfloor \frac{d-1}{2} \rfloor\right) = 2k + 1 - 1 - k = k = \lfloor \frac{d-1}{2} \rfloor < \lfloor \frac{d-1}{2} \rfloor + 1$
- Si $d = 2k$ (Par)
  $d - \left(1 + \lfloor \frac{d-1}{2} \rfloor\right) = 2k - 1 - (k-1) = 2k - 1 - k + 1 = k = k - 1 + 1 = \lfloor \frac{d-1}{2} \rfloor + 1$

■

Suposiciones acerca del medio de transmisión

1. No se pierden ni agregan bits.[26]
2. Sean:
   - E_i^+ el evento "Bit i cambia de $0 \to 1$".
   - E_i^- el evento "Bit i cambia de $1 \to 0$".

   Entonces: $P\left(E_i^+\right) = P\left(E_i^-\right)$. Denotamos E_i al evento: cambia el Bit i.
3. E_i es independiente de E_j $\forall i \neq j$
4. $0 < P(E_i) = P(E_j) < \frac{1}{2}$

5.1. Cota de Hamming

Teorema: sea C código de longitud n que corrija t errores entonces:

$$|C| \leq \frac{2^n}{1 + n + \ldots + \binom{n}{t}}$$

Prueba: sea $A = \bigcup_{x \in C} B_t(x)$
Como C corrije t errores, esa unión es disjunta.

$$\therefore |A| = \sum_{x \in C} |B_t(x)|$$

Calculemos $|B_t(x)|$:
Para ello definamos:

$$S_r(x) = \{y \in \mathbb{Z}_2^n : d(x,y) = r\}$$

[26] Sólo hay flips de bits.

$\therefore B_t(x) = \bigcup_{r=0}^{t} S_r(x)$ unión disjunta.

$$|B_t(x)| = \sum_{r=0}^{t} |S_r(x)|$$

$$\begin{aligned} y \in S_r(x) &\Leftrightarrow d(x,y) = r \\ &\Leftrightarrow (x \oplus y) = r \text{ donde defino } (x \oplus y) = e \\ &\Leftrightarrow \exists \underbrace{e : |e| = r}_{\in S_r(0)} : y = e \oplus x \end{aligned}$$

$\therefore S_r(x) = x \oplus S_r(0) \Rightarrow |S_r(x)| = |S_r(0)|$

$$\text{Luego } e \in S_r(0) \Leftrightarrow \text{ tiene } r\ 1's \text{ y } n-r\ 0's$$

De las n ubicaciones hay que elegir las r donde irán los $1's$.
$\therefore |S_r(0)| = \binom{n}{r}$

$$\begin{aligned} \therefore |A| &= \sum_{x \in C} |B_t(x)| \\ &= \sum_{x \in C} \left(\sum_{r=0}^{t} \binom{n}{r} \right) \\ &= \left| \sum_{r=0}^{t} \binom{n}{r} \right| * \left| C \right| \end{aligned}$$

Como $A \subseteq \mathbb{Z}_2^n \Rightarrow |A| \leq 2^n$

$$\Rightarrow \left| \sum_{r=0}^{t} \binom{n}{r} \right| * \left| C \right| \leq 2^n$$

$$\Rightarrow \left| C \right| \leq \frac{2^n}{1 + n + \ldots + \binom{n}{t}}$$

■

<u>Definición</u>: un código se dice perfecto si C es un subespacio vectorial de $\mathbb{Z}_2^n$
Recordemos que un subespacio vectorial[27] se da si:

1. $C \neq \emptyset$
2. Si $x, y \in C \Rightarrow x \oplus y \in C$
3. $x \in C, k \in \mathbb{Z}_2 \Rightarrow k * x \in C$

[27] SEV.

- Pero $\mathbb{Z}_2 = \{0, 1\}$
  - 1*X = X
  - 0*X = 0

$\therefore$ Trivialmente está en C, bastaría ver que $0 \in C$.

$$C \neq \emptyset \Rightarrow \exists x \in C \Rightarrow \underbrace{x \oplus x}_{=0} \in C$$

$\therefore$ C es lineal si y sólo si ($\Leftrightarrow$)

- $C \neq \emptyset$
- $x, y \in C \Leftrightarrow x \oplus y \in C$

<u>Teorema</u>: C lineal $\Rightarrow \delta = \text{mín}\Big\{|x| : x \in C, x \neq 0\Big\}$

<u>Prueba</u>:
Sea $w = \text{mín}\{|x| : x \in C, x \neq 0\}$
Sean $x, y \in C : x \neq y$ con $d(x, y) = \delta$

$$\delta = |x \oplus y|$$

Como C es lineal $x \oplus y \in C$ y $x \oplus y \neq 0$, pues $x \neq y$ $\boxed{\Rightarrow \delta \geq w}$
Sea ahora $x \in C, x \neq 0$ con $|x| = w$
Entonces $w = d(x, 0)$, $0 \in C$ pues C es lineal.
Como δ es la menor distancia tenemos que $\boxed{\delta \leq w}$

$$\therefore \delta = w$$

■

<u>Ejemplos</u>: C_1, C_2, C_3 son obviamente lineales

- $C_1 = \{11, 00, 01, 10\} = \mathbb{Z}_2^2 \Rightarrow w = \delta = 1$
- $C_2 = \{000, 110, 101, 011\} \Rightarrow w = \delta = 2$
- $C_3 = \{000000, 111111, 101010, 010101\} \Rightarrow w = \delta = 3$

<u>Notación</u>: si C es lineal k denotará la dimensión de C.

<u>Corolario</u>: si C es lineal de dimensión $k \Rightarrow |C| = 2^k$

<u>Prueba</u>: obvio pues $k = Dim(C) \Rightarrow C \simeq \mathbb{Z}_2^k \Rightarrow |C| = 2^k$ ■

5.2. Matriz generadora

Definición: una matriz generadora G para C es una matriz cuyas filas forman una base de C.

Por lo tanto si C es de dimesión k y longitud n y G es generadora de C entonces G es K_xN.

G generadora $\Leftrightarrow G$ *es* K_xN, *con* $k = Dim(x)$ *y* $C = EF(G)$

Recordemos que: $EF(G) = \{x \in \mathbb{Z}_2^n : \exists U \in \mathbb{Z}_2^k : x = U * G\}$

$$\text{Sea } G = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$C = \{x_1x_2x_3x_4x_5x_6 : \exists u_1, u_2, u_3 : x = U * G\}$

$$u_1u_2u_3 * G = u_1u_2u_3(u_1 \oplus u_2 \oplus u_3)u_1(u_1 \oplus u_2)$$

$$C = \left\{ \begin{matrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 \end{matrix} \right\} \Rightarrow \delta = 2$$

En particular:

$$G = \left[\begin{array}{c|c} \overbrace{\mathrm{I}}^{n-1} & \begin{matrix} 1 \\ \\ \vdots \\ \\ 1 \end{matrix} \end{array} \right]$$

Da el código:

$$C = \{x_1 \dots x_n : x_n = x_1 \oplus \dots \oplus x_{n-1}\} = \{x_1 \dots x_n : x_1 \oplus \dots \oplus x_{n-1} \oplus x_n = 0\}$$

"Bit parity Code" tiene $\delta = 2$

Otro ejemplo de código lineal: **códigos de repetición**

$$C = \{\underbrace{\overbrace{x_1 \dots x_k}\overbrace{x_1 \dots x_k} \dots \overbrace{x_1 \dots x_k}}_{r\ veces}\}$$

$n = r * k$

Una matriz generadora $G = \underbrace{\left[I \ \middle| I \ \middle| I \ \middle| I \ \middle| I \ \middle| I\right]}_{r\ I's} \Rightarrow \delta =$ peso mínimo no nulo $= r$

5.3. Matriz de chequeo

Una matriz H se dice matriz de chequeo si $C = Nu(H) = \{x \in \mathbb{Z}_2^n : Hx^t = 0\}$

Propiedad

$$k\left\{\overbrace{\left[\underbrace{I}_{k_x k} \;\middle|\; \underbrace{A}_{n-k}\right]}^{N}\right. \text{ es generadora de } C \Leftrightarrow n-k\left\{\left[\underbrace{\mathrm{A}^t}_{k} \;\middle|\; \underbrace{I}_{n-k}\right]\right. \text{ es de chequeo.}$$

Todo código tiene una matriz de chequeo.

Prueba: Hay que ver que $EF\big([I|A]\big) = Nu\big([A^t|I]\big)$

$\boxed{\subseteq}$ Sea $x \in EF\big([I|A]\big) \Rightarrow \exists U : x = U[I|A] = U(U.A)$

$$\begin{aligned}
[A^t|I]x^t &= [A^t|I]\begin{bmatrix} U^t \\ (U.A)^t \end{bmatrix} \\
&= A^tU^t \oplus (UA)^t \\
&= A^tU^t \oplus A^tU^t \\
&= 0
\end{aligned}$$

Entonces $x \in Nu([A^t|I])$

$\boxed{\supseteq}$ $EF[I|A] \subseteq Nu([A^t|I])$

$$\left.\begin{array}{c}
\text{Pero } Dim(EF([I|A])) = k \\
Dim(Nu([\underbrace{A^t}_{\substack{k \\ dep.}} | \underbrace{I}_{\substack{n-k \\ indeptes}}])) = k
\end{array}\right\} \therefore \text{ son iguales}$$

■

Ejemplo:

$$G = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \Rightarrow H = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned}
Entonces\ Nu(H) &= \left\{ x_1 \dots x_6 : \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = 0 \right\} \\
&= \left\{ x_1 \dots x_6 : \begin{array}{l} x_1 \oplus x_2 \oplus x_3 \oplus x_4 = 0 \\ x_1 \oplus x_5 = 0 \\ x_1 \oplus x_2 \oplus x_6 = 0 \end{array} \right\} \\
&= \left\{ x_1 \dots x_6 : \begin{array}{l} x_4 = x_1 \oplus x_2 \oplus x_3 \\ x_5 = x_1 \\ x_6 = x_1 \oplus x_2 \end{array} \right\} \text{que es lo que tenía antes.}
\end{aligned}$$

Ejercicio: calcular las palabras del código que genera la siguiente matriz G.

$$Si\ G = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} \Rightarrow H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \text{ en este caso } \delta = 3$$

Teorema: sea H la matriz de chequeo de C, si H no tiene la columna 0 ni columnas repetidas entonces $\delta \geq 3$. Y por lo tanto corrige al menos un error.

Prueba: supongamos que no $(\delta < 3) \Rightarrow \delta = 1$ ó $\delta = 2$

- Supongamos $\delta = 1$.

  Como C es lineal $\Rightarrow \exists x \in C : |x| = 1 \Rightarrow \exists i : x = e_i$[28]

  Como $x \in C \Rightarrow Hx^t = 0$

  $\therefore 0 = Hx^t = He_i^t = H^{(i)}$ Columna i de H. $\Rightarrow H^{(i)} = 0$ **absurdo**.

- Supongamos $\delta = 2$.

  Entonces $\exists x \in C : |x| = 2$

  Entoces $\exists_{i,j} : i \neq j : x = e_i \oplus e_j$

  $0 = Hx^t = He_i^t \oplus He_j^t = H^{(i)} \oplus H^{(j)}$

  $\therefore H^{(i)} = H^{(j)}$ **absurdo**.

■

Ejemplo

$$\text{Sea } H = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix} \text{ H es } 4_x 9$$

$C = Nu(H)$
$|C| \stackrel{?}{=} 2^k = 2^5 = 32$
$k \stackrel{*}{=} 9 - 4 = 5$
9 variables - 4 independientes
$\stackrel{*}{=} \#Columnas - Rango$ es n porque tengo la identidad.

$$\delta(C) \stackrel{?}{=}$$

Como H no tiene la columna 0 ni columnas repetidas $\Rightarrow \delta(C) \geq 3$
Como $H^{(1)} + H^{(5)} = H^{(9)}$

[28] Recordemos $e_i = 0 \ldots 0 \underbrace{1}_{i} 0 \ldots 0$.

Entonces $x = 010001001 \ \in C \Rightarrow \delta(C) \leq 3$

$$\therefore \delta(C) = 3$$

Una matriz que cumple esa condición es una matriz $r * (2^r - 1)$
Con todas las columnas posibles no nulas de altura r.
Entonces $C =$ código de Hamming

- $\mathscr{H}_2 = H = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$ orden posible.

$$\begin{aligned} \mathscr{H}_2 &= \left\{ xyz : \begin{matrix} x \oplus z = 0 \\ y \oplus z = 0 \end{matrix} \right\} \\ &= \{xyz : z = x = y\} \\ &= \text{código de repetición} \end{aligned}$$

- $\mathscr{H}_3 = H = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$ $K = 4$

- $\mathscr{H}_4 = H = \begin{bmatrix} 1010101 & 0110101 \\ 0110011 & 0010011 \\ 0001111 & 0001111 \\ 0000000 & 1111111 \end{bmatrix}$ $K = 11$

<u>Propiedad</u>: los Hamming son perfectos.

<u>Prueba</u>: Hay que ver $\left| \mathscr{H}_r \right| = \frac{2^n}{1+n}$
Donde $n = 2^r - 1 \Rightarrow 1 + n = 2^r$

$$\therefore \frac{2^n}{1+n} = \frac{2^n}{2^r} = 2^{n-r} = 2^{2^r-1-r} = 2^k = \left| \mathscr{H}_r \right|$$

■

<u>Algoritmo para corregir 1 error</u>

Cuando $H^{(i)} \neq 0, H^{(j)}$
Supongamos que el emisor manda $x \in C$ y el receptor recibe $y = x \oplus e$ con $|e| \leq 1$.
Si $e = 0$
$Hy^y = Hx^t = 0 \ \therefore y \in C$
Si $e = e$

$$\begin{aligned} \Rightarrow Hy^t &= Hx^t \oplus He_i^t \\ &= 0 \oplus H^{(i)} \\ &= H^{(i)} \end{aligned}$$

$\therefore$ El algoritmo es:

1. Calcular Hy^t.
2. Si $Hy^t = 0$ Aceptar y.
3. Si $Hy^t = H^{(i)}$ Aceptar $y \oplus e_i$.
4. Si $Hy^t \neq 0, H^{(i)}\ \forall i \Rightarrow$ hubo ≥ 2 errores, pedir retrasmisión.

<u>Ejemplo</u>:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Se recibe: $y = 01000100$

$$\begin{aligned} Hy^t &= H^{(2)} \oplus H^{(3)} \oplus H^{(7)} \\ &= \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \oplus \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \end{bmatrix} = H^{(6)} \end{aligned}$$

$\therefore$ La palabra enviada fue $x = y \oplus e_6 = 011001100$
Pero si recibo $z = 000011000$

$$\begin{aligned} Hz^t &= H^{(5)} \oplus H^{(6)} \\ &= \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \neq 0, H^{(i)}\ \forall i \therefore \text{ Hubo } \geq 2 \text{ errores} \end{aligned}$$

Por ejemplo me pueden haber mandado: 000000000 ó 110011000
En el caso del Hamming Hy^t siempre va a ser 0 ó algún $H^{(i)}$
El algoritmo si se usa la matriz con $H^{(i)} = \begin{bmatrix} (i) \\ Binario \end{bmatrix}$
Es más fácil:

1. Calcular Hy^t
2. Si $Hy^t = 0$ aceptar y
3. Sino tomar i: (representación binaria i)$= Hy^t$ y acepto $y \oplus e_i$

Ejemplo
$\mathscr{H}_5$ tiene $n = 2^5 - 1 = 31 \Rightarrow k = 31 - 5 = 26$
$\therefore \left| \mathscr{H}_5 \right| = 2^{26}$
Supongamos que llega $y = e_1 \oplus e_2 \oplus e_{11} \oplus e_{27} \oplus e_{31}$
Entonces:

$$\begin{aligned} Hy^t &= H^{(1)} \oplus H^{(2)} \oplus H^{(11)} \oplus H^{(27)} \oplus H^{(31)} \\ &= \begin{bmatrix}1\\0\\0\\0\\0\end{bmatrix} \oplus \begin{bmatrix}0\\1\\0\\0\\0\end{bmatrix} \oplus \begin{bmatrix}1\\1\\0\\1\\0\end{bmatrix} \oplus \begin{bmatrix}0\\1\\1\\0\\1\end{bmatrix} \oplus \begin{bmatrix}1\\1\\1\\1\\1\end{bmatrix} = \begin{bmatrix}1\\0\\0\\0\\0\end{bmatrix} = H^{(1)} \end{aligned}$$

$\therefore$ Mandaron $x = y \oplus e_1$
Si hubiese sido $y = e_2 \oplus e_7 \oplus e_{17}$

$$Hy^t = \begin{bmatrix}0\\1\\0\\0\\0\end{bmatrix} \oplus \begin{bmatrix}1\\1\\1\\0\\0\end{bmatrix} \oplus \begin{bmatrix}1\\0\\0\\0\\1\end{bmatrix} = \begin{bmatrix}0\\0\\1\\0\\1\end{bmatrix} = H^{(20)}$$

$\therefore$ Mandaron $x = y \oplus e_{20}$
Observación: Hamming más un bit de paridad avisa si hay 2 errores.

Teorema: sea $C = Nu(H) \Rightarrow \delta(C) = \text{mín}\{j : \exists j \text{ columnas L.D. de H}\}$[29]

Prueba:
$\boxed{\geq}$: Sea $x : |x| = \delta(C) = d$
Entonces $\exists i_1, i_2, \ldots, i_d : x = e_{i_1} \oplus \ldots \oplus e_{i_d}$

$$\begin{aligned} 0 &= Hx^t \\ &= He_{i_1}^t \oplus \ldots \oplus He_{i_d}^t \\ &= H^{(i_1)} \oplus \ldots \oplus H^{(i_d)} \end{aligned}$$

Entonces $\{H^{(i_1)}, \ldots, H^{(i_d)}\}$ es L.D.
$\therefore\ \delta \geq \text{mín}\{j : \exists j \text{ columnas dependientes de H}\} = j^*$
$\boxed{\leq}$ $\exists i_1, \ldots, i_{j^*} : \{H^{i_1}, \ldots, H^{i_{j^*}}\}$ es L.D.
Entonces $\exists c_1, \ldots, c_{j^*}$ no todos nulos tal que: $c_1 H_{i_1}, \ldots, c_{j^*} H_{i_{j^*}} = 0$
Como j^* es el mínimo $\Rightarrow c_1, \ldots, c_{j^*} \neq 0$ todos no nulos.
Como $C_i \in \mathbb{Z}_2 \Rightarrow c_i = 1\ \forall i$
$\therefore H^{(i_1)} \oplus \ldots \oplus H^{(i_{j^*})} = 0$
Entonces $x \in C$ donde $x = e_{i_1} \oplus \ldots \oplus e_{i_{j^*}}$
$\therefore \delta \leq j^*$ ■

[29] LD: Linealmente dependientes.

5.4. Cota Singleton

Corolario: si C es un código lineal de longitud n y dimensión k entonces $\delta(C) \leq n-k+1$
Prueba:
Sean $d = \delta(C), H : C = Nu(H)$

$$\begin{aligned} d &= \text{mín}\{j : \exists j \text{ columnas L.D. de } H\} \\ &= 1 + \text{máx}\{j : \ \underline{\text{Todo}} \text{ conjunto de j columnas es L.I.}\} \\ &\leq 1 + \text{máx}\{j : \ \underline{\text{Existe}} \text{ un conjunto de j columnas L.I.}\} \\ &\leq 1 + Rango(H) \\ &= 1 + n - k \end{aligned}$$

■

Definición: un Código con $\delta = n - k + 1$ se llama un código MDS[30].

5.5. Códigos Cíclicos

Definición: sea $W = w_0 w_1 \ldots w_{n-1}$ definimos $Rot(W) = w_{n-1} w_0 w_1 \ldots w_{n-2}$
Definición: un código C es cíclico si:

1. Es lineal.
2. Invariante por rotaciones: i.e. $W \in C \Rightarrow Rot(W) \in C$

Ejemplos

- $\mathbb{Z}^2+$ Bit de paridad es cíclico $C = \{000, 110, 101, 011\}$
- $C = \{000, 100, 010, 001\}$ es invariante por rotación pero no cíclico ya que no es lineal: $C[1] + C[2] \notin C$
- $C = \{000, 100, 011, 111\}$ es lineal pero no es cíclico.

Identificación: $w_0 w_1 \ldots w_{n-1} \leftrightarrow w_0 + w_1 x + \ldots + w_{n-1} x^{n-1}$
Definición: si $P(x), H(x)$ son polinomios.

$$P(x) \mod H(x) \text{ es el resto de dividir } P(x) \text{ por } H(x).$$

Teorema: si W tiene longitud n entonces:

$$Rot(W) = xW(x) \mod (1 + x^n)$$

Prueba:

$$\begin{aligned} xW(x) &= x\left(w_0 + w_1 x + \ldots w_{n-1} x^{n-1}\right) \\ &= w_0 x + w_1 x^2 + \ldots + w_{n-2} x^{n-1} + w_{n-1} x^n \end{aligned}$$

[30] MDS: maximun distances separable.

Aplico módulo $1 + x^n$

$$\begin{aligned} xW(x) \quad \text{mód } (1+x^n) &= w_0 x + w_1 x^2 + \ldots + w_{n-2}x^{n-1} + w_{n-1} && (24) \\ &= w_{n-1} + w_0 x + w_1 x^2 + \ldots + w_{n-2}x^{n-1} && (25) \\ &\leftrightarrow w_{n-1}w_0 \ldots w_{n-2} \\ &= Rot(W) \end{aligned}$$

(24) $w_{n-1}x^n = \underbrace{w_{n-1}(x^n+1)}_{Cociente} + \underbrace{w_{n-1}}_{Resto}$

(25) Conmuto. ■

<u>Ejemplo</u>

- $C = \{0000, 1010, 0101, 1111\}$
  $C \leftrightarrow \{0, 1+x^2, x+x^3, 1+x+x^2+x^3\}$
  - Tomo $w = 1010 \Rightarrow Rot(w) = 0101$
    $1 + x^2 \Rightarrow x(1+x^2) = x + x^3 \text{ mód } (1+x^4) = x + x^3 \leftrightarrow 0101$
  - Tomo $w = 0101 \Rightarrow Rot(w) = 1010$
    $x + x^3 \Rightarrow x(x+x^3) = x^2 + x^4$
    $x^2 + x^4 \text{ mód } (1+x^4) = x^2 + 1 = 1 + x^2 \leftrightarrow 1010$

<u>Corolario</u>:
Sea C cíclico de longitud n, $\left.\begin{array}{l} W(x) \in C \\ Q(x) \in \mathbb{Z}_2(x) \end{array}\right\} \Rightarrow W(x)Q(x) \text{ mód } (1+x^n) \in C$

<u>Propiedad</u>: sea C lineal entonces ∃! polinomio <u>no nulo</u> de <u>grado mínimo</u>.

<u>Definición</u>: si C es cíclico, el único polinomio no nulo de grado mínimo se llama el **polinomio Generador** de C, y usualemente se lo denomina G(x).
<u>Prueba</u>:
Sean g_1, g_2 dos de ellos : $Gr(g_1) = Gr(g_2) = t$
$g_1 + g_2 \in C$ pues C es lineal.
Entonces: $Gr(g_1 + g_2) \overset{\star}{=} Gr(g_1 - g_2) < t$ que era el menor grado.
$\therefore g_1 + g_2 = 0 \Rightarrow g_1 = g_2$

$$\begin{aligned} \boxed{\star} : g_1(x) &= a_0 + a_1 x + \ldots + a_{t-1}x^{t-1} + 1.x^t \\ : g_2(x) &= b_0 + b_1 x + \ldots + b_{t-1}x^{t-1} + 1.x^t \end{aligned}$$

Entonces: $g_1 + g_2 = (a_0 + b_0) + (a_1 + b_1)x + \ldots + (a_{t-1} + b_{t-1})x^{t-1} + 0$ ■

Propiedad: sea $G(x)$ Generador : $G(x) \neq 0 \Rightarrow Gr(G(x)) \leq Gr(p(x))\ \forall p(x) \in C$

Teorema: sea C un código cíclico de longitud $n, G(x)$ su polinomio generador entonces:

$$C = \Big\{ P(x) \in \mathbb{Z}_2(x) : Gr(P(x)) \leq n-1 \text{ y } G(x)|P(x) \Big\}$$

Prueba:

- $\boxed{\supseteq}$ Si G(x) dividide a P(x):

$$G(x)|P(x) \underset{clase\ pasada}{\Rightarrow} \exists q(x) : P(x) = q(x) * G(x) \Rightarrow P(x) \quad \text{mód } (1+x^n) \in C$$

$$\text{Como } Gr(P(x)) \leq n-1 \Rightarrow P(x) \quad \text{mód } (1+x^n) = P(x) \Rightarrow P(x) \in C$$

- $\boxed{\subseteq}$ La parte importante

$$\text{Sea } P(x) \in C \Rightarrow Gr(P(x)) \leq n-1$$

Para la segunda condición:

Sean $Q(x), R(x) \in \mathbb{Z}_2(x) : P(x) = Q(x)G(x) + R(x)$: con $Gr(R(x)) < Gr(G(x))$

Como $Gr(R(x)) < Gr(G(x)) \Rightarrow \left.\begin{array}{r} Gr(R(x)) < n-1 \\ Gr(P(x)) \leq n-1 \end{array}\right\} \Rightarrow Gr(Q(x)G(x)) \leq n-1$

Entonces por $\boxed{\supseteq}$ $Q(x)G(x) \in C$

$$\therefore R(x) = \underbrace{P(x)}_{\in C} + \underbrace{Q(x)G(x)}_{\in C} \in C \text{ por ser lineal.}$$

$$\left.\begin{array}{r} Gr(R(x)) < Gr(G(x)) \leq n-1 \\ R(x) \in C \end{array}\right\} \Rightarrow R(x) = 0$$

$$\therefore P(x) = Q(x)G(x) \Rightarrow G(x)|P(x)$$

■

<u>Teorema</u>

Sea C cíclico de longitud n, y $G(x)$ el polinomio generador de C, con $k = Dim(C)$

<u>Entonces</u>:

1. $Gr(G(x)) = n - k$
2. $G(x)|1 + x^n$
3. $G(x) = \boxed{1} + g_1(x) + \ldots$ i.e. $g_0 = 1$

<u>Prueba</u>

1. Sea $P(x) : Gr(P(x)) \leq n - 1$

   Por el teorema anterior $P(x) \in C \Leftrightarrow \exists Q(x) : P(x) = Q(x)G(x)$

   Si $t = Gr(G(x)) \Rightarrow n - 1 \geq Gr(P(x)) = Gr(Q(x)) + \underbrace{Gr(G(x))}_{=t}$

   $$\therefore Gr(Q(x)) \leq n - t - 1$$

   Viceversa si $P(x) = Q(x)G(x)$ con $Gr(Q(x)) \leq n - t - 1$

   Entonces: $Gr(P(x)) \leq n - t - 1 + t = n - 1$

   $\therefore P(x) \in C$

   $$C = \left\{ P(x) : \exists Q(x) : \left\{ \begin{array}{c} Gr(Q(x)) \leq n - t - 1 \\ P(x) = Q(x)G(x) \end{array} \right. \right\}$$

   Esto da una biyección entre C y $\left\{ Q(x) : Gr(Q(x)) \leq n - t - 1 \right\} = \star$

   $$\begin{aligned} \therefore |C| &= |\star| \\ &= |\{Q(x) = q_0 + q_1 x + \ldots + q_{n-t-1}x^{n-t-1}\}| \\ &= 2^{n-t} \end{aligned}$$

   Como $k = Dim(C) \Rightarrow |C| = 2^k \Rightarrow 2^k = 2^{n-t} \Leftrightarrow k = n - t \Leftrightarrow \boxed{t = n - k}$

2. Supongamos $G(x) = g_0 + g_1 x + \ldots + g_{n-k-1}x^{n-k-1} + x^{n-k}$

   Por lo demostrado en 1. sabemos que es de esa manera.

   <u>Multiplico por x^k</u>:

   $$\begin{aligned} x^k G(x) &= g_0 x^k + g_1 x^{k+1} + \ldots + g_{n-k-1}x^{n-1} + x^n \\ &= \underbrace{(1 + g_0 x^k + \ldots + g_{n-k-1}x^{n-1})}_{Rot^k(G)} + (1 + x^n) \end{aligned} \tag{26}$$

(26) Sumo dos veces 1 en $\mathbb{Z}_2$, que es lo mismo que sumar 0.

Como C es cíclico y $G(x) \in C \Rightarrow Rot^k(G(x)) \in C$

Como $G(x)$ es generador $\exists Q(x) : Rot^k(G) = Q(x)G(x)$

$$\begin{aligned} \therefore x^k G(x) &= Rot^k(G(x)) + (1 + x^n) \\ &= Q(x)G(x) + (1 + x^n) \\ \therefore 1 + x^n &= x^k G(x) + Q(x)G(x) \\ &= \Big(x^k + Q(x)\Big)G(x) \end{aligned}$$

$$\text{Entonces } G(x)|1 + x^n$$

3. $1 + x^n = V(x)G(x) \Rightarrow 1 = v_0 g_0 \Leftrightarrow g_0 = 1$

■

Esto da origen a un método para <u>codificar</u>:

- Primer método:

  Simplemente dada $m(x)$ de grado $\leq k - 1$.

  Tomo $V(x) = m(x)G(x)$

  <u>Ejemplo</u>:

  $G(x) = 1 + x^2 + x^3 : n = 7$

$$\text{¿Cuánto vale } k = Dim(C) \text{ ?}$$
$$n - k = Gr(G(x)) \;\; \therefore 7 - k = 3 \Rightarrow k = 4$$

  Por lo tanto el código tiene $2^4 = 16$ palabras.

  Supongamos que queremos codificar:

  - $m = 1011 \leftrightarrow m(x) = 1 + x^2 + x^3$

$$\begin{aligned} V(x) &= m(x)G(x) \\ &= (1 + x^2 + x^3)^2 \\ &= 1 + x^4 + x^6 \\ &\leftrightarrow 1000101 \end{aligned}$$

  $\therefore$ Mando 1000101

- Otra: $m = 0110 \leftrightarrow x + x^2$

$$\begin{aligned} V(x) &= (x+x^2)(1+x^2+x^3) \\ &= x + x^3 + \cancel{x^4} + x^2 + \cancel{x^4} + x^5 \\ &= x + x^2 + x^3 + x^5 \\ &\leftrightarrow 0111010 \end{aligned}$$

Esto corresponde a la siguiente matriz generadora:

$$\begin{bmatrix} 1 & g_1 & g_2 & \dots & g_{n-k-1} & 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & g_1 & g_2 & \dots & g_{n-k-1} & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & g_1 & g_2 & \dots & g_{n-k-1} & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & 1 & g_1 & g_2 & \dots & g_{n-k-1} & 1 \end{bmatrix}$$

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

- <u>Segundo Método</u>

  <u>Idea</u>: sea $\Big(P(x) + \Big(P(x) \mod G(x)\Big)\Big) = 0 \mod (G(x))$

  $G(x) | [P(x) + (P(x) \mod G(x))]$

  $\therefore$ Si el $Gr(P(x)) \leq n-1$ ya está: $P(x) + (P(x) \mod G(x)) \in C$

  <u>Truco</u>

  Dado $m(x)$ con $Gr(m(x)) \leq k-1 \Rightarrow Gr(x^{n-k}m(x)) \leq n-1$

  En definitiva $m(x) \to V(x) = x^{n-k}m(x) \mod G(x) + x^{n-k}m(x)$

  <u>Ejemplos</u>:

  - $m(x) = 1011 \leftrightarrow 1 + x^2 + x^3 : n = 7 \Rightarrow k = 4$
    $x^3 m(x) \mod G(x) = 0$
    $\therefore V(x) = 0 + x^3 m(x) = x^3 + x^5 + x^6 \leftrightarrow 0001011$
  - $m(x) = 0110 \leftrightarrow x + x^2$
    $x^3 m(x) = x^4 + x^5$
    $x^3 m(x) \mod G(x) \stackrel{?}{=}$
    <u>Truco</u>: $G(x) = 1 + x^2 + x^3 \Rightarrow 1 + x^2 + x^3 \equiv 0(G(x))$

$$x^3 \equiv 1 + x^2(G(x))$$

$$
\begin{aligned}
x^4 &\equiv x + \boxed{x^3}\ (G(x)) \\
&\equiv x + 1 + x^2\ (G(x)) \\
&\equiv 1 + x + x^2\ (G(x))
\end{aligned}
$$

$$
\begin{aligned}
x^5 &\equiv x + x^2 + \boxed{x^3}\ (G(x)) \\
&\equiv x + \cancel{x^2} + 1 + \cancel{x^2}\ (G(x)) \\
&\equiv 1 + x\ (G(x))
\end{aligned}
$$

$$\therefore x^4 + x^5 \mod G(x) \equiv \cancel{1} + \cancel{x} + x^2 + \cancel{1} + \cancel{x} \equiv x^2$$

$$
\begin{aligned}
\therefore V(x) &= (x^4 + x^5) \mod G(x) + x^4 + x^5 \\
&= x^2 + x^4 + x^5 \\
&\leftrightarrow 001 \underbrace{\boxed{0110}}_{m(x)}
\end{aligned}
$$

Esto corresponde a la matriz generadora:

$$
G = \begin{bmatrix} x^{n-k} & \text{mód } G(x) & + & x^{n-k} \\ x^{n-k-1} & \text{mód } G(x) & + & x^{n-k-1} \\ \vdots & \vdots & \vdots & \vdots \\ x^{n-1} & \text{mód } G(x) & + & x^{n-1} \end{bmatrix}
$$

Teníamos:

$$
\begin{array}{ll}
x^3 & \text{mód } G(x) \equiv 1 + x^2 \\
x^4 & \text{mód } G(x) \equiv 1 + x + x^2 \\
x^5 & \text{mód } G(x) \equiv 1 + x \\
x^6 & \text{mód } G(x) \equiv 1 + x^2
\end{array}
\Rightarrow G = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}
$$

Es decir obtenemos la matriz G con identidad a derecha.

Una matriz de chequeo H es:

$$
H = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}
$$

En general esto vale claramente: es decir, $H^{(i)} = x^i \mod G(x) : 0 \leq i \leq n-1$

Ejemplo: sea $G(x) = 1 + x^2 + x^3 + x^4 : n = 7$
Hallar una matriz de chequeo con la identidad a izquierda.

$$\begin{aligned}
x^4 \quad \text{mód } G(x) &\equiv 1 + x^2 + x^3 \\
x^5 \quad \text{mód } G(x) &\equiv x + x^3 + \boxed{x^4} \\
&\equiv x + \cancel{x^3} + 1 + x^2 + \cancel{x^3} \\
&\equiv 1 + x + x^2 \\
x^6 \quad \text{mód } G(x) &\equiv x + x^2 + x^3
\end{aligned}$$

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

5.5.1. Correción de errores "Error Trapping"

Teorema: sea C cíclico de longitud n que corrige t errores, con $G(x) =$ generador.
Asuma que se manda $V(x)$ y llega $W(x) : E(x) = V(x) \oplus W(x)$
Con $|E| \leq t$, "error corregible".

Sean S_i los "síndromes": $S_i(x) = x^i W(x) \text{ mód } (G(x)) : 0 \leq i \leq n-1$

Si $\exists i : |S_i| \leq t \Rightarrow$

$$\begin{aligned}
E(x) &= Rot^{n-i}(S_i) \\
&= x^{n-i} S_i \quad \text{mód } (1 + x^n)
\end{aligned}$$

Prueba de "Error Trapping":

Observación: $Rot(U) \text{ mód } G(x) = xU(x) \text{ mód } G(x)$
Pues $Rot(U) = xU(x) \text{ mód } (1 + x^n)$
$\therefore \exists Q(x) : xU(x) = Q(x)(1 + x^n) + Rot(U)$

$$xU(x) \quad \text{mód } G(x) = \Big(\underbrace{Q(x)(1 + x^n \quad \text{mód } G(x))}_{0 \ pues \ G(x)|1+x^n}\Big) + Rot(U) \quad \text{mód } (G(x))$$

$\therefore Rot\ U(x) \text{ mód } G(x) = xU(x) \text{ mód } G(x)$
$\therefore Rot^i U(x) \text{ mód } G(x) = x^i U(x) \text{ mód } G(x)$
Sea i : $\boxed{|S_i| \leq t}$ $\star$
$S_i(x) = x^i U(x) \text{ mód } G(x) \Rightarrow$ **absurdo**.
$S_i(x) = Rot^i(W(x)) \text{ mód } G(x) \Rightarrow \exists P(x)$: $\boxed{Rot^i(W(x)) = P(x)G(x) + S_i(x)}$ $\oslash$
$\star$ y $\oslash \Rightarrow \boxed{Rot^i(W(x)) \in B_t(P(x)G(x))}$ $\Diamond$
Pero $P(x)G(x) \in C$

Pero $W(x) = V(x) + E(x)$

$\boxed{Rot^i(W(x)) = Rot^i(V(x)) + Rot^i(E(x))}$ †

Y $|Rot^i(E(x)| = |E(x)| \leq t$

$\therefore \boxed{Rot^i(W(x)) \in B_t(\underbrace{Rot^i(V(x))}_{\in C})}$ ♣

$\Diamond$ y ♣ $\Rightarrow B_t(\underbrace{P(x)G(x)}_{\in C}) \cap B_t(\underbrace{Rot^i(V(x))}_{\in C}) \neq \emptyset$

$\Rightarrow \boxed{P(x)G(x) = Rot^i(V(x))}$ ♠

Entonces por ♠ y $\oslash$ y † $\Rightarrow S_i(x) = Rot^i(E(x))$

$$\therefore E(x) = Rot^{n-i} S_i$$

■

Ejemplo

$$G(x) = 1 + x^4 + x^6 + x^7 + x^8 : n = 15 \text{ con } t = 2$$

Llega $W(x) = 1100\ 1110\ 1100\ 010$

$W(x) = 1 + x + x^4 + x^5 + x^6 + \boxed{x^8} + \boxed{x^9} + \boxed{x^{13}}$

Al aplicar $\quad$ mód $G(x)$ solo tengo que resolver los $x^i : i > k = 7$ (los encuadrados).

$$\begin{aligned}
x^8 \quad \text{mód } G(x) &\equiv 1 + x^4 + x^6 + x^7 \\
x^9 \quad \text{mód } G(x) &\equiv x + x^5 + x^7 + \boxed{x^8} \\
&\equiv x + x^5 + \cancel{x^7} + 1 + x^4 + x^6 + \cancel{x^7} \\
&\equiv 1 + x + x^4 + x^5 + x^6 \\
x^{10} \quad \text{mód } G(x) &\equiv x + x^2 + x^5 + x^6 + x^7 \\
x^{11} \quad \text{mód } G(x) &\equiv x^2 + x^3 + x^6 + x^7 + \boxed{x^8} \\
&\equiv x^2 + x^3 + \cancel{x^6} + \cancel{x^7} + 1 + x^4 + \cancel{x^6} + \cancel{x^7} \\
&\equiv 1 + x^2 + x^3 + x^4 \\
x^{12} \quad \text{mód } G(x) &\equiv x + x^3 + x^4 + x^5 \\
x^{13} \quad \text{mód } G(x) &\equiv x^2 + x^4 + x^5 + x^6
\end{aligned}$$

$$\begin{aligned}
S_0 &= W(x) \quad \text{mód } G(x) \\
&= 1 + x + x^4 + x^5 + x^6 + \underbrace{1 + x^4 + x^6 + x^7}_{x^8 \quad \text{mód } G(x)} + \underbrace{1 + x + x^4 + x^5 + x^6}_{x^9 \quad \text{mód } G(x)} + \underbrace{x^2 + x^4 + x^5 + x^6}_{x^{13} \quad \text{mód } G(x)} \\
&= 1 + x^2 + x^5 + x^7
\end{aligned}$$

$\therefore |S_0| = 4 > 2$

Continúo calculando $|S_1|$, hasta encontrar uno tal que $|S_i| \leq 2$

$$\begin{aligned}
S_1 &= x(W(x) \mod G(x)) \\
&= x\Big(1 + x^2 + x^5 + x^7\Big) \\
&= x + x^3 + x^6 + \boxed{x^8} \\
&= x + x^3 + \cancel{x^6} + 1 + x^4 + \cancel{x^6} + x^7 \\
&= 1 + x + x^3 + x^4 + x^7
\end{aligned}$$

$$\begin{aligned}
S_2 &= 1 + x + x^2 + x^3 + x^5 + x^6 + x^7 \\
S_3 &= 1 + x + x^2 + x^3 + x^4 \\
&\vdots \\
S_6 &= x^3 + x^4 + x^5 + x^6 + x^7
\end{aligned}$$

$$\begin{aligned}
S_7 &= x^4 + x^5 + x^6 + x^7 + \boxed{x^8} \\
&= \cancel{x^4} + x^5 + \cancel{x^6} + \cancel{x^7} + 1 + \cancel{x^4} + \cancel{x^6} + \cancel{x^7} \\
&= 1 + x^5 \\
\therefore |S_7| &= 2 \leq 2 = 7
\end{aligned}$$

$$\begin{aligned}
\text{Entonces: } E(x) &= x^{n-i} S_i \mod (1 + x^n) \\
&= x^{15-7} S_7 \mod (1 + x^{15}) \\
&= x^8(1 + x^5) \mod (1 + x^{15}) \\
&= x^8 + x^{13}
\end{aligned}$$

$\therefore V(x) = W(x) + E(x) = 1 + x + x^4 + x^5 + x^6 + x^9$

<u>Ejercicio</u>: sea $G(x) = 1 + x^2 + x^4 + x^5 + x^6 + x^{10} + x^{11} : n = 23$ con $t = 3$

$W(x) = 1 + x^7 + x^8 + x^9 + x^{12} + x^{13}$

Respuesta:

- $V(x) = 1 + x^8 + x^9 + x^{12} + x^{13} + x^{14} + x^{15} + x^{16}$
- Con $S_{16} = 1 + x^8 + x^9$

<u>Definición</u>:

$$\textbf{Polinomio Chequeador } H(x) = \frac{1 + x^n}{G(x)}$$

<u>Propiedad</u> (ejercicio)

$$W(x) \in C \Leftrightarrow H(x)W(x) \mod (1 + x^n) = 0 : Gr(W(x)) \leq n - 1$$

6. P-NP

Definición: un "problema" de decisión es un problema cuyas únicas respuestas posibles son Sí o No (alternativamente 1 ó 0).

Ejemplos

- Matching: dado un grafo G ¿tiene G un matching perfecto?
- Primo: dado un $n \in \mathbb{N}$ ¿es n primo?
- K-color: dado un grafo G ¿es $\chi(G) \leq k$?

Semiformalización de "problema de decisión".
Un conjunto I (conjunto de instancias del problema y una función $\Pi : I \to \{\text{Sí,No}\}$
Alternativamente, tomando el conjunto de instancias positivas.

$$S = \{x \in I : \Pi(x) = \text{Sí}\}$$

Se puede pensar que un problema es un par (I,S) con $S \subseteq I$.
En general, dado $x \in I$ tenemos una "medida" del "tamaño" de x.
En general, # de bits necesarios para representar a x, o algo ligado a eso.
Lo denotaremos $|x|$.
Por ejemplo si $I = \{G : G \text{ es un grafo finito}\}$, podemos tomar $|G| = \#de$ vértices.
Si I $= \mathbb{N} \Rightarrow |n| = \lceil \log(n) \rceil = \#$bits de n

6.1. La clase P

P = clase de problemas de decisión tal que existe un polinomio $P(x)$, y un algoritmo A tal que A resuelve el problema para $x \in I$ en tiempo $O(P(x))$

$$\text{Resolver es que: } A(x) = \begin{cases} \text{Sí} & Si\ \Pi(x) = \text{Sí} \\ No & Si\ \Pi(x) = \text{No} \end{cases}$$

O bien en la forma "{I,S}" $A(x) = \text{Sí} \Leftrightarrow x \in S$

Ejemplos:

- $2 - color \in P$
- Matching bipartito: como Matching con instancia reducida a grafos bipartitos.

  Vimos que Matching bipartito[31] $\in P$

6.2. La Clase NP

NP **no** significa "No Polinomial". (de hecho $P \subseteq NP$).

NP significa: "Non deterministic Polynomial".

Otra forma de definir esta clase que es la clase de problemas de decisión para los cuales existen "certificados" que verifican la respuesta "Sí" en tiempo polinomial.

[31] Edmonds probó que Matching $\in P$.

Formalmente

$(I, S) \in NP \Leftrightarrow \exists j, \exists t \subseteq I_{\mathrm{X}} J$ y $\exists P(x)$ tales que:

1. $(I_{\mathrm{X}} J, T) \in P$
2. Dado $x \in I$ vale $x \in S \Leftrightarrow \exists y \in J : \begin{cases} (x, y) \in T \\ |y| \leq P(|x|) \end{cases}$

6.3. Co-NP

Definición: **Co-NP** = problema de decisión tal que el "complemento" está en P.
Ejemplo: Primo $\in$ Co-NP
Certificado para "NO" una factorización.
(En 1976 se probó que Primo $\in$ NP) y en 2002 se probó que PRIMO $\in P$.

"Complemento de Π"

$$\text{Es } \widehat{\Pi}(x) = \begin{cases} No & Si\ \Pi(x) = \text{Sí} \\ \text{Sí} & Si\ \Pi(x) = No \end{cases} \text{ ó } (I, S) \to (I, I - S)$$

k-color $\in$ NP $\forall k$

Certificado: dado un coloreo con k colores ver que es propio es: $O(m) = O(n^2)$
Observación:

- Claramente $P \subseteq NP$.
- **"El" problema del millón es ¿P=NP?**

6.4. SAT

Un problema importate: SAT
Una variable booleana x es una variable que sólo puede tomar dos valores 0 ó 1.
Dado un conjunto de variables Booleanas: $x_1, \ldots, x_n$.
Un literal: es una de esas variables o la negación de ellas. ($\overline{x} = 1 \oplus x$).
Una claúsula: ("Clause") es un "$\vee$" de literales ("Disjunción").
Ejemplo: $x_1 \vee \overline{x_2} \ldots$
Una expresión Booleana en forma Conjuntiva Normal[32], es una conjunción ("$\wedge$") de claúsulas.
Ejemplo:

$$(x_1 \vee x_2) \wedge (\overline{x_1} \vee x_3 \vee \overline{x_7}) \wedge (x_2 \vee \overline{x_3} \vee x_4)$$

SAT es:[33] dada B en CNF:

¿$\exists$ un asignamiento de valores a las variables que la vuelva verdadera?

[32] CNF: Conjuctive Normal Form.
[33] Nota: "SAT", sería sin pedir CNF, esto sería "CNF-SAT, pero todo el mudo lo llama SAT".

$SAT \in NP$: Certificación una valuación.
Otras formas de SAT están en P.

- $DNF - SAT \in P$: $\vee's$ de $\wedge's$
- $Xor - SAT \in P$: $\wedge's$ de $\oplus's$ (Xors)

Pero SAT no se sabe, de hecho Cook probó que $SAT \in P \Rightarrow P = NP$
No probaremos este teorema pero daremos una "olfateada" al mismo.
La clave está en lo que Cook llamó: "Reducción Polinomial".
Que ahora también se conoce como "Reducción de Cook".

6.4.1. Reducción Polinomial

Definición: dados dos problemas de decisión $\pi = (I, S)$, $\tau(J, T)$
Diremos que π se reduce a τ ($\pi \leq_{\mathcal{P}} \tau$) si:
$\exists$ Polinomio $P(x)$ y una función $\psi : I \to J$:

1. Calcular $\psi(x)$ es $O(P(x))$ y $\psi(x)$ es polinomial en x.
2. $x \in S \Leftrightarrow \psi(x) \in \tau$ i.e. $\pi(x) =$ "Sí" $\Leftrightarrow \tau(\psi(x)) =$"Sí".

Ejemplos sencillos:

- Tonto y no del todo correcto:
  - π : Dada $ax^2 + b^x + c = 0$ ¿tiene solución $\in \mathbb{R}$?
  - τ : Dado $t \in \mathbb{R}$ ¿es $t \geq 0$?

  Entonces: $\pi \leq_{\mathcal{P}} \tau$ tomando $t = b^2 - 4 * a * c$
- Otro ejemplo
  - π : Dada A $\in \mathbb{Q}^{n*n}$ ¿A es invertible?
  - Dado $t \in \mathbb{Q}$ ¿es $t \neq 0$?

  $\pi \leq_{\mathcal{P}} \tau$ tomando $Det(A) = t$.

6.4.2. $k - color \leq_{\mathcal{P}} SAT$

Teorema: $k - color \leq_{\mathcal{P}} SAT$
Prueba: Dado un grafo G debemos construir polinomialmente una $B \in CNF$ tal que: $\chi(G) \leq k \Leftrightarrow B$ es satisfactible i.e. $\exists$ un asignamiento que la vuelve verdadera.
Sea $n = \#$ vértices de G.
Tomemos variables $x_{i,j} : 1 \leq i \leq n,\ 1 \leq j \leq k$
Para $i = 1, \ldots, n$ defino $C_i = x_{i,1} \vee x_{i,2} \vee \ldots \vee x_{i,k}$

$$C = C_1 \wedge C_2 \wedge \ldots \wedge C_n$$

$$D_i = \bigwedge_{\substack{j,r \\ j<r \\ j,r=1,\ldots,k}} \left(\overline{x}_{i,j} \vee \overline{x}_{i,r}\right)$$

$$D = D_1 \wedge \ldots \wedge D_n$$

Sea $F_{i,j,l} = \overline{x}_{i,j} \vee \overline{x}_{l,j}$

$$F = \bigwedge_{\substack{i,l=1 \\ il \in E}}^{n} \bigwedge_{j=1}^{k} F_{i,j,l}$$

Tomo $B = C \wedge D \wedge F$ es $O(n+m)$

Ahora hay que probar que $\chi(G) \leq k \Leftrightarrow B$ es satisfactible.

$\boxed{\Rightarrow}$ Sea C un coloreo propio con k colores de G.

Debemos construir un <u>asignamiento de valores a las variables</u>.

$x_{i,j} \to \widehat{x}_{i,j} \in \{0,1\}$

Definimos $\widehat{x}_{i,j} = \begin{cases} 1 & Si\ C(i) = j \\ 0 & c.c. \end{cases}$

Debemos ver que : $\widehat{B} = B(\widehat{x}_{i,j}) = 1$

Todo vértice tiene un color

$\Rightarrow \forall i\ \exists j \leq k : C(i) = j$

$\Rightarrow \forall i\ \exists j \leq k : \widehat{x}_{i,j} = 1$

$\Rightarrow \forall i\ : \underbrace{\widehat{x}_{i,1} \vee \widehat{x}_{i,2} \vee \ldots \vee \widehat{x}_{i,k}}_{\widehat{C_i}} = 1$

$$\therefore \widehat{C} = \widehat{C}_1 \wedge \ldots \wedge \widehat{C}_n = 1$$

Todo vértice tiene un <u>sólo</u> color

$\forall i\ \exists!\ j \leq k : C(i) = j$

$\therefore \forall i\ \nexists j \neq r : C(i) = j, C(i) = r$

$\forall i, \forall j < r :\ C(i) \neq j \vee C(i) \neq r$

$\widehat{x}_{i,j} = 0 \vee \widehat{x}_{i,r} = 0$

o lo mismo: $\overline{\widehat{x}}_{i,j} = 1$ ó $\overline{\widehat{x}}_{i,r} = 1$

$\overline{\widehat{x}}_{i,j} \vee \overline{\widehat{x}}_{i,r} = 1\ \forall i, \forall j < r \Rightarrow \widehat{D} = 1$

<u>Finalmente</u> como el coloreo es <u>propio</u>

$\forall il \in E$ Tenemos $C(i) \neq C(l)$

i.e. $\forall il \in E, \nexists j : C(i) = C(l) = j$

$\Rightarrow \forall il \in E, \forall j :$

$\underbrace{(C(i) \neq j \text{ ó } C(l) \neq (j)}_{\overline{\widehat{x}}_{i,j}=1 \text{ ó } \overline{\widehat{x}}_{i,r}=1}$

$$\therefore \forall il \in E, \forall j : \overline{\widehat{x}}_{i,j} \vee \overline{\widehat{x}}_{i,r} = 1 \Rightarrow \widehat{F} = 1$$

$$\widehat{B} = \widehat{C} \wedge \widehat{D} \wedge \widehat{F} = 1$$

Fin ($\Rightarrow$)

$\boxed{\Leftarrow}$ Ahora sabemos que <u>existe</u> $\widehat{x}_{i,j}$ con $\widehat{B} = 1$ y queremos ver que: $\chi(G) \leq k$

$$\widehat{B} = 1 \Rightarrow \left| \begin{array}{ll} \widehat{C} = 1 & \Rightarrow \forall i, \exists j : \widehat{x}_{i,j} = 1 \quad (\star) \\ \widehat{D} = 1 & \Rightarrow \forall i, \exists! j : \widehat{x}_{i,j} = 1 \text{ definimos } C(i) = \text{el único j con } \widehat{x}_{i,j} = 1 \\ \widehat{F} = 1 & \Rightarrow \forall ij \in E, C(i) \neq C(l) \therefore C \text{ es propio} \end{array} \right.$$

$\star$ Cuento con la demostración de la ida.

■

<u>Observación</u>: Cook probó $\pi \in NP \Rightarrow \pi \leq_{\mathcal{P}} SAT$
<u>Corolario</u>: $SAT \in P \Rightarrow P = NP$

6.5. NP Completo

<u>Definición</u>: un problema τ es **NP completo** si:

1. $\tau \in NP$
2. $\pi \in NP \Rightarrow \pi \leq_{\mathcal{P}} \tau$

Un año después de Cook, Karp dió otros 23 problemas NP completos, en la actualidad hay más de 2000.

La clave es lo siguiente: Si $\tau \in NP$ Completo y $\tau \leq_{\mathcal{P}} \tau^* : \tau^* \in NP$

Entonces τ^* es NP Completo.

La demostración sale del hecho que $\leq_{\mathcal{P}}$, es transitiva, ya que es una composición de polinomios.

A continuación daremos algunos ejemplos NP completos.

6.5.1. 3-SAT es NP completo

3-SAT es como SAT pero en cada claúsula debe haber exactamente 3 literales distintos.

<u>3-SAT es NP completo</u>

<u>Prueba</u>: probaremos que SAT $\leq_{\mathcal{P}}$ 3-SAT (por Cook).

Es decir dada B en CNF debemos construir polinomialmente una $\widehat{B}$ en CNF pero que tenga exactamente 3 literales por claúsula y tal que B sea satisfactible $\Leftrightarrow B'$ lo es.

Supongamos que $B = D_1 \wedge \ldots \wedge D_m : D_i$ claúsulas.

Variables de $B = x_1, \ldots, x_n$

Dada una claúsula D arbitraria, definiremos una <u>conjunción</u> de claúsulas E (con más variables), y luego tomaremos:

$$B' = E_1 \wedge E_2 \wedge \ldots \wedge E_m \text{ donde } E_i \text{ corresponde a } D_i.$$

Cada E_i crea sus <u>propias</u> variables extras.
<u>Construcción</u>:

- Si D tiene 3 literales: E =D
- Si D tiene 2 literales : $D = L_1 \vee L_2$

  Tomamos una variable extra $y \notin \{x_1, \ldots, x_n\}$.

  Tomamos $E = (L_1 \vee L_2 \vee y) \wedge (L_1 \vee L_2 \vee \overline{y})$

  Claramente si $\widehat{L_1} \vee \widehat{L_2} = 1 \Rightarrow (\widehat{L_1} \vee \widehat{L_2} \vee \widehat{y}) \wedge (\widehat{L_1} \vee \widehat{L_2} \vee \overline{\widehat{y}}) = 1$

  <u>Viceversa</u>:

  Si $\exists \{\widehat{x_1}, \ldots, \widehat{x_n}, \widehat{y}\}$

  Tal que $(\widehat{L_1} \vee \widehat{L_2} \vee \widehat{y}) \wedge (\widehat{L_1} \vee \widehat{L_2} \vee \overline{\widehat{y}}) = 1$

  Entonces si:

  - $\widehat{y} = 0 \Rightarrow \widehat{L_1} \vee \widehat{L_2} = 1$
  - $\widehat{y} = 1 \Rightarrow \widehat{L_1} \vee \widehat{L_2} = 1$

  <u>Observación</u>: probamos un poco más de lo que dijimos

  $$\exists \vec{\widehat{x}} : D\left(\vec{\widehat{x}}\right) = 1 \Leftrightarrow \exists \vec{\widehat{y}} : E\left(\vec{\widehat{x}}, \vec{\widehat{y}}\right) = 1$$

  Esto valdrá en todos los casos.

- D tiene un literal: i.e. $D = L$

  Tomamos:

  $$E = (L \vee y_1 \vee y_2) \wedge (L \vee y_1 \vee \overline{y_2}) \wedge (L \vee \overline{y_1} \vee y_2) \wedge (L \vee \overline{y_1} \vee \overline{y_2})$$

  Y es casi obvio que:

  $$\widehat{L} = 1 \Leftrightarrow \widehat{E} = 1 \text{ i.e. cualquier } \widehat{y_1}, \widehat{y_2}$$

- D tiene ≥ 4 literales.

  $D = L_1 \vee L_2 \vee \ldots \vee L_k$

  Entonces tomamos $k - 3$ variables nuevas: $y_1, y_2, \ldots, y_{k-3}$ polinomial en k

  <u>$K >> 4$</u>:

  $$E = (L_1 \vee L_2 \vee y_1) \wedge (\overline{y_1} \vee y_2 \vee L_3) \wedge (\overline{y_2} \vee y_3 \vee L_4) \wedge \ldots \wedge (\overline{y}_{k-4} \vee y_{k-3} \vee L_{k-2}) \wedge (\overline{y}_{k-3} \vee L_{k-1} \vee L_k)$$

#claúsulas $= k-2$ $\therefore$ lineal en k.
Supongamos ahora $\widehat{D} = 1 \Rightarrow \exists j : \widehat{L}_j = 1$
<u>Definamos</u>:

$$\widehat{y_1} = \widehat{y_2} = \ldots = \widehat{y}_{j-2} = 1$$
$$\widehat{y}_{j-1} = \widehat{y}_j = \ldots = \widehat{y}_{k-3} = 0$$

$$\widehat{E} = (\widehat{L}_1 \vee \widehat{L}_2 \vee \underbrace{\widehat{y_1}}_{=1}) \wedge (\overline{\widehat{y}}_1 \vee \underbrace{\widehat{y_2}}_{=1} \vee \widehat{L}_3) \wedge \ldots \wedge (\overline{\widehat{y}}_{j-2} \vee \widehat{y}_{j-1} \vee \underbrace{\widehat{L}_j}_{HIP=1}) \wedge (\underbrace{\overline{\widehat{y}}_{j-1}}_{=1} \vee \widehat{y}_j \vee \widehat{L}_{j+1})$$
$$\wedge \ldots \wedge (\underbrace{\overline{\widehat{y}}_{k-3}}_{=1} \vee \widehat{L}_{k-1} \vee \widehat{L}_k)$$

$\therefore \widehat{E} = 1$
<u>Veamos la vuelta</u>: supongamos que $\widehat{E} = 1$
Queremos ver que $\widehat{D} = 1$. Supongamos que **no**.
$\widehat{D} = 0 \Rightarrow \widehat{L_1} \vee \ldots \vee \widehat{L_k} = 0 \Rightarrow \widehat{L}_r = 0 \ \forall r$
Nos queda: $\widehat{E} = \widehat{y_1} \wedge \left(\overline{\widehat{y}}_1 \vee \widehat{y_2}\right) \wedge \left(\overline{\widehat{y}}_2 \vee \widehat{y_3}\right) \wedge \ldots \wedge \left(\overline{\widehat{y}}_{k-4} \vee \widehat{y}_{k-3}\right) \wedge \overline{\widehat{y}}_{k-3}$
Como $\widehat{E} = 1$
Tenemos:

$$\left.\begin{array}{r} \left.\begin{array}{r} \left.\begin{array}{r} \left.\begin{array}{r} \widehat{y_1} = 1 \\ \overline{\widehat{y}}_1 \vee \widehat{y_2} = 1 \end{array}\right\} \Rightarrow \widehat{y_2} = 1 \\ \overline{\widehat{y}}_2 \vee \widehat{y}_3 = 1 \end{array}\right\} \Rightarrow \widehat{y}_3 = 1 \\ \overline{\widehat{y}}_3 \vee \widehat{y}_4 = 1 \end{array}\right\} \ldots \Rightarrow \widehat{y}_{k-4} = 1 \\ \ldots \ \overline{\widehat{y}}_{k-4} \vee \widehat{y}_{k-3} = 1 \end{array}\right\} \Rightarrow \widehat{y}_{k-3} = 1 \\ \overline{\widehat{y}}_{k-3} = 1 \end{array}\right\} \textbf{Absurdo}$$

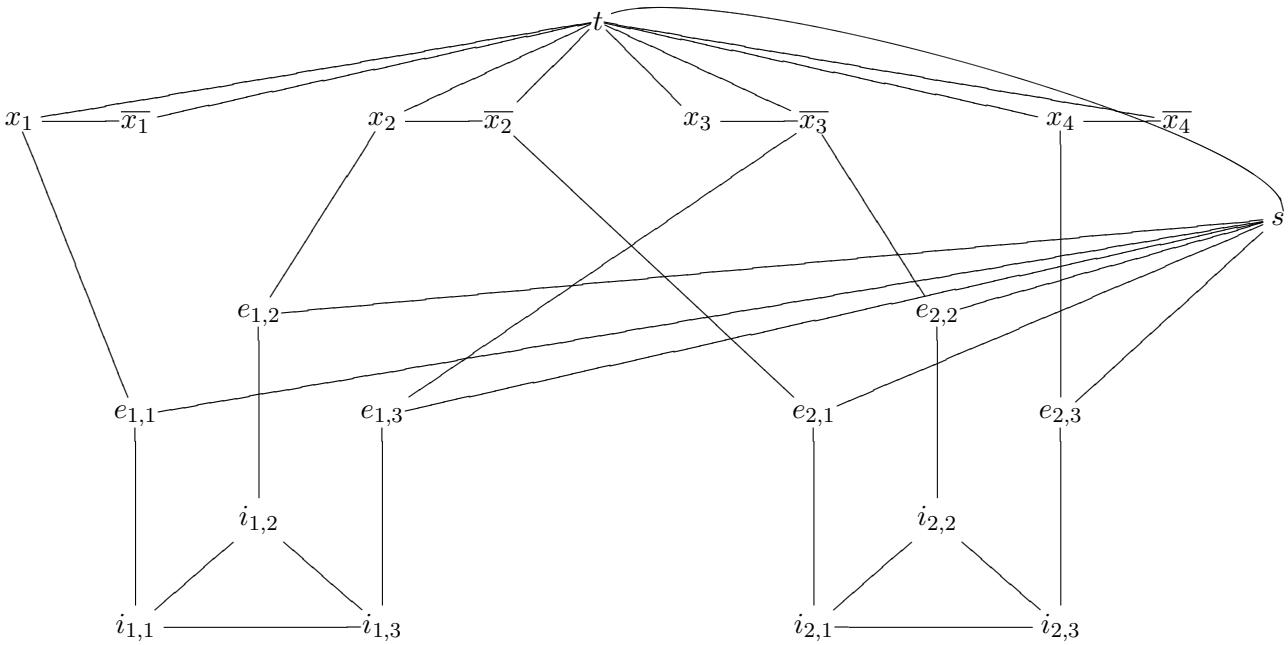
■

6.5.2. 3-color es NP completo

<u>Teorema</u>: 3-color es NP completo
<u>Prueba</u>: veremos que 3-SAT $\leq_{\mathcal{P}}$ 3-color.
Es decir necesitamos : $B(3\text{-}CNF) \to G$
Sea $B = D_1 \wedge \ldots \wedge D_m$ con variables $\{x_1, \ldots, x_n\}$.
$D_j = L_{1,j} \vee L_{2,j} \vee L_{3,j}$
Vamos a construir $G = (V, E)$ con $|V|, |E|$ polinomiales en n, m
Ya damos la construcción en general, primero un ejemplo.

6.5.3. Ejemplo 3-SAT $\leq_P$3-color

$$B = (x_1 \vee x_2 \vee \overline{x}_3) \wedge (\overline{x}_2 \vee \overline{x}_3 \vee x_4)$$



<u>En general</u>:

$$V = \{s, t\} \cup \{x_1, \ldots, x_n, \overline{x}_1, \ldots, \overline{x}_n\} \cup \{W_1, \ldots, W_m\}$$

Con $W_j = \{e_{j,1}, e_{j,2}, e_{j,3}, i_{j,1}, i_{j,2}, i_{j,3}\}$

$$E = \{st\} \cup \{tx_k, t\overline{x}_k, x_k\overline{x}_k\}_{k=1}^{n} \cup T_1 \cup \ldots \cup T_m \cup E_1 \cup \ldots \cup E_m \cup F_1 \cup \ldots \cup F_m \cup \{se_{j,k}\}_{\substack{1 \leq j \leq m \\ k = 1,2,3}}$$

Donde:

$$T_j = \{i_{j,1}i_{j,2}; i_{j,2}i_{j,3}; i_{j,3}i_{j,1}\}$$
$$E_j = \{e_{j,1}i_{j,1}; e_{j,2}i_{j,2}; e_{j,3}i_{j,3}\}$$
$$F_j = \{e_{j,1}L_{1,j}; e_{j,2}L_{2,j}; e_{j,3}L_{3,j}\}$$

G tiene: $2 + 2n + 6m$ vértices, polinomial.
G tiene: $1 + 3n + 3m + 9m = 1 + 3n + 12m$ lados, polinomial.

Hay que ver B satisfactible $\Leftrightarrow \chi(G) \leq 3$

(Como G tiene triángulos $\chi(G) \leq 3 \Leftrightarrow \chi(G) = 3$)

Veamos $\boxed{\Leftarrow}$ primero:

Sea C un coloreo de G de 3 colores queremos dar un asignamiento de valores a las variables tal que: $B(\widehat{\vec{x}}) = 1$

Definamos:

$$\widehat{x_k} = \begin{cases} 1 & \text{Si } C(x_k) = C(s) \\ 0 & \text{Sino} \end{cases}$$

Queremos ver que $B(\widehat{\vec{x}}) = 1$

$$\therefore \forall j \text{ basta ver: } D_j(\widehat{\vec{x}}) = 1$$

$\therefore$ Hay que probar $\forall j\ \widehat{L}_{1,j} \vee \widehat{L}_{2,j} \vee \widehat{L}_{3,j} = 1$

<u>Veamos</u> como T_j es un triángulo entonces los 3 colores están representados.

Entonces:

- $\exists k : C(i_{j,k}) = C(t)$
- $i_{j,k}e_{j,k} \in E \Rightarrow C(e_{j,k}) \neq C(i_{j,k}) = C(t)$
- $se_{j,k} \in E \Rightarrow C(e_{j,k}) \neq C(s)$
- $st \in E \Rightarrow C(s) \neq C(t)$
- $C(e_{j,k}) =$ "tercer color"

  $$\left.\begin{array}{l} \text{Como } e_{j,k}L_{j,k} \in E \Rightarrow C(L_{k,j}) \neq \text{"tercer color"} \\ tL_{k,j} \in E \Rightarrow C(L_{k,j}) \neq C(t) \end{array}\right\} \Rightarrow C(L_{k,j}) = C(s)$$

Por definición $\widehat{L}_{k,j} = 1$ **Fin** $\boxed{\Leftarrow}$

$\boxed{\Rightarrow}$ Dado un asignamiento de valores $\widehat{\vec{x}} : B(\widehat{\vec{x}}) = 1$ debemos construir un <u>coloreo propio</u> con 3 colores.

<u>Definimos</u>: $C(s) = 1$, $C(t) = 2 \Rightarrow st$ No Crea Problemas (NCP).

Coloreamos:

$$\left.\begin{array}{c} C(x_i) = \widehat{x_i} \\ C(\overline{x}_i) = 1 \oplus \widehat{x_i} \end{array}\right\} \Rightarrow x_i\overline{x}_i \qquad (NCP)$$

<u>Además</u>:

$$\left.\begin{array}{c} C(x_i), C(\overline{x}_i) \in \{0,1\} \\ C(t) = 2 \end{array}\right\} \Rightarrow tx_i, t\overline{x}_i \qquad (NCP)$$

Como $B(\widehat{\vec{x}}) = 1$

$\Rightarrow D_j(\widehat{\vec{x}}) = 1\ \forall j = 1, \ldots, m$

i.e. $\widehat{L_{1,j}} \vee \widehat{L_{2,j}} \vee \widehat{L_{3,j}} = 1$

$\Rightarrow \forall j, \exists k : \widehat{L_{k,j}} = 1$ (si hay más de uno elijo alguno)

<u>Coloreamos</u>:

$$\left.\begin{array}{c} C(e_{j,k}) = 0 \\ \text{Como } \widehat{L}_{k,j} = 1 \Rightarrow C(L_{k,j}) = 1 \end{array}\right\} \Rightarrow L_{k,j}e_{j,k} \qquad (NCP)$$

Y coloreo:

$$\left.\begin{array}{c} C(e_{j,r}) = 2 : r \neq k \\ C(L_{r,j}) \in \{0,1\} \end{array}\right\} \Rightarrow e_{j,r}L_{r,j} \qquad (NCP)$$

Como:

$$\left.\begin{array}{r} C(e_{j,k}) = 0 \\ C(e_{j,r}) = 2 \\ C(s) = 1 \end{array}\right\} \Rightarrow se_{j,k} \text{ y los } se_{j,r} \qquad (NCP)$$

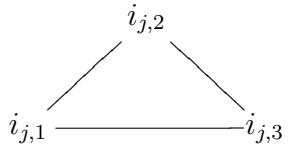
Finalmente coloreamos:

$$\left.\begin{array}{r} C(i_{j,k}) = 2 \\ C(e_{j,k}) = 0 \end{array}\right\} \Rightarrow e_{j,k} i_{j,k} \qquad (NCP)$$

Y coloreamos: $C(i_{j,r}) = 0$ ó $1 : r \neq k$

$$\therefore \underbrace{i_{j,r}}_{0 \text{ ó } 1} \underbrace{e_{j,r}}_{=2} \qquad (NCP)$$

Finalmente el triángulo:



Tiene un vértice de color 2 el (j,k) uno de color 0 y otro de color 1 $\qquad (NCP)$

■

6.5.4. 2-SAT $\in$ P

$D[j]$ = Claúsula j-ésima, un par de números (L, R).

Registros	x[0]	, ...	, x[n-1]	; x[n]	, ...,	x[2n-1]
Representan	$\widehat{x}_1$	, ...	, $\widehat{x}_n$	; $\widehat{\widehat{x}}_1$	, ...,	$\widehat{\widehat{x}}_n$

<u>2-SAT</u>

```
Variables: x_0, ..., x_{n-1}
SETEAR(v) : //MACRO
        X[v] = 1;
        X[2n - 1 - v] = 0;
        Q = Q ∪ P[v] ∪ P[2n - 1 - v]
N = {0 ,... ,n-1} ; //Variables no seteadas
M = {0 ,... ,m-1} ; //Disjunciones no satisfechas
Q = ∅;
for (i = 0 , i < 2n, i++){
    x[i] = -1 ;
    P[i] = {j ∈ M : i aparece en D[j]};
}
for (j = 0 , j < m, j++){ Sat[j] = 0; }
while( N ≠ ∅)
    Tomar i ∈ N, SETEAR(i);
    while(Q ≠ ∅){
        Elejir j ∈ Q;
            L,R: D(j) = {L,R};
            if ((x[L] == 1) ||(x[R] == 1)){
                Sat[j] = 1;
                Remover j de Q; //Pero no de M
            }else if( x[R]<0){
                SETEAR(R);
                /*Si X[R] < 0 ,debe ser x[i] = 0 entonces debo asignar x[R]
                    = 1*/
            }else if(x[i]<0){
                SETEAR(L)
                /*Este estado es cuando queda el caso 00, por lo tanto nos
                    equivocamos coon el seteo de x[i]= 1, por lo tanto hay
                    que cambiar*/
            }else if(x[i] == 1){ //Anular todos
                ∀t ∈ N, ∀s ∈ M{
                    SAT(s) = 0;
                    x[t] = x[2n-1-t] = -1;
                }
                Q = ∅;
                SETEAR[2n-1-i]
            }else
                return "NO SAT"
    }
    ∀i ∈ N, ∀j : SAT[j] = 1{
        Borrar j de P[i] y P[2n-1-i]
    }
}
Return "SAT"
```

6.5.5. HORN-SAT

HORN-SAT es como CNF-SAT pero donde cada claúsula tiene a lo sumo un literal "positivo" : $x_i \to$ "positivo" y $\overline{x_i} \to$ "negativo"

<u>Teorema</u>: $HORN\text{-}SAT \in P$

<u>Prueba</u>:

$$\begin{aligned}\overline{x_1} \vee \overline{x_2} \vee \ldots \vee \overline{x_k} \vee x_{k+1} &\equiv \overline{x_1 \wedge x_2 \wedge \ldots \wedge x_k} \vee x_{k+1} \\ &\equiv x_1 \wedge x_2 \wedge \ldots \wedge x_k \Rightarrow x_{k+1}\end{aligned}$$

El algoritmo empieza seteando todas las variables a $false$.

Recordamos: $true \Rightarrow true \equiv true$

$False \Rightarrow \star \equiv true$

$true \Rightarrow false \equiv false$

Entonces todas las disjunciones de la forma:

$$\Big(\underbrace{x_1 \wedge x_2 \wedge \ldots \wedge x_k}_{false} \Rightarrow \underbrace{x_{k+1}}_{false}\Big) \text{ van a ser } true$$

Las de la forma:

$$\begin{aligned}\overline{x_1} \vee \overline{x_2} \vee \ldots \vee \overline{x_k} &\equiv \overline{x_1} \vee \overline{x_2} \vee \ldots \vee \overline{x_k} \vee false \\ &\equiv \underbrace{x_1 \wedge x_2 \wedge \ldots \wedge x_k}_{false} \Rightarrow false \\ &\equiv true\end{aligned}$$

El único problema son las disjunciones de la forma:

$$\begin{aligned}x \equiv true &\Rightarrow \underbrace{x}_{false} \\ &\equiv false\end{aligned}$$

Si hay una de esas, listo.

Si hay, seteamos estas variables a $true$ (permanentemente).

Luego de eso se hacen $true$ todas las variables x_{k+1} que estén en una disjunción de la forma: $x_1 \wedge \ldots \wedge x_k \Rightarrow x_{k+1}$

En donde $x_1, \ldots, x_k$ sean $true$.

Si en algún momento se llega a una contradicción, no se puede satisfacer.

La contradicción vendría de alguna de la forma:

$$\overline{x_1} \vee \overline{x_2} \vee \ldots \vee \overline{x_k} \text{ con todas las } x_i \; true$$

■

<u>Ejemplo</u>:

$(\overline{x} \vee \overline{y} \vee \overline{z} \vee w) \wedge (\overline{x} \vee \overline{y}) \wedge (\overline{z} \vee \overline{w} \vee x) \wedge (\overline{x} \vee z) \wedge (\overline{y} \vee z) \wedge y$

$\widehat{y} = 1 \Rightarrow \widehat{z} = 1 \Rightarrow \widehat{x} = 0, \widehat{z} = 0$

Pero supongamos que siguiera con $\ldots \wedge (\overline{y} \vee \overline{z} \vee U) \wedge (\overline{u} \vee x)$

$\widehat{y} = 1 \Rightarrow \widehat{z} = 1 \Rightarrow \widehat{U} = 1 \Rightarrow \widehat{x} = 0$

$\therefore$ No se puede resolver $\overline{x} \vee \overline{y}$

7. Introducción a la Inteligencia Artificial

7.1. Hill Climbing

7.2. Simulated Annealing

7.3. Algoritmos Evolutivos

Tomar una población de posibles soluciones (por ejemplo una cadena de bits).

1. Elegir un miembro de la población.
2. Cambiar cada bit de ese miembro y evaluar si el cambio mejora la solución.

   Nos quedamos con el mejor:

   - Si hay uno repetir 2.
   - Sino, guardar el individuo en un conjunto B, y repetir 1.
3. Finalmente elegir el mejor de B.

Este algoritmo es evolutivo pero no genético

7.4. Algoritmos Genéticos

Genético $\Rightarrow$ Interacción entre individuos.

Los algoritmos genéticos[34], se basan más o menos en la evolución biológica.

Cada individuo, tiene cromosomas, que forman cadenas de ADN, se dividen en "Genes" que codifican un aspecto del individo, por ejemplo el color de los ojos.

El conjunto de cromosomas se llama "genotipo" y las características inducidas en el individuo por la acción del genotipo y el medio ambiente es el "fenotipo".

También podemos distinguir entre especies haploides y diploides, (humanos).

Los cromosomas vienen en pares con genes dominantes y recesivos.

Haploide: los cromosomas no están emparejados.

En la mayoría de los algoritmos genéticos los individuos son haploides y con un solo cromosoma.

A veces a los individuos se les llama "cromosomas". Pero a veces el individuo tiene algo más que cromosomas.

Una parte importante de un AG es codificar el problema con cromosomas.

Un cromosoma será una cadena de símbolos, usualmente bits o un vector de números.

Ejemplos:

1. Maximizar una función en $\mathbb{R}^3$ la codificación puede ser una cadena de 3 números.
2. Hallar un recorrido de n ciudades que no repita ninguna y minimice la distancia (TSP)[35]. Codifica una cadena con el nombre de cada ciudad (sin repetir).

[34] AG.

[35] Travel Salesman Problem.

En todo AG necesitamos una función a maximizar, llamada función de "**Fitness**[36]".
En los ejemplos anteriores:

1. f es la función.

2. Sería la f del costo del camino.

A veces como en TSP no toda cadena representa una solución, por ejemplo puede ser que no haya camino entre las dos ciudades.

Generalmente se soluciona "creativamente", por ejemplo en TSP, podemos suponer que hay ruta entre 2 ciudades cualesquiera, asignándole, un peso muy grande si no existe en la realidad. Pero en otros problemas esto es más delicado.

$\therefore$ Es la primera característica de un AG.

1) Una población de individuos con cromosomas y una función de fitness adecuada.

Otra característica es:

2) Selección de individuos a reproducirse mediante la fitness.

Un AG tendrá lo que se llaman "Generaciones".

$$\begin{array}{c}\text{Población Inicial (generación 0)}\\ \Downarrow\\ \text{Algo}\\ \Downarrow\\ \text{Población nueva (generación 1)}\\ \Downarrow\\ \text{Algo}\\ \Downarrow\\ \text{Población nueva (generación 2)}\end{array}$$

7.4.1. Selección

Los individuos a reproducirse se seleccionan de a pares y luego viene lo esencial:

3) Los progenitores producen nuevos individuos ("**offspring**") por medio del cruzamiento ("**crossover**") de genes.

4) Los "offspring" son sometidos a mutaciones al azar. Operador **mutación**.

Esto depende de una tasa de mutación, generalmente baja $\approx 0,001$

5) Debe haber un operador de Selección entre Progenitor 1, Progenitor 2, offspring 1 y offspring 2. (es decir tengo que matar a 2).

6) Debe haber una condición de terminación.

Por ejemplo, un número máximo de generaciones.

7.5. Ejemplo de un AG

Problema: dado un número natural escribirlo como suma, resta, producto y división de los dígitos: $0, 1, \ldots, 9$ quizás repetidos (asociando a izquierda).

[36] Adaptabilidad.

7.5.1. Codificación del ejemplo

Una posibilidad sería una cadena de símbolos del alfabeto $A = \left\{0, \ldots, 9, +, *, /\right\}$ pero que debe tener: $N^{\underline{o}}$ Símbolo $N^{\underline{o}}$.

Problema: crossover y mutaciones se pueden complicar.

Una posibilidad es hacer cadenas de pares ($N^{\underline{o}}$,operador) u (operador, $N^{\underline{o}}$) y suponer que empezamos con 0.

Otra posibilidad: simplemente usar cadenas de bits, entonces un gen serán 4bits.

$$\begin{array}{ll} 0000 \to 0 & 1011 \to - \\ 0001 \to 1 & 1100 \to * \\ \vdots & 1101 \to / \\ 1001 \to 9 & 1110 \to Nada \\ 1010 \to + & 1111 \to Nada \end{array}$$

El cromosoma será el genotipo, el fenotipo en cambio tiene que ser una cadena:
0 Símbolo $N^{\underline{o}}$ Símbolo ... que además no tenga "/0".

$$Genotipo \to Fenotipo$$

- Si es 14 ó 15 lo descartamos.
- Si es $N^{\underline{o}}$ y se requiere un Símbolo lo descartamos.
- Si es un Símbolo y se requiere un $N^{\underline{o}}$ lo descartamos.
- Sino se usa.
- Si tenemos un "/0" cambiar 0 por 1.

7.5.2. Fitness del ejemplo

Fitness será algo como : $= \begin{cases} \frac{1}{\sum Fenotipo - 31} & \text{si } \sum Fenotipo \neq 31 \\ \text{“}\infty\text{”} & \text{si } \sum Fenotipo = 31 \end{cases}$

7.5.3. Entrecruzamiento del ejemplo

Crossover: Singled Point Crossover
Progenitor 1: $b_1 b_2 b_3 \ldots b_{400}$
Progenitor 2: $\widehat{b_1}\widehat{b_2}\widehat{b_3} \ldots \widehat{b_{400}}$
Elijo al azar un número $x : 0 < x < 400$ y obtengo:
Por ejemplo $x = 127$
Offspring 1: $b_1 b_2 b_3 \ldots b_{127}\widehat{b}_{128} \ldots\ldots \widehat{b}_{400}$
Offspring 2: $\widehat{b_1}\widehat{b_2}\widehat{b_3} \ldots \widehat{b}_{127} b_{128} \ldots\ldots b_{400}$

7.5.4. Mutación del ejemplo

Para cada bit lo cambio con probabilidad p.

7.5.5. Selección del ejemplo

Mato a los padres, me quedo con los offsprings.

7.5.6. Operadores de Selección

Asumimos que cada individuo tiene una fitness F_i calculada.

Queremos seleccionar de los n individuos, quiénes se van a reproducir, tomando, en cuenta las F_i.

Hay varios métodos, los más usados caen en un categoría llamada:

"Fitness Proportional Selection"

- Sea $\overline{F} = \frac{\sum_{i=1}^{n} F_i}{n}$ la fitness promedio.
- Sea $E_i = \frac{F_i}{\overline{F}} = \#$ esperado de veces que el individuo i, debería ser seleccionado de acuerdo a su fitness.

$$E_i = n * \frac{F_i}{\sum_{j=1}^{n} F_j}$$

Queremos seleccionar i con un método que lo elija cerca de E_i veces.

El primer método propuesto por Holland, hace esto lo más cercano posible se llama "Stochastic Sampling With Replaciment" (SSWR), o **Ruleta**.

Elijo un número al azar y elijo ese número.

Calculando las $E_i's$ se toma un número al azar A entre 0 y n y se toma el primer $i : \sum_{j=1}^{i} E_j > A$

Ejemplo: 6 individuos.

"Fitnesses"

$$\begin{aligned} F_1 &= 17{,}118 \\ F_2 &= 9{,}51 \\ F_3 &= 36{,}138 \\ F_4 &= 22{,}824 \\ F_5 &= 91{,}296 \\ F_6 &= 13{,}314 \end{aligned} \qquad \therefore \sum_{j=1}^{6} F_j = 190,200$$

Lo más simple es tomar un número al azar entre 0 y 1 que por ejemplo tomando $r \in (0,1)$ y calculo $r * 190,2$.

Para entender mejor otros conceptos, haremos el cálculo sin embargo con los $E_i's$

$$E_i = \frac{F_i}{190,2} * 6 \Rightarrow \begin{cases} E_1 = 0,54 \\ E_2 = 0,30 \\ E_3 = 1,14 \\ E_4 = 0,72 \\ E_5 = 2,88 \\ E_6 = 0,42 \end{cases}$$

Ruleta: # al azar: entre 0 y 1:
$0,15 \quad 0,7 \quad 0,22 \quad 0,91 \quad 0,46 \quad 0,63$
Multiplico por $n = 6$
$0,90 \quad 4,2 \quad 1,32 \quad 5,46 \quad 2,76 \quad 3,78$

Ahora calculo: $\sum_{j=1}^{i} E_j \Rightarrow \begin{cases} 1 \to 0,54 & \\ 2 \to 0,84 & \\ 3 \to 1,98 & 1^\bullet\ 3^\bullet \\ 4 \to 2,70 & \\ 5 \to 5,58 & 2^\bullet, 4^\bullet, 5^\bullet, 6^\bullet \\ 6 \to 6 & \end{cases}$

Este método puede producir lo que se llama "**Crowding**" de la población por un individuo o grupo de individuos y "convergencia temprana" a un máximo local, además tiene otro problema.

Hay 3 medidas de un operador de selección:

- Bias.
- Spread.
- Eficiencia computacional.

Bias: es la diferencia entre $\frac{F_i}{\overline{F_i}}$ y el valor esperado de veces que el individuo será seleccionado con el método propuesto.

- $Bias(Ruleta) = 0$.

Spread: rango de offspring que un individuo puede tener:

- $Ruleta(Spread(i)) = [0, n]$.

Conclusión: ruleta tiene Bias = 0 y Spread Máximo (malo).
Eficiencia: buena, pero se necesitan n randoms.
Alternativa a Ruleta: "Stochastic Universal Sampling" (SUS), usa un sólo número al azar, digamos r y toma los puntos: $r, r+1, r+2, \ldots, r+(n-1)$.

En nuestro caso: $r = 0,15$

0,15	1,15	2,15	3,15	4,15	5,15
$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$
I_1	I_3	I_4	I_5	I_5	I_5

Entonces nos queda:

- $\begin{array}{ll} 1 \to 1 & (0,54) \\ 2 \to 0 & (0,34) \\ 3 \to 1 & (1,14) \\ 4 \to 1 & (0,72) \\ 5 \to 3 & (2,88) \\ 6 \to 0 & (0,42) \end{array}$
- $Bias = 0$.
- Pero $Spread(i) = \{\lfloor E_i \rfloor, \lceil E_i \rceil\}$.
- Y un factor extra que tengo que elegir a quién reproduzco con quién.

7.5.7. Remainde Methods (Métodos del "Resto")

El individuo, se selecciona determinísticamente.
Se toma $\lfloor E_i \rfloor$ veces.
Los $n - \sum_{i=1}^{n} \lfloor E_i \rfloor$ posibles se llenan con algún otro método.

Por ejemplo se usan los restos: $R_i = E_i - \lfloor E_i \rfloor$ y se distribuyan usando Ruleta o SUS.

<u>Ejemplo</u>:

$$\begin{array}{lll}
\lfloor E_1 \rfloor = \lfloor 0,54 \rfloor & = 0 & \rightarrow R_1 = 0,54 \\
\lfloor E_2 \rfloor = \lfloor 0,30 \rfloor & = 0 & \rightarrow R_2 = 0,30 \\
\lfloor E_3 \rfloor = \lfloor 1,14 \rfloor & = \boxed{1} & \rightarrow R_3 = 0,14 \\
\lfloor E_4 \rfloor = \lfloor 0,72 \rfloor & = 0 & \rightarrow R_4 = 0,72 \\
\lfloor E_5 \rfloor = \lfloor 2,88 \rfloor & = \boxed{2} & \rightarrow R_5 = 0,88 \\
\lfloor E_6 \rfloor = \lfloor 0,42 \rfloor & = 0 & \rightarrow R_6 = 0,42
\end{array}$$

Entonces me quedan 3 slots, por ejemplo uso ruleta para asignarlos.
Elijo 3 números al azar: $r = 0,15|0,7|0,22$.

Multiplico por $n = 3$

$0,45$	$2,1$	$0,6$
$\downarrow$	$\downarrow$	$\downarrow$
I_1	I_5	I_2

<u>Orden</u>

0,54	0,54
0,30	0,84
0,14	0,98
0,72	1,7
0,88	2,58
0,42	3

El asignamiento final nos queda: $\begin{cases} 1 \rightarrow 1 \\ 2 \rightarrow 1 \\ 3 \rightarrow 1 \\ 4 \rightarrow 0 \\ 5 \rightarrow 3 \\ 6 \rightarrow 0 \end{cases}$ y luego apareo al azar.

7.5.8. Otros métodos Fitness Proportional

Los métodos que hemos visto con $BIAS = 0$ o casi pueden producir que al principio cuando hay gran disparidad de Fitness, un grupo de individuos con alta fitness relativa, cope la población.

Impidiendo una búsqueda exhaustiva, de todo el espacio de búsqueda, por otro lado, luego de que se estabilice la población es difícil para un individuo con mayor Fitness influenciar la búsqueda produciendo un estancamiento.

Por esto se han propuesto alternativas.

7.5.9. Sigma Scaling

En este método se calculan las E_i en forma distinta $\widehat{E_i}$

$$\text{Se define } \widehat{E_i} = \begin{cases} 1 + \frac{F_i - \overline{F}}{2\sigma} & \sigma \neq 0 \\ 1 & \sigma = 0 \end{cases} : \sigma = \text{desvío estándar de los } F_i.$$

Ventaja: al principio, σ suele se grande, entonces se evita "Crownding", al final $\sigma \approx 0$. Entonces el mejor individuo se va a reproducir mucho más.
Ejemplo: con las Fitness de antes obtengo $\sigma = 30,627$
Y voy calculando los $\widehat{E_i}$

$$\text{Primero: } \widehat{E_1} = 1 + \frac{17,118 - 31,7}{2\sigma} = 0,762$$

$\widehat{E_i}$	$\sum_{j=1}^{i} E_j$
$\widehat{E_1} = 0,762$	$0,762$
$\widehat{E_2} = 0,638$	$1,400$
$\widehat{E_3} = 1,072$	$2,472$
$\widehat{E_4} = 0,855$	$3,327$
$\widehat{E_5} = 1,973$	$5,3$
$\widehat{E_6} = 0,700$	6

Aplico Ruleta
Elijo n = 6, números al azar

$0,15 \quad 0,7 \quad 0,22 \quad 0,91 \quad 0,46 \quad 0,63$
Multiplico por $n = 6$
$0,90 \quad 4,2 \quad 1,32 \quad 5,46 \quad 2,76 \quad 3,78$
$I_2 \quad I_5 \quad I_2 \quad I_6 \quad I_4 \quad I_5$
Obtengo algo más balanceado.

Si hubiera aplicado SUS: con $r = 0,15$

$0,15 \quad 1,15 \quad 2,15 \quad 3,15 \quad 4,15 \quad 5,15$
$I_1 \quad I_2 \quad I_3 \quad I_4 \quad I_5 \quad I_5$

Roletamiento

$\widehat{E}_i = \frac{e^{\frac{F_i}{T}}}{\sum e^{\frac{r_j}{T}}}$: T temperatura, inicialmente alta y luego se enfría

Otros métodos no usan Fitness Proporcion

Por ejemplo: métodos de Rango, se usan las F_i para ordenar los individuos y luego se elije por algún método.

Basándonos sólo en la posición en ese ordenamiento.

Otras Selecciones: no tan basadas en Fitness.

- Métodos de rango: se usa la Fitness para ordenar la población y luego se usa una medida que dependa sólo de la posición en esa población.
- Lineal Ranking: $LR(pos) = (2 - SP) + 2(SP - 1)\left(\frac{pos-1}{n-1}\right)$

  SP parámetro en $[1, 2]$

  SP es por "Selective Pressure"

  Estas tomarían el lugar de los E_i

  i.e. sería: $E_i = LR(Pos(i))$

Por ejemplo $SP = 1 \Rightarrow LR(pos) = 1 \forall pos$, no hay presión.
$SP = 2 \Rightarrow LR(pos) = 2\left(\frac{pos-1}{n-1}\right)$
$SP \in (1; 2)$, por ejemplo $SP = 1,2$

$$LR(pos) = 0,8 + 0,4\left(\frac{pos-1}{n-1}\right)$$

SP no tiene que ser fijo, podría ir aumentando para aumentar la presión.
<u>Ejemplo</u>: con las Fitness del ejemplo anterior

$$\begin{array}{cccccc} F_2 & < F_6 & < F_1 & < F_4 & < F_3 & < F_5 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 1 & 2 & 3 & 4 & 5 & 6 \end{array}$$

Tomando $SP = 1,2$:

$$\begin{array}{l} E_1 = LR(3) = 0,96 : 3 \text{ es la posición} \\ E_2 = LR(1) = 0,8 \\ E_3 = LR(5) = 1,12 \\ E_4 = LR(4) = 1,04 \\ E_5 = LR(6) = 1,2 \\ E_6 = LR(2) = 0,8 + \frac{0,4}{5} = 0,88 \end{array}$$

<u>Selección "Torneo"</u>
Seleccionar k (usualmente 2) individuos y quedarse con el "mejor", en un torneo determinísticamente o probabilísticamente.

7.5.10. Crossover

- SinglePoint
- TwoPointCroospoint
- Máscara Binaria

Supongo que tengo esta máscara, generada al azar: 1,0,0,1,1,0,1,0,1,1.

$$\begin{array}{rcccccccccc} P_1 = & \boxed{2} & 9 & 7 & \boxed{2} & \boxed{5} & 3 & \boxed{1} & 6 & \boxed{5} & \boxed{8} \\ P_2 = & 4 & \boxed{2} & \boxed{3} & 6 & 6 & \boxed{3} & 9 & \boxed{8} & 0 & 9 \end{array}$$

Obtengo:

$$O_1 = 2, 2, 3, 2, 5, 3, 1, 8, 5, 8$$
$$O_2 = 4, 9, 7, 6, 6, 3, 9, 6, 0, 9$$

Partial Map Crossover

¿Qué pasa con codificación del problema que depende de la posición?
$P_1 = BIGAECJFDH$
$P_2 = DJBFECIGAH$
Corta en dos lugares:
$O_1 = BIG|JECA|FDH$
$O_2 = DJB|FIEC|GAH$
También se puede hacer con máscaras. "Order Based"

Cyclic Crossover

VERRRRRRRRRRRRRRRR

7.5.11. Mutación

- Strings de bits: cambiar cada bit con probabilidad $p \approx 0$.
- Strings de enteros o reales: también se puede hacer algo así, pero en el caso en que haya "fronteras" se hace otra cosa.

  Por ejemplo supongamos que la frontera es:

  $|I_i| \leq 10$ es $\forall i$

  $I = 0, 5|7, 8|5$

  $\widetilde{I} = 0, 5|10|5$

  O bien: $\widetilde{I} = 0, 5 \left|\frac{10+7,8}{2}\right| 5$

7.5.12. Mutaciones con Permutaciones

Mutación para codificaciones basadas en posiciones TSP.

- Permutar dos elementos al azar.
- Elegir uno al azar y cambiarlo con un vecino
- "Inversion Mutation", elijo dos puntos al azar y cambio todos los lugares entre si: GRAFICO

Algunas otras arquitecturas

1. Elitismo (los mejores miembros permanecen). Graf
2. A veces la mutación no se aplica solo a los offsprings.

3. Steady State

4. Migraciones

Inundación: mato a los peores cada tanto, se usa mucho con restricciones muy cortas, cada tanto tengo una subpoblación no válida pero la penalizo, pero es muy buena, luego la mato.

7.6. No Free Lunch

Todos los algoritmos que optimizan[37] en funciones se comportan, promediando sobre todos las funciones posibles, igual.

Referencias

[1] Biggs: "Matematica Discreta"

[2] Roberts, "Applied Combinatorics"

[3] Tarjan: "Data Structures and Network Flows"

[4] BFS: `http://en.wikipedia.org/wiki/Breadth-first_search`

[5] DFS: `http://en.wikipedia.org/wiki/Depth-first_search`

[6] Árboles en Latex:

`http://www.essex.ac.uk/linguistics/external/clmt/latex4ling/trees/pstrees/`

[7] Código C en latex:

`http://stackoverflow.com/questions/586572/make-code-in-latex-look-nice`

[37] Por ejemplo calcular máximo